

POLITECHNIKA POZNAŃSKA
Wydział Automatyki, Robotyki i Elektrotechniki



Praca inżynierska dyplomowa
Bezprzewodowa sieć kontrolno-pomiarowa

Ryszard Napierała, Dawid Dziedzic,
kierunek: Automatyka i robotyka

Promotor:
dr inż. Damian Cetnarowicz

Poznań 2022



Karta pracy dyplomowej inżynierskiej

Uczelnia:	Politechnika Poznańska	Profil studiów:	ogólnoakademicki
Kierunek:	Automatyka i Robotyka	Forma studiów:	niestacjonarne
Studia w zakresie:		Poziom studiów:	I stopnia

Zobowiązuję/zobowiązujemy się samodzielnie wykonać pracę w zakresie wyspecyfikowanym niżej. Wszystkie elementy (m.in. rysunki, tabele, cytaty, programy komputerowe, urządzenia itp.), które zostaną wykorzystane w pracy, a nie będą mojego/naszego autorstwa będą w odpowiedni sposób zaznaczone i będzie podane źródło ich pochodzenia.

Jeżeli w wyniku realizacji pracy zostanie dokonany wynalazek, wzór użytkowy, wzór przemysłowy, znak towarowy, prawa do rozwiązań przysługiwać będą Politechnice Poznańskiej. Prawo to zostanie uregulowane odrębną umową.

Oświadczam, iż o wyniku prac wskazanych powyżej, a także o innych, w tym tych, które mogą być przedmiotem tajemnicy Politechniki Poznańskiej, niezwłocznie powiadomię promotora pracy.

Zobowiązuję się ponadto do zachowania w tajemnicy wszystkich informacji technicznych, technologicznych, organizacyjnych, uzyskanych w Politechnice Poznańskiej w okresie od daty rozpoczęcia realizacji prac do 5 lat od daty zakończenia wykonania prac.

	Imię i nazwisko	Nr albumu	Data i podpis
Student:	Dawid Dziedzic	136110	
Student:	Ryszard Napierała	142303	
Student:			
Student:			

Tytuł pracy:	Bezprzewodowa sieć kontrolno-pomiarowa
Wersja angielska tytułu:	Wireless control and measurement network
Dane wejściowe:	Wiadomości z przedmiotów: teoria i przetwarzanie sygnałów, podstawy elektroniki, technika cyfrowa, systemy mikroprocesorowe, wprowadzenie do sztucznej inteligencji, wprowadzenie do przetwarzania obrazów, podstawy informatyki, programowanie strukturalne i obiektowe. Dokumentacja platformy sprzętowej wybranej do realizacji zadania.
Zakres pracy:	Omówić koncepcję systemu bezprzewodowej sieci sensorycznej (protokoły transmisji danych, zagadnienie synchronizacji danych). Zaprojektować system bezprzewodowej akwizycji pomiarów od wybranych czujników (należy rozważyć mikrofony akustyczne oraz czujniki bezwładnościowe IMU). Zbudować prototypowy system w oparciu o wybraną platformę sprzętową i środowiska programistyczne. Opracować oprogramowanie dla elementów składowych systemu, do transmisji i gromadzenia danych. Wykonać eksperymenty potwierdzające działanie systemu.
Termin oddania pracy:	15.11.2022
Promotor:	dr inż. Damian Cetnarowicz
Jednostka organizacyjna promotora:	Instytut Automatyki i Robotyki, Zakład Układów Elektronicznych i Przetwarzania Sygnałów

podpis dyrektora/kierownika jednostki organizacyjnej promotora

data i podpis Dziekana

Składamy serdeczne podziękowania promotorowi niniejszej pracy, dr. inż. Damianowi Cetnarowiczowi, za możliwość napisania pracy dyplomowej pod Pana opieką, a także za poświęcony nam czas podczas tworzenia pracy.

Spis treści

Streszczenie	11
Abstract	12
Wstęp	13
1 Cel i zakres pracy	14
1.1 Podział pracy	14
1.2 Założenia projektowe	14
1.3 Analiza literatury	15
1.3.1 Komunikacja bezprzewodowa	15
1.3.2 Synchronizacja węzłów w WSN	16
1.3.3 Zastosowania WSN	16
1.4 Pojęcia dotyczące sieci bezprzewodowych	17
2 Elementy projektu	18
2.1 Struktura projektu	18
2.2 Użyte moduły węzłów	18
2.3 Użyte czujniki	20
2.4 Użyte elementy sieciowe	20
2.5 Użyte oprogramowanie i języki programowania	21
2.5.1 Język oprogramowania na mikrokontrolerach	21
2.5.2 freeRTOS	22
2.5.3 Zintegrowane środowisko programistyczne - węzły	23
2.5.4 Język oprogramowania aplikacji serwerowej	25
2.5.5 Zintegrowane środowiska programistyczne - aplikacja serwerowa	25
2.5.6 Jednolite środowisko developerskie - Docker	26
2.5.7 System kontroli wersji - Git	26
2.5.8 Serwer NTP	27
2.5.9 Narzędzie do analizy poprawności zebranych danych	27
3 Konstrukcja sieci	28
3.1 Topologia sieci	28
3.2 Standard sieci bezprzewodowej	28
3.3 Adresy i DHCP	29

4	Przepływ informacji w sieci	32
4.1	Przyjęte kody żądań i odpowiedzi	32
4.2	Nawiązywanie połączenia	32
4.2.1	Handshake	33
4.3	Komunikacja po nawiązaniu połączenia	35
4.3.1	Wysyłanie pakietów danych	35
4.3.2	Wysyłanie poleceń do węzła	36
4.4	Bezpieczeństwo	36
4.4.1	Bezpieczeństwo danych	36
4.4.2	Odporność na chwilowe zerwanie połączenia	37
5	Aplikacja serwerowa	38
5.1	Architektura aplikacji	38
5.1.1	Interakcja z węzłami	39
5.1.2	Interakcja z użytkownikiem	40
5.1.3	Przepływ informacji i zapis danych	41
5.1.4	Przekazywanie poleceń od użytkownika do węzła	43
5.2	Wydajność i ograniczenia	44
5.2.1	Wydajność węzła i specyfikacja danych wejściowych	44
5.2.2	Wydajność routera	45
5.2.3	Wydajność karty sieciowej serwera	45
5.2.4	Wydajność aplikacji serwerowej	45
5.3	Interfejs webowy	46
5.3.1	REST API	46
5.3.2	Interfejs HTML i JavaScript	46
6	Aplikacja węzłów	48
6.1	Architektura aplikacji	48
6.2	Użyte biblioteki	50
6.3	Schematy działania aplikacji	51
6.4	Debugowanie	55
6.4.1	Separacja kodu debugowego od kodu produkcyjnego	55
6.4.2	Debugowanie z użyciem Pythona	57
6.4.3	Debugowanie z użyciem Wiresharka	59
6.5	Modułość i skalowalność	61
7	Testy systemu pomiarowego	64
7.1	Proces zbierania danych	64
7.2	Przygotowania do pomiarów	65
7.2.1	Konfiguracja i kompilacja aplikacji węzłów	65
7.2.2	Konfiguracja i kompilacja aplikacji serwera	68
7.3	Uruchomienie środowiska do analizy danych	71
7.3.1	Konfiguracja	72
7.3.2	Uruchomienie	72
7.4	Analiza danych	72

7.4.1	Skrypty do wstępnego przetwarzania danych	72
7.4.2	Wstępne przetworzenie danych	74
7.4.3	Przetworzenie danych	77
7.4.4	Wnioski	80
8	Podsumowanie	81
	Literatura	82
A	Kod źródłowy aplikacji węzłów	86
A.1	main.cpp	86
A.2	globals.cpp	92
A.3	globals.hpp	93
A.4	link.cpp	95
A.5	link.hpp	100
A.6	mpu.cpp	101
A.7	mpu.hpp	107
A.8	mike.cpp	110
A.9	mike.hpp	113
A.10	platformio.ini	115
A.11	test_tool.py	115
B	Wybrane fragmenty kodu źródłowego aplikacji serwera	124
B.1	exchange.go	124
B.2	handshake.go	125
B.3	interpreter.go	126
B.4	receiver.go	131
B.5	command.go	133
B.6	readings.go	136
B.7	main.go	141
B.8	listen.go	143
C	Skrypty użyte podczas testów systemu	149
C.1	splitter.py	149
C.2	union.py	150
C.3	printer.py	150
C.4	cut_by_time.py	152
C.5	time_cut_receipt.py	153
C.6	final.py	154
C.7	wav_creator.py	157
C.8	mic_synchro.py	157
C.9	roll_synchro.py	159

Spis rysunków

2.1	M5StickC PLUS - rysunek poglądowy	19
2.2	M5StickC PLUS - złącze 2.54mm	19
2.3	M5StickC PLUS - schemat elektroniczny	20
2.4	Router TpLink Archer C6 v3	21
2.5	Logo języka C++	22
2.6	Logo języka Python	22
2.7	Logo freeRTOS	22
2.8	Przykład użycia FreeRTOS do wywołania funkcji jako autonomicznego wątku. xTaskCreatePinnedToCore [26] uruchamia funkcję "mike_task" jako wątek pomiarowy mikrofonu A.8 na rdzeniu "pro_cpu".	23
2.9	Logo vscode	24
2.10	Logo PlatformIO	24
2.11	Dodawanie PlatformIO do Visual Studio Code. a) Należy otwożyć wyszukiwarkę dodatków. b) Następnie wyszukać "PlatformIO". c) Na końcu kliknąć przycisk instalacji.	24
2.12	Otwieranie wybranego folderu w Visual Studio Code. a) Należy kliknąć "File" w głównym menu aplikacji. b) Następnie kliknąć "Open Folder" i wybrać folder.	25
2.13	Logo języka Go	25
2.14	Logo środowiska Golang	26
2.15	Logo technologii Docker	26
2.16	Logo systemu kontroli wersji Git	26
2.17	Logo oprogramowania serwera NTP - Chrony	27
2.18	Logo Pandas	27
2.19	Logo Matplotlib	27
3.1	Topologia gwiazdy	28
3.2	Uproszczony schemat sieci	29
3.3	Logo technologii Wi-Fi	29
3.4	Ustawienia DHCP w routerze TpLink Archer C6	30
3.5	Ustawienia sieciowe na serwerze	30
4.1	Pomyślne nawiązanie połączenia	34
4.2	Niepowodzenie w nawiązywaniu połączenia	34
5.1	Schemat architektury aplikacji serwerowej	38

5.2	Schemat wątków obsługi węzłów	39
5.3	Logo technologii HTML	40
5.4	Logo frameworka Gin	40
5.5	Logo technologii CSS	41
5.6	Logo frameworka Vue	41
5.7	Schemat części aplikacji obsługującej serwer HTTP	41
5.8	Schemat przepływu informacji zapisywanych do bazy danych	42
5.9	Ogólny schemat wzorca obserwator w graficznej reprezentacji notacji UML	43
5.10	Schemat działania szyny danych	44
5.11	Interfejs webowy, panel kontrolny systemu	47
5.12	Schemat przedstawiający działanie skryptu w interfejsie webowym . . .	47
6.1	Struktura Msg	49
6.2	Architektura aplikacji węzłów	50
6.3	Inicjalizacja aplikacji - Dodatek A.1	51
6.4	Główny wątek aplikacji - Dodatek A.1	51
6.5	Wątek led_task - A.1	52
6.6	Wątek send_task - A.1	52
6.7	Funkcja przetwarzająca komendy - A.1	53
6.8	Funkcja startująca wątek pomiarowy - A.6 i A.8	53
6.9	Funkcja kończąca wątek pomiarowy - A.6 i A.8	54
6.10	Wątek pomiarowy - A.6 i A.8	54
6.11	Handshake - A.4	55
6.12	Przepływ danych	55
6.13	Makro debugowe.	56
6.14	Sposób użycia makra debugowego. Tekst zostanie wydrukowany tylko wtedy gdy zmienna DEBUG ma wartość 1.	56
6.15	Logi debugowe widoczne poprzez REPL	57
6.16	Przycisk uruchamiający REPL. Widoczny na dolnym pasku narzędzi w Visual Studio Code.	57
6.17	Na górze - prezentacja pomiarów z mikrofonu, żyroskopu, oraz akcelero- metru. Na dole - logi debugowe.	59
6.18	Logo aplikacji Wireshark	60
6.19	Ramka danych pomiarowych z IMU widoczna w aplikacji Wireshark. Aby uzyskać logi należy uruchomić aplikację, wybrać sieć w której pracują czujniki i kliknąć interesujący nas pakiet. Dodatkowo możemy przefil- trować pakiety, np po adresie IP, wpisując w polu filtra (zielony pasek) "ip.address == adres IP serwera, lub czujnika".	60
6.20	Debugowanie błędów związanych z wydajnością systemu we wczesnych etapach projektu, za pomocą aplikacji Wireshark.	61
6.21	Część logu kompilacji mówiąca o zajętości pamięci.	61
6.22	Rejestracja funkcji uruchamiających wątki pomiarowe. A.1	62
6.23	Rejestracja funkcji kończących wątki pomiarowe. A.1	62
6.24	Rejestracja funkcji ustawiających parametry modułów pomiarowych. A.1	63

7.1	Urządzenia świecą światłem ciągłym, sygnalizując, że połączenie z serwerem nie zostało jeszcze nawiązane.	64
7.2	Pomiar dźwięku klaśnięcia	65
7.3	Pomiar prędkości kątowej podczas toczenia rolki.	65
7.4	Przycisk uruchamiający REPL	66
7.5	Log kompilacji aplikacji węzłów	67
7.6	Przycisk zasilania węzła	68
7.7	Pomyślne uruchomienie aplikacji serwera	70
7.8	Raport z działania aplikacji serwera, odczytany podczas zbierania odczytów z czujników.	71
7.9	Lista używanych obecnie klas ustawień.	73
7.10	Klasy typu T.	73
7.11	Wizualizacja danych z mikrofonu. Każdy kolor oznacza inny mikrofon.	75
7.12	Zbliżenie na serię klaśnięć z pomiarów mikrofonu.	76
7.13	Odczyty DPS osi Y w doświadczeniu z rolką.	76
7.14	Zbliżenie na zbocze z niepełnymi danymi dla czujnika 0	78

Spis tabel

3.1	Lista adresów składających się na sieć pomiarową	31
4.1	Lista kodów żądań	32
4.2	Lista kodów odpowiedzi i błędów	33
4.3	Struktura wiadomości handshake wysyłanej przez węzeł	33
4.4	Struktura odpowiedzi handshake wysyłanej przez serwer	35
4.5	Struktura nagłówka paczki danych	36
4.6	Kody czujników w komunikacji	36
4.7	Struktura pakietu polecenia	36
5.1	Specyfikacja maszyny, użytej do testów wydajności i ograniczeń	45
5.2	Zestawienie zużycia zasobów przez aplikację działającą w pełnej wersji	46
7.1	Zestawienie komend wykonujących skrypty	73
7.2	Zestawienie wyników pomiarów stabilności czasowej. Częstotliwości próbkowania zostały wyliczone na podstawie średnich różnic czasu pomiędzy próbkami.	77
7.3	Zestawienie czasów zarejestrowania pierwszych próbek rejestrujących impuls	78
7.4	Zestawienie czasów wejścia czujników DPS w stan nasycenia	80

Streszczenie

Celem pracy jest zaprojektowanie i zbudowanie bezprzewodowej sieci kontrolno-pomiarowej, komunikującej się przez Wi-Fi ze szczególnym uwzględnieniem synchronizacji odczytów z czujników.

Dokonano analizy literatury, w tym przeglądu dotychczas istniejących sposobów komunikacji bezprzewodowej w bezprzewodowych sieciach czujników, podano zestaw zastosowań dla takich sieci oraz sporządzono przegląd technik synchronizacji odczytów informacji z czujników.

Zdefiniowano założenia systemu do samodzielnej konstrukcji, podano terminy obowiązujące w niniejszej pracy, zaprezentowano wykaz oprogramowania i urządzeń używanych w projekcie oraz zaprezentowano ogólny zarys infrastruktury sieci komputerowej. Zaprezentowano planowany przepływ informacji w sieci, kody i strukturę komunikatów. W opisanych warunkach oraz zgodnie z przedstawionym przepływem informacji i schematem działania zaprojektowano aplikację serwerową z użyciem języka Go oraz oprogramowanie przeznaczone na węzły w języku C++ i zaprezentowano ich działanie poprzez szczegółowe omówienie poszczególnych elementów.

Dokonano analizy poprawności pomiarów, zebranych z systemu składającego się z czterech połączonych do sieci węzłów, z których każdy jest wyposażony w identyczny zestaw czujników oraz jednego serwera. Zauważono, że system jest w stanie przetworzyć i zapisać przekazywane przez te czujniki dane. Ostatecznie ustalono, że system działa z precyzją rzędu pojedynczych milisekund. Jest to spodziewany efekt, spowodowany zastosowaną technologią IP, która nie została zaprojektowana z myślą o synchronizacji bardziej precyzyjnej.

Abstract

The target of this paper is to design and build wireless control and measurement network, which can communicate through Wi-Fi, with special concern for sensor data synchronization.

The analysis of the available literature was done, and with it an overview of already existing ways of communicating wirelessly in wireless sensor networks, a set of usages for wireless sensor networks has been given and an overview of techniques, allowing sensor in such networks to synchronize their data.

Presumptions to the system for self-construction has been given, the terms appearing in paper explained and lists of software, devices and tools has been written down. General network infrastructure has been presented, as well as schematics of information flow, codes and structures of communicates present in communication between sensors and a server. In conditions given, a server application has been written with use of Go language and a node application, written in C++. Their usage and their operation was presented, by detailed explanation of each element.

Analysis of the correctness of the measurements has been made. There were four nodes connected simultaneously to the network, each of them equipped with identical set of sensors. It has been noticed that the system is capable of processing and saving the data generated by the nodes. Finally, the conclusion has been drawn that the designed system works and synchronizes data between sensor with the precision of single milliseconds. It's an expected result and is caused by the used IP technology, which was never designed to reach greater time synchronization precision.

Wprowadzenie

W rozbudowanych systemach pomiarowych, składających się z wielu czujników, dodatkowy koszt projektu stanowi sposób połączenia składników tego systemu. Połączenia przewodowe są trudne do utrzymania, ze względu na swoją naturę: każdy z przewodów zajmuje miejsce, wymaga możliwości identyfikacji w celu redukcji kosztów utrzymania systemu i stanowi koszt sam w sobie, ze względu na potrzebę zakupu materiału z którego jest zbudowany oraz jego wymiany w przypadku uszkodzenia przewodu. W przypadku wystąpienia wady przewodu, lokalizowanie miejsca usterki jest problematyczne i często niemożliwe bez odpowiednich narzędzi, bo uszkodzenie kryje się gdzieś pod izolacją. O ile są to akceptowalne kompromisy w przypadku rozwiązań o niewielkiej liczbie czujników bądź w przypadku systemów kluczowych, w których koszt nie gra najważniejszej roli, to jeżeli jednym z głównych założeń jest optymalizacja kosztów, technologia bezprzewodowa będzie lepszym wyborem.

Bezprzewodowa sieć czujników (ang. WSN - Wireless Sensor Network) to sieć przestrzennie rozproszonych, wyspecjalizowanych czujników, monitorujących fizyczne warunki występujące w środowisku. Ich rozwój został zapoczątkowany przez potrzeby militarne, takie jak wywiad i nadzór pola walk. Topologia sieci czujników może być różna, poczynając od sieci gwiazdowej, w której wszystkie czujniki są połączone z centralnym serwerem zbierającym odczyty aż po topologię mesh, w której czujniki są połączone ze sobą nawzajem i przesyłają za swoim wzajemnym pośrednictwem informacje, które ostatecznie trafiają na serwer zbierający dane. [21]. W przypadku rozpatrywanym w tej pracy używamy WSN do tego, aby zoptymalizować koszty budowy i utrzymania sieci czujników. Tak zbudowana sieć może zostać zintegrowana z dowolnym rodzajem czujników i wykorzystana na wiele sposobów, na przykład do macierzy mikrofonów, która pozwoli określić kierunek nadejścia dźwięku albo do IMU (Inertial Measurement Unit) [58], które pozwoli precyzyjnie mapować przemieszczenia, prędkości i przyspieszenia obiektów fizycznych, do którego czujniki zostaną przytwierdzone.

Problemem w przypadku sieci bezprzewodowych jest natomiast synchronizacja odczytów pomiędzy tymi czujnikami. W przypadku sieci przewodowych odczyty z czujników są synchronizowane z innymi węzłami i serwerem w sieci niemal natychmiastowo. Jeżeli kluczowym aspektem systemu jest wysoka precyzja pozycji odczytów ze wszystkich czujników na osi czasu, to występuje potrzeba programowego skompensowania opóźnienia spowodowanego przez komunikację bezprzewodową.

Rozdział 1

Cel i zakres pracy

Celem pracy jest opracowanie sieci kontrolno-pomiarowej, której głównym zadaniem jest optymalizacja kosztów utworzenia oraz utrzymania, a także kompensacja opóźnień wynikającego z zastosowania komunikacji bezprzewodowej zamiast przewodowej. Kwestia synchronizacji odczytów jest kluczowa w projekcie - autorzy starają się osiągnąć jak największą precyzję synchronizacji odczytów z czujników. Sieć opiera się na istnieniu jednego serwera, którego rolą jest zapis i przechowywanie danych oraz udostępnianie ich w wybranej formie do dalszego użytku. Serwer nie zajmuje się przetwarzaniem danych. Komunikacja pomiędzy węzłami i serwerem nie jest w żaden sposób zaszyfrowana, gdyż nie stanowi to konieczności i wprowadza zbędną złożoność, stąd zakłada się, że sieć nie będzie używana w miejscach, gdzie bezpieczeństwo przesyłania danych pomiarowych jest kluczowe. Sieć jest być możliwa do utworzenia nawet na najprostszej, domowej infrastrukturze sieciowej, to znaczy za pomocą jednego komputera, domowego routera oraz zestawu węzłów wyposażonych w czujniki.

1.1 Podział pracy

Praca została wykonana w zespole dwuosobowym. Za oprogramowanie serwera, infrastrukturę sieci i dodatkowe elementy obecne w infrastrukturze sieci odpowiedzialny jest Dawid Dziedzic. Natomiast za oprogramowanie węzłów, wybór czujników, oraz narzędzie do analizy danych odpowiedzialny jest Ryszard Napierała. Struktura wiadomości przesyłanych pomiędzy urządzeniami była opracowywana z udziałem obu stron i na bieżąco dostosowywana do potrzeb.

1.2 Założenia projektowe

Projekt powstał w oparciu o założenie, że sieć może działać bez dostępu do internetu. Oznacza to, że router nie jest niezbędnym urządzeniem w infrastrukturze sieciowej i może zostać z powodzeniem zastąpiony switchem i access pointem. Serwer powinien być w stanie przechować dane pobrane z czujnika, pracować z uruchomionym oprogramowaniem serwerowym napisanym w ramach pracy oraz oprogramowaniem NTP. Dane zebrane przez aplikację będą zapisywane do plików w formacie CSV. Format CSV

to popularny format, służący do przedstawienia danych w formacie tabelarycznym z użyciem separatorów wartości [66]. Pliki zapisane zostaną do systemu plików serwera, skąd będą pobierane do narzędzia do analizy danych. Bezpieczeństwo danych przed wykradnięciem nie jest brane pod uwagę i komunikacja pozostaje niezaszyfrowana. Zakłada się brak konieczności ręcznego ustawiania adresów IP węzłów. Adresy IP to unikatowe identyfikatory urządzeń, połączonych do sieci komputerowej w sieci komputerowej. Manualne ustawianie adresów IP nie będzie konieczne ze względu na obecność w sieci serwera DHCP, czyli oprogramowania, zajmującego się dynamicznym przydzielaniem adresów IP do urządzeń w sieci.

Węzeł ma możliwość podłączenia do sieci bezprzewodowej. Węzeł pomiarowy komunikuje się z serwerem za pomocą TCP, czyli protokołu warstwy transportowej, gwarantującego niezawodność w przesyłaniu datagramów pomiędzy urządzeniami sieciowymi. W celu synchronizacji czasu z serwerem wykorzystany został protokół NTP, czyli protokół synchronizacji czasu [22], oraz wewnętrzny zegar RTC w mikrokontrolerze. Węzły są urządzeniami modułowymi zarówno pod względem sprzętowym jak i programowym aby umożliwić ich łatwe rozbudowywanie o kolejne czujniki lub funkcjonalności. Zadbano również o odpowiednią separację głównego kodu aplikacji węzłów od kodu obsługującego poszczególne czujniki. Węzły są sterowane za pomocą panelu sterowania otwieranego w przeglądarce internetowej. Sygnalizują swój stan za pomocą LED.

1.3 Analiza literatury

Prace nad bezprzewodowymi sieciami czujnikowymi (dalej: WSN) zostały rozpoczęte około 1980 roku. Powstawały one na bazie protoplasty dzisiejszego internetu - ARPANETu. [5] Dzisiaj, do WSN używa się rozmaitych technologii komunikacyjnych, oprogramowania oraz urządzeń, które obsługuje się za pomocą różnorodnych interfejsów.

1.3.1 Komunikacja bezprzewodowa

W celu komunikacji w WSN, korzysta się z rozmaitych technologii komunikacji bezprzewodowej: ZigBee, Bluetooth, Wi-Fi, czy w przypadku większych dystansów w przestrzeni cywilnej: sieci komórkowej, w tym technologii LTE, 5G i innych bezprzewodowych środków dostępu do internetu. [65] Pomimo, że istnieją protokoły komunikacyjne lepiej przystosowane do niskoenergetycznej transmisji danych, to w przypadku projektu, będącego tematem niniejszej pracy została wykorzystana technologia Wi-Fi [64]. Technologia Wi-Fi, to popularny standard, wykorzystywany w lokalnych sieciach komputerowych do komunikacji bezprzewodowej. Jej popularność obniża koszty rozwoju projektu na niej opartego. [8]. Jednocześnie, niniejszy projekt jest oparty o technologię IP, co sprawia że jest niezależny od standardu sieci bezprzewodowej i będzie działał z każdym innym tak długo, jak będzie on możliwy do zastosowania w sieciach komputerowych, komunikujących się za pomocą protokołu internetowego. [2] Technologia IP jest dobrym wyborem w przypadku WSN, w których nie występuje duża ilość węzłów i/lub nie jest niezbędna wysoka niezawodność. W sytuacjach, w których węzłów jest wiele a niezawodność jest kluczowa, odrzuca się technologię IP jako zbyt powolną i

energochłonną, ze względu na wysokie koszty routingu pakietów w takiej sieci i zastępuję się ją technologią specjalnie przystosowaną do WSN, pozbawioną wysokich kosztów routingu, tworzenia i przesyłania pakietów danych. Jednakże, często pożądane jest, aby dostęp do wyników pomiarów z WSN był możliwy przez sieci IP. Najpopularniejszym sposobem radzenia sobie z tym problemem jest ustawienie serwera proxy, działającego po części w ramach WSN w celu zebrania danych, a po części w sieci opartej o IP w celu ich udostępniania. [9] [10]

Topologia WSN może przybierać różne formy, w zależności od potrzeb i użytej technologii. W przypadku technologii IP, ze względu na konstrukcję sieci na niej opartych będzie to najczęściej topologia gwiazdy, jednak możliwe jest użycie również innych topologii. Topologia siatki gwarantuje dużą niezawodność, ponieważ likwiduje pojedynczy punkt błędu, którego awaria powoduje niesprawność całego systemu. [11]

1.3.2 Synchronizacja węzłów w WSN

Zaproponowano wiele rozwiązań synchronizacji węzłów w WSN. Jednym z nich jest najbardziej oczywisty - masowe rozsyłanie danych na temat obecnego czasu w WSN. Osiąga się to za pomocą protokołu nazwanego FTSP. FTSP przewiduje, że węzeł będzie wysyłał informacje o czasie do innych węzłów w WSN w określonych interwałach czasowych. Ma to swoje wady, wśród których wymienia się obniżoną dokładność i skalowalność. W literaturze proponuje się sposoby na zmniejszenie tego efektu. [6]

Proponowane są również kolejne protokoły, mające zupełnie inny charakter u podstaw. Jednym z nich jest synchronizacja wewnętrznych zegarów węzłów, oparta na systematycznym wysyłaniu i odbieraniu informacji zwrotnej, ze szczególną uwagą poświęconą wykorzystaniu mocy obliczeniowej i pamięci węzłów. [7].

1.3.3 Zastosowania WSN

Współcześnie WSN wykorzystuje się w celach zarówno militarnych, dla spełniania których zostały pierwotnie skonstruowane jak i w celach cywilnych, a w tym medycznych, badawczych, infrastrukturalnych. [12]

W wojsku, które było pierwszym powodem, dla którego badania i prace nad WSN rozpoczęły się, dzisiaj ta technologia znajduje wiele rozmaitych zastosowań. Jest używana do obserwowania ruchów wroga na polu bitwy, wykrywania intruzów oraz ogólnie pojętego zwiadu. Jest używana również do ochrony obiektów i ludzi o szczególnym znaczeniu, pomaga w wykrywaniu obecności i użycia broni radiologicznej, biologicznej, chemicznej i konwencjonalnej wśród osób nieupoważnionych do jej posiadania czy użycia. [13]

W ochronie zdrowia WSN są wykorzystywane do monitorowania stanu zdrowia pacjentów na pewnym obszarze, na przykład w szpitalu. Czujniki przytwierdzone do ciała pacjenta potrafią z wyprzedzeniem zaalarmować personel szpitala o potencjalnym zagrożeniu dla zdrowia i życia pacjenta. [14]

W inżynierii środowiska WSN wykorzystywane są do monitorowania jego stanu. Wykorzystuje się je do monitorowania stanu wody i powietrza, zbierania danych

statystycznych na ich temat oraz alarmowania obszarowego w przypadku, gdy wartości odbiegają od normy. Pozwala to na wczesne ostrzeganie przed zmianami pogodowymi, zanieczyszczeniami, czy klęskami żywiołowymi takimi jak trzęsienia ziemi i tsunami. [15]

1.4 Pojęcia dotyczące sieci bezprzewodowych

W pracy występują określenia powszechnie znane w zagadnieniu WSN, których definicję autorzy uznali za zasadne przytoczyć. Podawanie tych definicji ma na celu ograniczenie możliwości nieporozumienia, wynikającego z różnych definicji pojęć krążących w literaturze oraz internecie a także z braku powszechnie przyjętych, polskich odpowiedników dla niektórych pojęć. Przytoczone definicje zawierają również swoje anglojęzyczne określenia.

Węzeł (node) stanowi określenie dla pojedynczego urządzenia, będącego autonomiczną jednostką. Zawiera: czujniki, mikrokontroler, moduł komunikacji radiowej oraz własne źródło zasilania. Każdy węzeł może się łączyć z serwerem. [21]

Czujnik (sensor) to urządzenie, które zbiera dane ze swojego otoczenia i przetwarza je na sygnały, które mogą być interpretowane przez inne urządzenia, np. węzły. [21]

Serwer (server) to urządzenie centralne, którego zadaniem jest zbieranie danych i ich dalsze przetwarzanie. [21]

NTP (Network Time Protocol) jest protokołem sieciowym służącym do synchronizacji zegara między systemami komputerowymi w sieciach danych z przełączaniem pakietów o zmiennym opóźnieniu. [22]

REPL (Read-eval-print loop) proste, interaktywne środowisko programowania. Jest to wiersz poleceń powłoki. Dzięki REPL użytkownik może wprowadzać polecenia i otrzymywać wyniki bezpośrednio, bez konieczności kompilacji i uruchamiania programu. [31]

.csv (Comma-separated values) pliki tekstowe, które zawierają dane oddzielone przecinkami. Często zawierają w pierwszej linii nazwy kolumn. Są używane do przechowywania danych w formie tabel. [27]

Rozdział 2

Elementy projektu

2.1 Struktura projektu

Projekt składa się z pięciu części. Każdej z nich odpowiada inny folder. W dalszej części pracy autorzy będą się odwoływać do tych folderów w relacji do głównego folderu projektu.

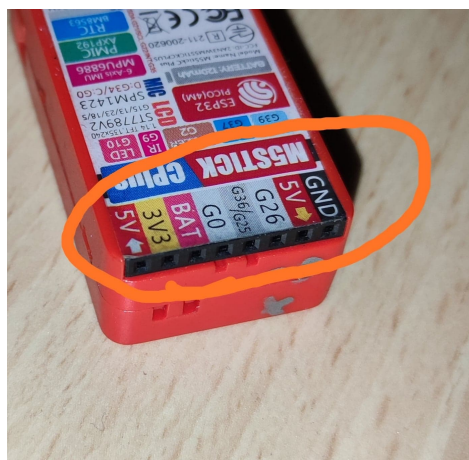
- **doc** - folder zawierający źródło projektu w formacie LaTeX.
- **server** - folder zawierający kod źródłowy programu serwera.
- **M5StickC/device** - folder zawierający kod źródłowy aplikacji węzłów.
- **file_decoder** - folder zawierający kod źródłowy programu, oraz sam program dekodujący dane do pliku .csv.
- **data_tests** - folder zawierający skrypty użyte podczas analizy danych.

2.2 Użyte moduły węzłów

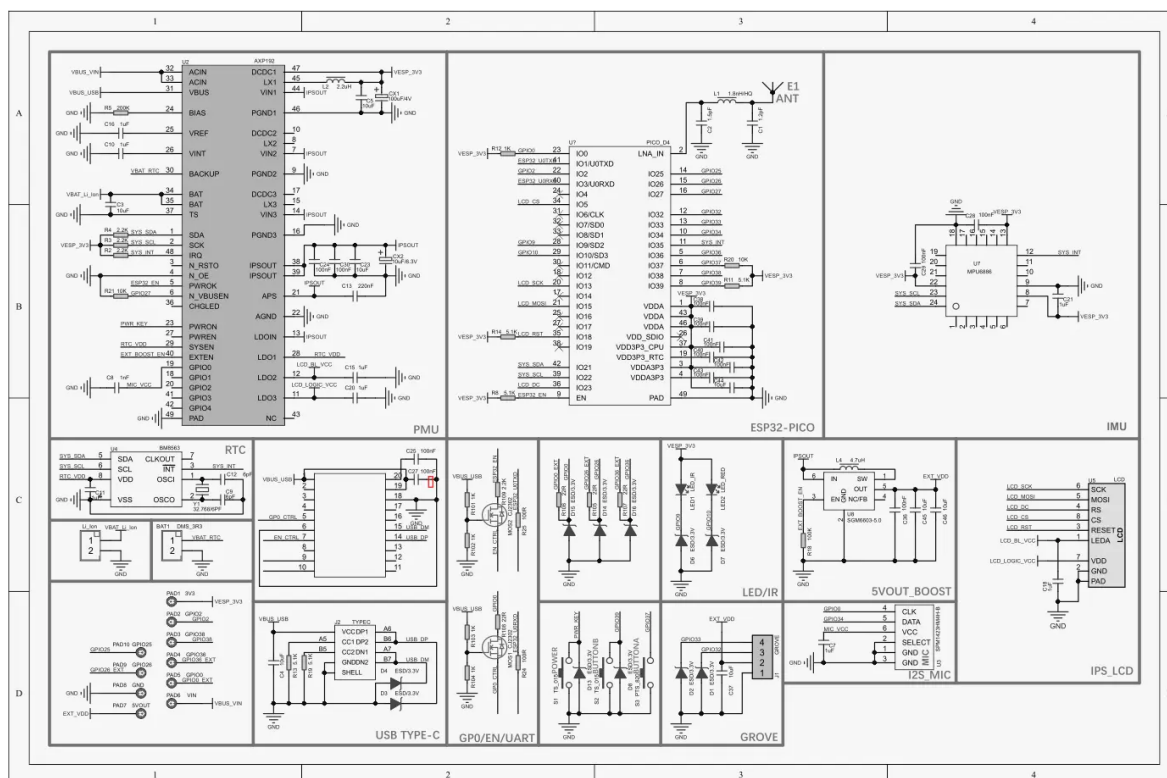
Jako węzłów użyto modułów deweloperskich M5StickC PLUS [23]. Są to moduły zbudowane na bazie mikrokontrolerów ESP32-PICO [24]. W obudowie zawarto kilka czujników oraz RTC, LCD, LED, IR, przyciski i moduły komunikacji radiowej 2.1. Dodatkowo są wyposażone w złącze USB-C, służące do ładowania baterii oraz programowania mikrokontrolera. W obudowie znajduje się również miejsce na baterię LiPo 120 mAh, 3.7V wraz z układem ładowania. Przewidziane zostało standardowe złącze 2.54mm 2.2 do podłączenia dodatkowych czujników. W tym celu do dyspozycji mamy takie protokoły jak I2C, SPI, UART, GPIO, ADC, I2S, IR, 1-Wire czy CAN.



Rysunek 2.1: M5StickC PLUS - rysunek poglądowy



Rysunek 2.2: M5StickC PLUS - złącze 2.54mm



Rysunek 2.3: M5StickC PLUS - schemat elektroniczny

2.3 Użyte czujniki

Do prezentacji działania skonstruowanej WSN wykorzystano czujniki:

- **MPU6886** - Jest to urządzenie które zawiera w obudowie 3-osiowy żyroskop oraz 3-osiowy akcelerometr. Komunikacja z mikrokontrolerem przebiega za pomocą protokołu I2C. [42]
- **SPM1423** - Układ scalony zawierający mikrofon cyfrowy. Komunikacja z mikrokontrolerem przebiega za pomocą protokołu I2S. [43]

2.4 Użyte elementy sieciowe

Jako serwer DHCP, przełącznik i punkt dostępowy użyto routera TpLink Archer C6 v3.20, przedstawiony na rys. 2.4. Każdy współczesny router domowy jest w stanie spełnić tę rolę, dlatego został wybrany model, który był dostępny.



Rysunek 2.4: Router TpLink Archer C6 v3

2.5 Użyte oprogramowanie i języki programowania

Języki oprogramowania zostały dobrane tak, aby pozwoliły osiągnąć zadowalającą szybkość i efektywność pamięciową i obliczeniową. Tam, gdzie wybór był szeroki, preferowane były języki programowania, w których autorzy pracy posiadają doświadczenie. Dodatkowym atutem była również szeroka społeczność użytkowników danego języka, co zwiększało pewność, że w razie napotkania ciężkich do samodzielnego rozwiązania problemów autorzy pracy będą w stanie znaleźć odpowiedzi poprzez odwołanie się do tych społeczności. Podobne zasady przyświecały autorom pracy przy wyborze oprogramowania, którego użyto do rozwijania projektu, budowania środowiska projektowego bądź jako części uzupełniające projekt w jego gotowej postaci.

2.5.1 Język oprogramowania na mikrokontrolerach

Językiem programowania użytym do implementacji oprogramowania na mikrokontrolerach jest język C++. Jego wybór spowodowany został głównie potrzebą napisania oprogramowania w języku o wysokiej wydajności, który pozwoli na wykorzystanie możliwości mikrokontrolera ESP32-PICO. C++ to język programowania ogólnego przeznaczenia stworzony przez duńskiego informatyka Bjarne Stroustrupa jako rozszerzenie języka programowania C lub „C z klasami”. Z biegiem czasu język znacznie się rozwinął, a współczesny C++ ma teraz funkcje zorientowane obiektowo, ogólne i funkcjonalne, oprócz udogodnień do manipulacji pamięcią niskiego poziomu.

W celu zwiększenia wydajności i osiągnięcia większej izolacji elementów oprogramowania wykorzystano framework freeRTOS [25]. Jest to framework pozwalający na tworzenie aplikacji wielowątkowych, które mogą być wykonywane na różnych procesorach i kolejgowane na bazie nadanych priorytetów.

W celach testowania oprogramowania podczas jego tworzenia wykorzystano narzędzie testowe, które zostało napisane w języku Python[30] w wersji 3.9.14 A.11. Jego głównym zadaniem było symulowanie działania serwera. Było to istotne ze względu na to że serwer powstawał równolegle do oprogramowania na mikrokontrolerach i było to jedynym sposobem na testowanie komunikacji między mikrokontrolerami a serwerem.



Rysunek 2.5: Logo języka C++



Rysunek 2.6: Logo języka Python

2.5.2 freeRTOS

Opracowany we współpracy z wiodącymi światowymi producentami układów scalonych przez 18 lat. FreeRTOS to wiodący na rynku system operacyjny czasu rzeczywistego (RTOS [28]) dla mikrokontrolerów i małych mikroprocesorów. Rozpowszechniany bezpłatnie na licencji MIT open source, FreeRTOS zawiera jądro i rosnący zestaw bibliotek IoT odpowiednich do użytku we wszystkich sektorach przemysłu. FreeRTOS został zbudowany z naciskiem na niezawodność i łatwość użytkowania [25].

FreeRtOS został użyty w celu zwiększenia wydajności aplikacji, dając nam dostęp do wielowątkowości, co pozwoliło na wykonywanie zadań w tle, które nie wymagały natychmiastowej reakcji. Dodatkowo mogliśmy użyć dwóch rdzeni procesora, dzięki czemu komunikacja z programem serwera oraz zbieranie danych z czujników mogło następować równolegle.



Rysunek 2.7: Logo freeRTOS

```
124 bool start_Mike_task(Connection* connection) {  
125     xSemaphoreGive(mutexMike);  
126     xTaskCreatePinnedToCore(mike_task, "Mike", 8192, connection, configMAX_PRIORITIES -1, &taskMike, pro_cpu);  
127     return true;  
128 }  
129
```

Rysunek 2.8: Przykład użycia FreeRTOS do wywołania funkcji jako autonomicznego wątku. `xTaskCreatePinnedToCore` [26] uruchamia funkcję "mike_task" jako wątek pomiarowy mikrofonu A.8 na rdzeniu "pro_cpu".

2.5.3 Zintegrowane środowisko programistyczne - węzły

Jako środowiska programistycznego (IDE - Integrated Development Environment) użytego do tworzenia aplikacji węzłów wybrano Visual Studio Code[39]. Jest to darmowe, otwarte oprogramowanie, które zostało stworzone przez Microsoft. Jest to edytor kodu źródłowego, który oferuje wsparcie dla wielu języków programowania, w tym C++. Jest to jedno z najpopularniejszych środowisk programistycznych. Zostało one użyte w połączeniu z rozszerzeniem PlatformIO[40], które pozwala na tworzenie projektów dla wielu typów mikrokontrolerów.

PlatformIO to wieloplatformowe, wieloarchitekturowe, wielowarstwowe, profesjonalne narzędzie dla inżynierów systemów wbudowanych i programistów, którzy piszą aplikacje dla produktów wbudowanych. [41]

Dużą zaletą tego rozszerzenia jest to, że utworzony plik konfiguracyjny wystarcza do skompilowania projektu na dowolnym komputerze z zainstalowanym rozszerzeniem PlatformIO, bez dodatkowych konfiguracji. A.10

Odpowiednia wersja kompilatora C++, frameworków, czy FreeRTOS'a zostanie pobrana automatycznie na podstawie wyżej wymienionego pliku. Nie podawanie konkretnych wersji w wypadku użytych narzędzi jest celowe i zwiększa uniwersalność kodu, pozwalając IDE zdecydować które wersje kompilatora czy frameworka należy pobrać w zależności od użytego środowiska. Taka praktyka pozwala w sporej części wyeliminować brak kompatybilności pomiędzy środowiskami użytymi przez różnych programistów. Na rysunku 2.11 przedstawiono proces dodawania PlatformIO do Visual Studio Code. IDE skonfiguruje się automatycznie po otwarciu Visual Studio Code w folderze zawierającym plik konfiguracyjny platformio.ini (Folder źródłowy aplikacji węzłów).

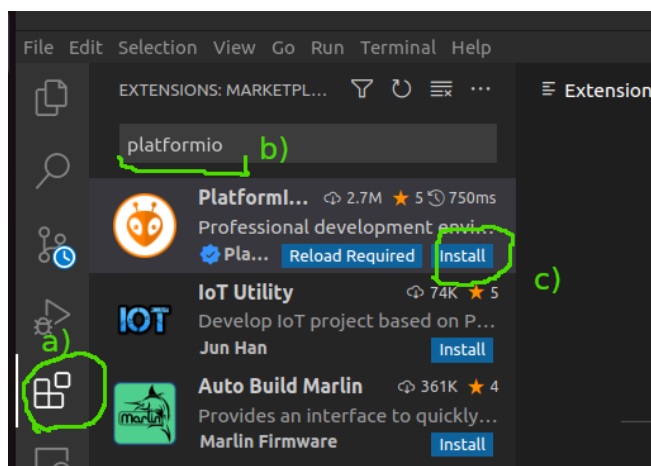
Otwieranie Visual Studio Code w wybranym folderze zostało przedstawione na rysunku 2.12.



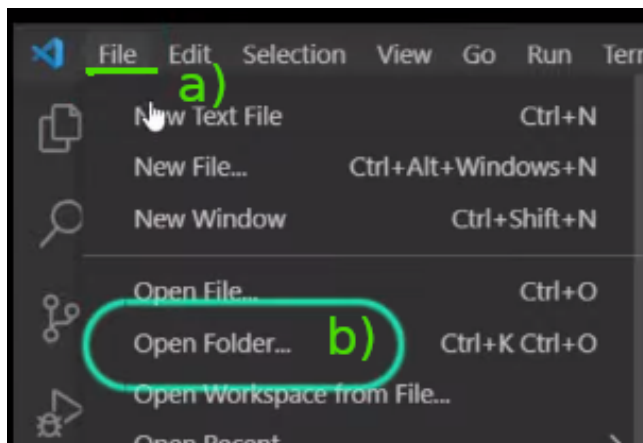
Rysunek 2.9: Logo vscode



Rysunek 2.10: Logo PlatformIO



Rysunek 2.11: Dodawanie PlatformIO do Visual Studio Code. a) Należy otwożyć wyszukiwarkę dodatków. b) Następnie wyszukać "PlatformIO". c) Na końcu kliknąć przycisk instalacji.



Rysunek 2.12: Otwieranie wybranego folderu w Visual Studio Code. a) Należy kliknąć "File" w głównym menu aplikacji. b) Następnie kliknąć "Open Folder" i wybrać folder.

2.5.4 Język oprogramowania aplikacji serwerowej

Głównym językiem programowania w aplikacji serwerowej został język Go v1.18. Jest on używany za pośrednictwem kontenera Dockera, a więc nie wymaga się od użytkownika jego ręcznej instalacji na komputerze. Proces konfiguracji środowiska szczegółowo opisano w podrozdziale 7.2.2. Go to kompilowany język programowania open source, który ułatwia budowanie prostego, niezawodnego i wydajnego oprogramowania. Stworzony we wrześniu 2007 r., przez Roberta Griesmera, Roba Pike'a i Kena Thompsona (Google), stworzony na podobieństwo języka C. Język ten wyposażony jest w zaawansowane mechanizmy w zakresie współbieżności. Sam język jak i społeczność wokół niego są w dużej mierze skupione wokół budowania infrastruktury sieciowej. Do tego język posiada mechanizm garbage collector, który czyni go językiem tanim w produkcji i utrzymaniu i będącym wyjątkowo dobrze dopasowanym do potrzeb projektu [3]. Z racji tych jego właściwości oraz większego doświadczenia w używaniu tego języka, został on wybrany spośród swoich bezpośrednich konkurentów, tj. Python, C, C++, PHP, Java, Rust



Rysunek 2.13: Logo języka Go

2.5.5 Zintegrowane środowiska programistyczne - aplikacja serwerowa

Do rozwijania oprogramowania serwera użyto IDE IntelliJ GoLand, firmy JetBrains. Jest to zaawansowane środowisko programistyczne, które znacznie przyspieszyło tworze-

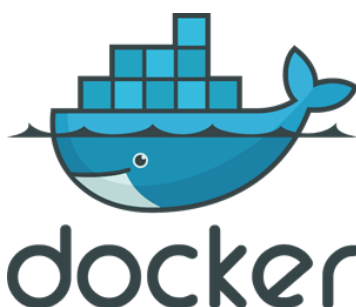
nie oprogramowania. Projekt nie jest jednakże zależny od tego konkretnego środowiska programistycznego - po odpowiednim przygotowaniu dowolnego innego IDE będzie możliwa samodzielna, dalsza praca nad projektem.



Rysunek 2.14: Logo środowiska Golang

2.5.6 Jednolite środowisko developerskie - Docker

Docker to oprogramowanie do konteneryzacji środowisk. Z jego pomocą osoby odpowiedzialne za rozwój oprogramowania na czujniki nie muszą znać szczegółów budowy i kompilacji oprogramowania na serwerze: wystarczy, że uruchomią środowisko przez jedno polecenie. [59] [17]



Rysunek 2.15: Logo technologii Docker

2.5.7 System kontroli wersji - Git

Git to najczęściej wybierany w wytwarzaniu oprogramowania system kontroli wersji. Zapewnia on między innymi wersjonowanie aplikacji, czy możliwość tworzenia osobnych gałęzi, na których mogą być prowadzone prace bez wpływu na stabilność działania wersji programu na gałęzi głównej. [60] [18] [19]



Rysunek 2.16: Logo systemu kontroli wersji Git

2.5.8 Serwer NTP

Działający na serwerze NTP jest niezbędny dla poprawnego działania projektu, jednak nie było potrzeby implementować go od początku. Dlatego wybrana została gotowa implementacja dostępna na systemy Linux: Chrony v4.1. W projekcie obecność tej implementacji jest zapewniona przez kontener Dockera, a więc nie wymaga ręcznej instalacji pakietów ani rekompilacji jądra. Szczegółowe instrukcje konfiguracji środowiska aplikacji serwerowej przedstawiono w podrozdziale 7.2.2.



Rysunek 2.17: Logo oprogramowania serwera NTP - Chrony

2.5.9 Narzędzie do analizy poprawności zebranych danych

Jako narzędzia do analizy poprawności zebranych danych użyto Języka Python wraz z bibliotekami Pandas[45] oraz Matplotlib[46]. Pandas to biblioteka służąca do wykonywania obliczeń tablicowych. Matplotlib to biblioteka do tworzenia wykresów.



Rysunek 2.18: Logo Pandas



Rysunek 2.19: Logo Matplotlib

Rozdział 3

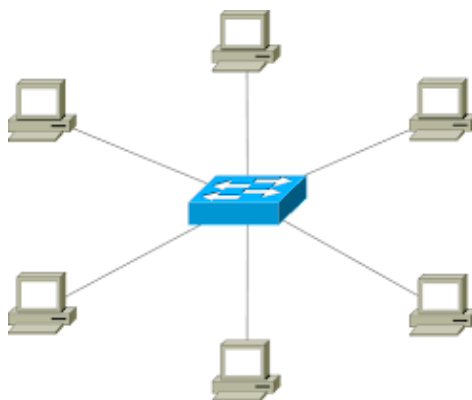
Konstrukcja sieci

3.1 Topologia sieci

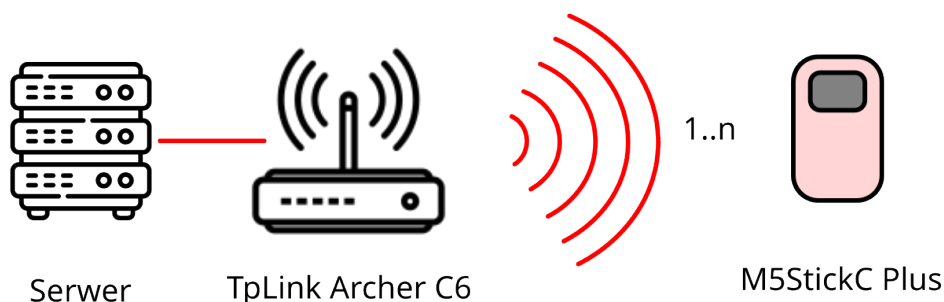
Sieć czujników została zaprojektowana w oparciu o topologię gwiazdy. Jej uproszczony schemat zaprezentowano na rys. 3.1. Topologia gwiazdy jest najprostszym i najbardziej efektywnym rozwiązaniem w sieci bezprzewodowej, w której cały ruch przechodzi przez jeden, centralny węzeł. Na etapie projektowania pod uwagę była również brana możliwość użycia topologii siatki, jednak pomysł ten został odrzucony ze względu na dodatkową złożoność projektu i koszty utworzenia z użyciem technologii IP. Schemat konkretnej sieci użytej w projekcie został przedstawiony na rys. 3.2

3.2 Standard sieci bezprzewodowej

Sieć bezprzewodowa jest oparta na technologii IEEE 802.11, szerzej znanej jako Wi-Fi. Technologia ta została wybrana jako najtańsza dostępna, wszechstronna i wystarczająca do wymagań [2].



Rysunek 3.1: Topologia gwiazdy



Rysunek 3.2: Uproszczony schemat sieci

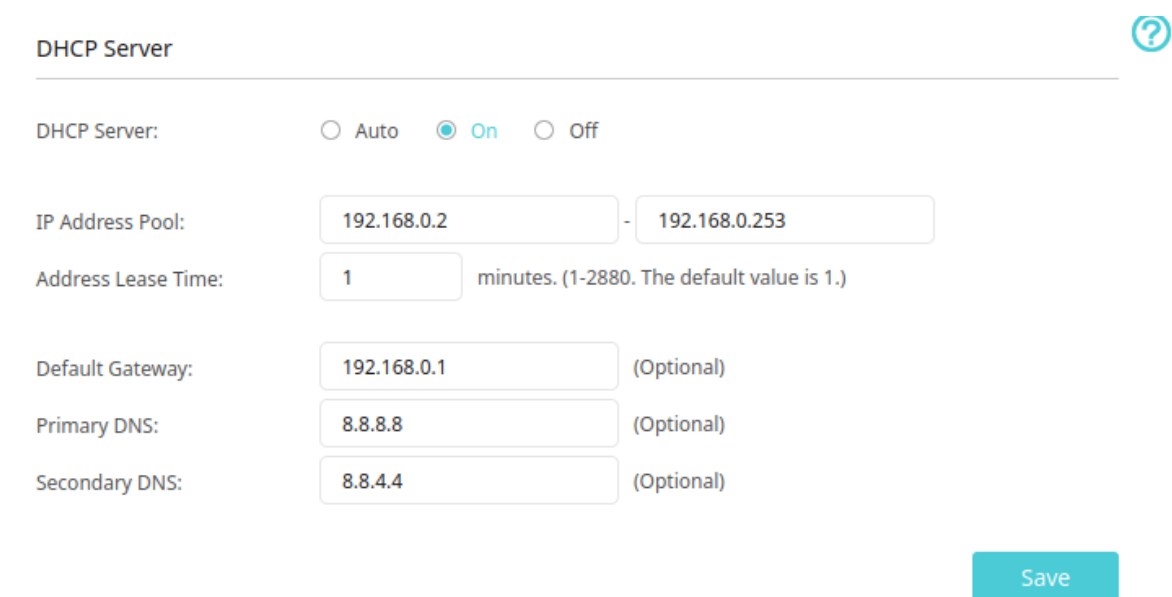


Rysunek 3.3: Logo technologii Wi-Fi

3.3 Adresy i DHCP

Serwer DHCP umożliwia pominięcie ręcznego ustawiania IP dla węzłów. Jest on niezbędny dla poprawnego działania projektu. Urządzenie na którym działa aplikacja serwerowa oraz serwer NTP mają przydzielone stałe IP, mieszczące się w tej samej sieci co węzły, jednak nie mieszczące się w puli DHCP.

Dla potrzeb projektu założono, że cały system będzie się mieścił w jednej sieci. Adres tej sieci to **192.168.0.0**, natomiast maska to **255.255.255.0**. Biorąc pod uwagę, że ostatni adres w sieci jest adresem broadcast, a adres "zerowy" jest adresem sieci, liczba dostępnych adresów wynosi $2^8 - 2 = 254$. Dodatkowo, należy zarezerwować adresy IP dla routera oraz dla serwera aplikacji. Przyjmijmy, że będą to odpowiednio **192.168.0.1** oraz **192.168.0.254**. Pozostawia to **252 adresy** do wykorzystania na rzecz węzłów. Zakres adresów DHCP zaczyna się więc na **192.168.0.2** oraz kończy na **192.158.0.253**. Listę adresów w sieci zaprezentowano w tabeli 3.1. Odpowiednią konfigurację routera zaprezentowano na rys. 3.4. Konfigurację portu sieciowego serwera przedstawiono na rys. 3.5.



DHCP Server 

DHCP Server: ☐ Auto ☒ On ☐ Off

IP Address Pool: 192.168.0.2 - 192.168.0.253

Address Lease Time: 1 minutes. (1-2880. The default value is 1.)

Default Gateway: 192.168.0.1 (Optional)

Primary DNS: 8.8.8.8 (Optional)

Secondary DNS: 8.8.4.4 (Optional)

Save

Rysunek 3.4: Ustawienia DHCP w routerze TpLink Archer C6

```
dawid@bombelelek:~$ ip a show enp3s0
2: enp3s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 60:a4:4c:50:9e:67 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.254/24 brd 192.168.0.255 scope global enp3s0
        valid_lft forever preferred_lft forever
    inet6 fe80::62a4:4cff:fe50:9e67/64 scope link
        valid_lft forever preferred_lft forever
dawid@bombelelek:~$
```

Rysunek 3.5: Ustawienia sieciowe na serwerze

Adres	Przeznaczenie	Nadany przez DHCP
192.168.0.0	Adres sieci	n/d
192.168.0.1	Adres routera, brama domyślna	NIE
192.168.0.2...253	Pula adresów dla węzłów	TAK
192.168.0.254	Adres serwera	NIE
192.168.0.255	Adres rozgłoszeniowy	n/d

Tabela 3.1: Lista adresów składających się na sieć pomiarową

Rozdział 4

Przepływ informacji w sieci

Inicjowanie połączenia pomiędzy serwerem a węzłami to zadanie węzłów. Po podłączeniu do sieci, próbują się one połączyć do serwera, do portu 7122. Port ten został wybrany na podstawie udostępnionej przez organizację IANA listy portów, gdzie widnieje jako niezarezerwowany. [49]

4.1 Przyjęte kody żądań i odpowiedzi

W strukturach wiadomości na różnych etapach komunikacji pojawiają się kody żądań i odpowiedzi, zdefiniowane unikatowo dla całego systemu. Kody zapisane są w jednym bajcie. Przedstawiono je w tabelach 4.1 i 4.2

Kod	Znaczenie	Etap komunikacji
0x01	Żądanie rejestracji węzła przez wygenerowanie dla niego ID	Nawiązywanie połączenia
0x02	Żądanie autoryzacji przy użyciu wcześniej nadanego ID	Nawiązywanie połączenia
0x03	Informacja o przesyłaniu paczki danych przez węzeł do serwera	Wymiana informacji
0x04	Żądanie zmiany konfiguracji węzła	Polecenia kontrolne
0x05	Żądanie wstrzymania lub rozpoczęcia pomiarów	Polecenia kontrolne

Tabela 4.1: Lista kodów żądań

4.2 Nawiązywanie połączenia

Nawiązywanie połączenia może przebiegać według kilku scenariuszy. Wśród nich znajdują się pozytywne, zakończone nawiązaniem połączenia oraz w zależności od wybranej przez węzeł metody nawiązania połączenia różne negatywne, zakończone zamknięciem przez serwer połączenia TCP. Ogólny schemat połączenia pomyślnego

Kod	Znaczenie	Etap komunikacji
0x01	Ok: wiadomość odebrana i przetworzona pomyślnie	Każdy
0x02	Podane ID nie istnieje	Nawiązywanie połączenia
0x03	Węzeł o podanym ID jest już połączony	Nawiązywanie połączenia
0x04	Brak wolnych ID w puli	Nawiązywanie połączenia
0x05	Handshake niepoprawny	Nawiązywanie połączenia
0x06	Kod żądania nie został rozpoznany	Każdy
0x07	Niepoprawna struktura nagłówka pakietu danych	Wymiana informacji
0x08	Niepoprawna struktura treści pakietu danych	Wymiana informacji
0x09	Pomiar już rozpoczęty	Polecenia kontrolne
0x0A	Pomiar już zatrzymany	Polecenia kontrolne
0x0B	Struktura komendy niepoprawna	Polecenia kontrolne

Tabela 4.2: Lista kodów odpowiedzi i błędów

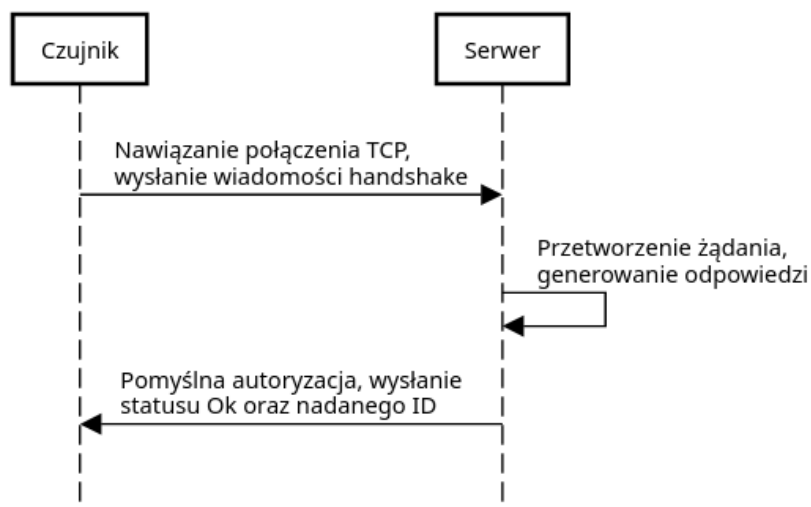
zaprezentowano na rys. 4.1, natomiast ogólny schemat połączenia nieudanego zaprezentowano na rys 4.2. Komunikacja zawsze jest rozpoczynana przez węzeł. Tworzy on połączenie TCP, a następnie wysyła pakiet handshake.

4.2.1 Handshake

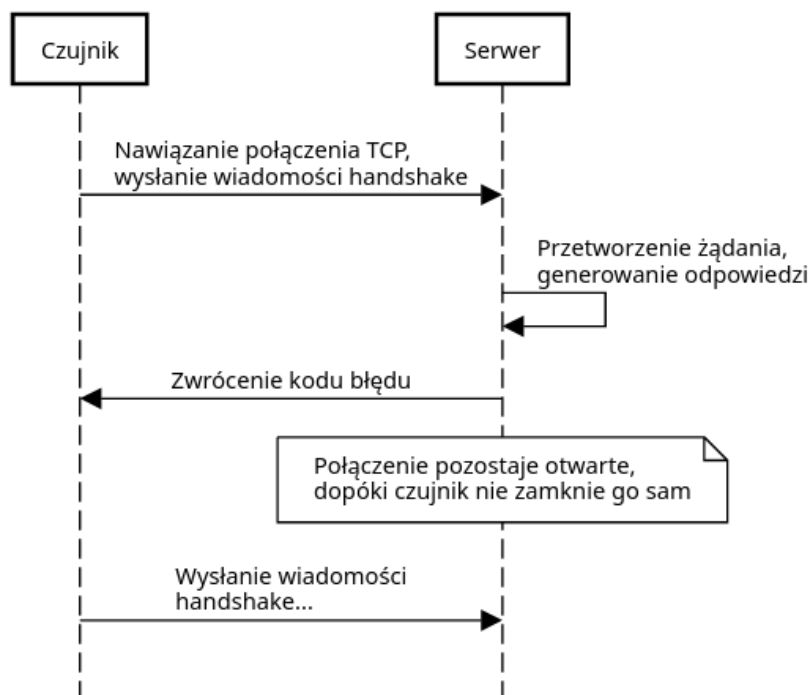
Handshake to nazwa dla co najmniej dwóch komunikatów przesyłanych jako pierwsze po zainicjowaniu połączenia TCP pomiędzy węzłem i serwerem. Węzeł może poprosić serwer o rejestrację, kiedy nie ma zapisanego w pamięci, wcześniej nadanego ID. Może też zażądać autoryzacji przy użyciu zapisanego ID. Handshake trwa, dopóki węzeł nie zerwie połączenia lub serwer pomyślnie nie zautoryzuje węzła, jak zaprezentowano na rysunkach 4.1 i 4.2. Strukturę wiadomości w handshake opisano w tabelach 4.3 oraz 4.4.

1 bajt	1 bajt
kod żądania	ID
Kod żądania (patrz tab. 4.1, etap nawiązywanie połączenia)	ID, jeżeli kod to 0x02, w innym wypadku bez znaczenia

Tabela 4.3: Struktura wiadomości handshake wysyłanej przez węzeł



Rysunek 4.1: Pomyślne nawiązanie połączenia



Rysunek 4.2: Niepowodzenie w nawiązywaniu połączenia

1 bajt	1 bajt
kod odpowiedzi	ID
Kod odpowiedzi (patrz tab. 4.2, etap nawiązywanie połączenia)	ID, jeżeli kod to 0x02

Tabela 4.4: Struktura odpowiedzi handshake wysyłanej przez serwer

4.3 Komunikacja po nawiązaniu połączenia

Po pomyślnym nawiązaniu połączenia następuje obustronne wysyłanie wiadomości pomiędzy węzłem a serwerem. Węzeł ma możliwość, według zasad obecnie aktywnej konfiguracji, żeby wysyłać do serwera pakiety danych. Serwer natomiast ma możliwość wysyłać do czujnika polecenia konfiguracji i przełączania.

4.3.1 Wysyłanie pakietów danych

Wysyłanie pakietów danych przez węzeł odbywa się w regularnych, lub nieregularnych odstępach czasu, w zależności od czujnika i jego konfiguracji. Dane z czujników są zbierane w paczki, które opatrzone odpowiednimi informacjami w nagłówku są przesyłane do serwera. Serwer nie zwraca potwierdzenia pomyślnego odebrania danych, choć takie rozwiązanie może być pomocne w testowaniu wdrożenia rozwiązań opartych na tym systemie i może stanowić niezbędne usprawnienie komunikacji w dalszym rozwoju projektu.

Długość paczek danych jest zmienna i dla każdego czujnika może być określana indywidualnie. Przyjęty przez nas maksymalny rozmiar pakietu TCP dla czujnika IMU wynosi 1460 bajtów. Limit ten jest jednak umowny i może zostać zmieniony kosztem zwiększenia zajętości pamięci dynamicznej mikroprocesora, przez co zmniejszy się możliwość rozbudowy systemu. Dla mikrofonu rozmiar paczki to 16 bajtów nagłówka plus ilość danych, która akurat znajdowała się w buforze w momencie wysłania pakietu.

Struktura pakietu danych składa się z nagłówka oraz treści. Treść to zbiór kolejnych odczytów, które są definiowane przez nagłówek paczki oraz obecne ustawienia czujnika. Struktura nagłówka została przedstawiona w tabeli 4.5. Nagłówek zawiera informacje niezbędne do prawidłowej interpretacji treści. Na podstawie danych z nagłówka określa się czas wykonania poszczególnych pomiarów. Pierwszy pomiar został wykonany w podanej sekundzie i mikrosekundzie, a kolejne były wykonywane z podaną w tym nagłówku częstotliwością.

16 bajtów					
1 bajt	1 bajt	4 bajty	4 bajty	4 bajty	2 bajty
Kod komunikatu (0x03)	Typ danych	Sekundy	Mikrosekundy	Częstotliwość	Ilość
Kod komunikatu (patrz tab. 4.1)	Typ przesyłanych danych: mikrofon lub IMU 4.6	Sekundy wykonania pierwszego pomiaru	Mikrosekundy wykonania pierwszego pomiaru	Częstotliwość wykonywania pomiarów	Ilość wykonanych pomiarów (little endian)

Tabela 4.5: Struktura nagłówka paczki danych

1 bajt	
kod czujnika	czujnik
0x01	IMU
0x02	Mikrofon

Tabela 4.6: Kody czujników w komunikacji

4.3.2 Wysyłanie poleceń do węzła

Pakiety poleceń, a więc żądania wyłączenia, włączenia lub przełączania czujnika oraz konfiguracji czujników składają się z kilku (co najmniej trzech) bajtów i są wysyłane przez serwer do węzła.

3+ bajtów		
1 bajt	1 bajt	1 bajt (lub więcej)
Kod polecenia (0x03)	Rodzaj polecenia	Dodatkowe argumenty
Kod polecenia (patrz tab. 4.1)	Rodzaj polecenia: ustawienia lub przełączenie	Dodatkowe argumenty, jeżeli rodzaj polecenia ich wymaga. Są nieindywidualne dla każdego czujnika.

Tabela 4.7: Struktura pakietu polecenia

4.4 Bezpieczeństwo

4.4.1 Bezpieczeństwo danych

Połączenie pomiędzy węzłem i serwerem pozostaje niezaszyfrowane. Bezpieczeństwo danych nie jest wymogiem koniecznym w założeniach projektu, natomiast jest możliwe jego wprowadzenie, jeżeli bezpieczeństwo przesyłanych danych stanowi priorytet.

Polecanym rozwiązaniem jest warstwa TLS, pozwalająca zapewnić autentyczność i bezpieczeństwo przesyłanych danych [2] [50]. Wprowadzenie takich modyfikacji będzie wymagało ingerencji w kod oprogramowania serwerowego oraz oprogramowania węzła, wychodzącej daleko poza zakres tej pracy.

4.4.2 Odporność na chwilowe zerwanie połączenia

Chwilowe zaniki połączenia są obsługiwane przez kartę sieciową serwera oraz węzłów. Fakt korzystania z protokołu TCP/IP sprawia, że nie ma potrzeby przykładać wagi do tego problemu z poziomu oprogramowania aplikacji.

Jeżeli połączenie zostanie chwilowo zerwane pomiędzy serwerem a węzłem, to próbują one nawiązać komunikację raz jeszcze. Węzeł zatrzymuje w tym momencie pracę i próbuje nawiązać połączenie z serwerem. Jeżeli połączenie zostanie poprawnie nawiązane i handshake zakończy się sukcesem, to węzeł wznowia pracę w miejscu w którym nastąpiło przerwanie.

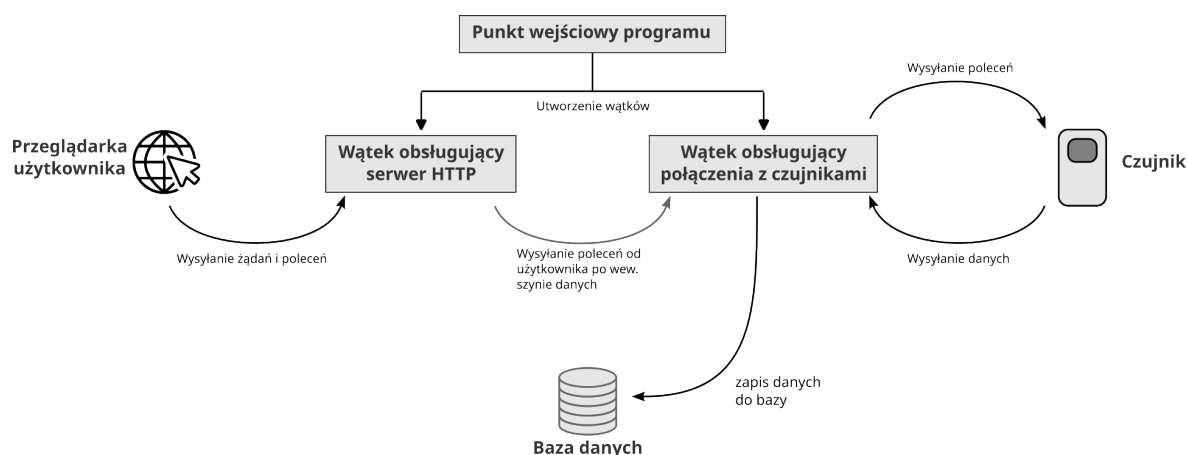
Rozdział 5

Aplikacja serwerowa

5.1 Architektura aplikacji

Oprogramowanie napisane w języku Go można podzielić na dwa serwisy: jeden obsługujący połączenie z węzłami, odbierający z nich dane i zapisujący je, a także wysyłający do nich polecenia oraz drugi, odpowiedzialny za interakcję z użytkownikiem lub innym programem poprzez interfejs webowy oraz REST API, czyli interfejsu programistycznego dla oprogramowania interfejsu użytkownika. Wewnętrzna szyna danych pozwala na przekazywanie danych wejściowych (poleceń) od użytkownika do odpowiedniego węzła. Całościowy obraz systemu został zaprezentowany na rys. 5.1.

Punkt wejściowy programu, który jest głównym wątkiem, ma za zadanie połączyć się z bazą i utworzyć wszystkie struktury, które będą wykorzystywane przez wiele części aplikacji, jak np. obiekt połączenia z bazą, obiekt rejestr węzłów, szyna danych. Następnie tworzy on wątki programu, którym przekazuje utworzone wcześniej struktury. Kod opisujący to zachowanie został załączony jako B.7.

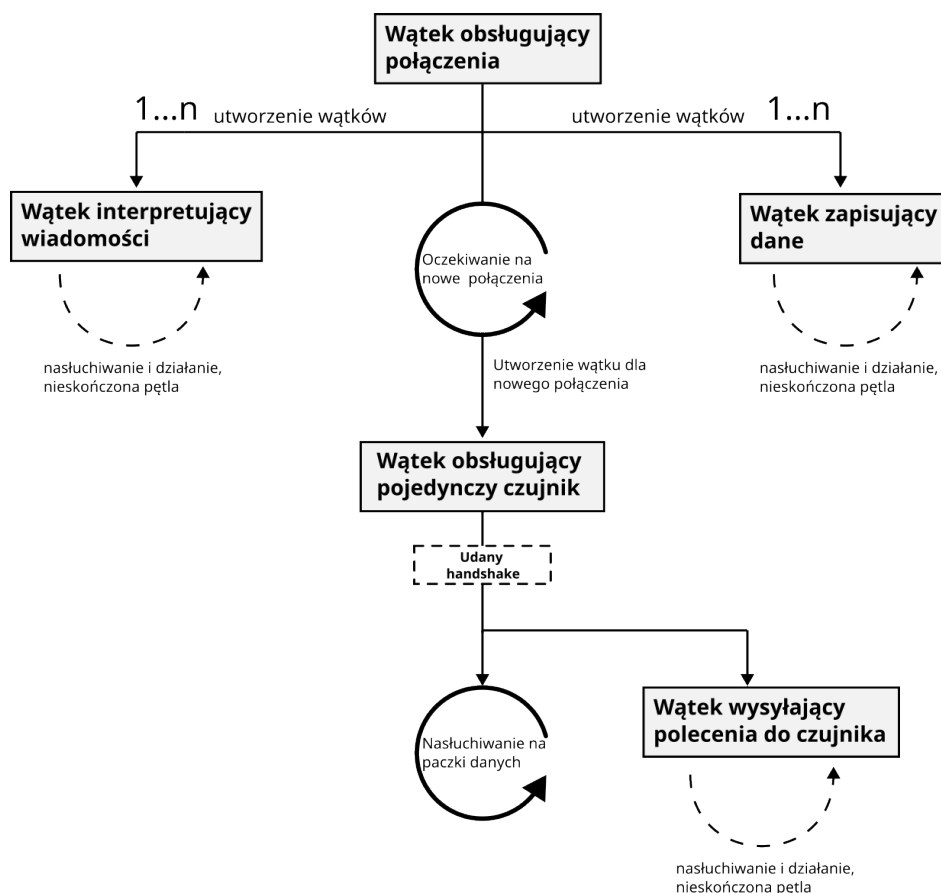


Rysunek 5.1: Schemat architektury aplikacji serwerowej

5.1.1 Interakcja z węzłami

Wątek odpowiedzialny za interakcję z węzłami rozpoczyna się utworzeniem wątków interpreterów wiadomości. Zostaną one wykorzystane później, do zrównoleglenia obsługi danych przychodzących z różnych węzłów. Potem następuje otwarcie nasłuchu na porcie 7122. Jeżeli połączenie TCP/IP zostanie nawiązane, to utworzony zostaje kolejny wątek obsługujący to połączenie, a sam wątek macierzysty wraca do nasłuchiwanie. Nowy wątek oczekuje od węzła wiadomości handshake. Kod odpowiedzialny za interpretację wiadomości handshake jest załączony jako B.2. Po pomyślnym handshake tworzony jest osobny wątek przesyłający polecenia do węzła, jeżeli jakieś się pojawiają, a sam wątek macierzysty rozpoczyna obsługę danych przychodzących z węzła w pętli. Zachowanie zostało przedstawione na rys. 5.2, kod wątku został załączony jako B.8, a kod odpowiedzialny za odbiór wiadomości jako B.4.

W przypadku zakończenia pracy wątku obsługującego połączenie usuwane są wszystkie wątki przez niego utworzone, natomiast główny wątek tworzący wątki interpreterów jest kończony tylko kiedy główny program zostanie zakończony.



Rysunek 5.2: Schemat wątków obsługi węzłów

5.1.2 Interakcja z użytkownikiem

Użytkownik może wchodzić w interakcje z aplikacją, to znaczy wysyłać polecenia rekonfiguracji do wybranych węzłów. Możliwe jest to dzięki modułowi aplikacji odpowiedzialnemu za serwer HTTP. Serwer udostępnia interfejs użytkownika oraz REST API, z którego interfejs użytkownika korzysta.

Zadaniem serwera HTTP jest odbieranie i wysyłanie żądań protokołu HTTP i HTTPS, czyli szyfrowanych. W naszym przypadku połączenie będzie niezaszyfrowane, choć możliwe jest jego zaszyfrowanie przy pomocy dedykowanego oprogramowania, np. serwera Nginx. [51] [53]

Interfejs użytkownika, zawarty w katalogu projektu poświęconym aplikacji serwerowej `server/gui` jest utworzony na podstawie HTML, CSS oraz JavaScript. Jest on opisany na wielu plikach, rozdzielonych według swojego przeznaczenia (struktura, stylowanie, skrypt). W przypadku HTML i CSS do stylizacji został wykorzystany framework Bootstrap, czyli zestaw gotowych komponentów, pozwalający skrócić pracę i uzyskać pewność estetyki [54]. W przypadku JavaScript, czyli języka odpowiedzialnego za skrypty i mechanikę interfejsu, użyto frameworka Vue. [55] Sam serwer HTTP został zbudowany na bazie frameworka Gin. [56] Architektura rozwiązania po stronie serwera, w więc z pominięciem interfejsu użytkownika, została przedstawiona na rys. 5.7. Kod kontrolera, odpowiedzialny za odebranie od użytkownika danych nt komend, które mają zostać przekazane do czujnika załączono jako B.5.



Rysunek 5.3: Logo technologii HTML



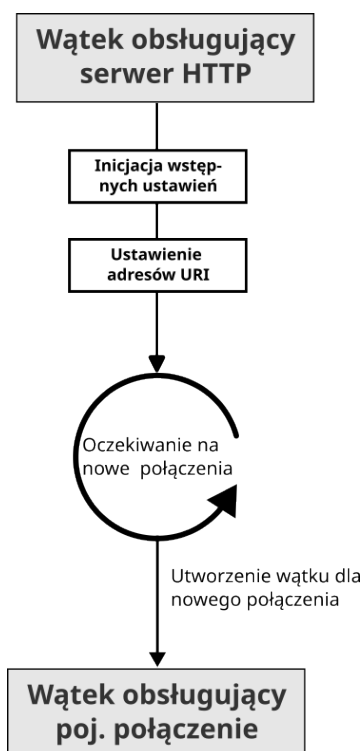
Rysunek 5.4: Logo frameworka Gin



Rysunek 5.5: Logo technologii CSS



Rysunek 5.6: Logo frameworka Vue

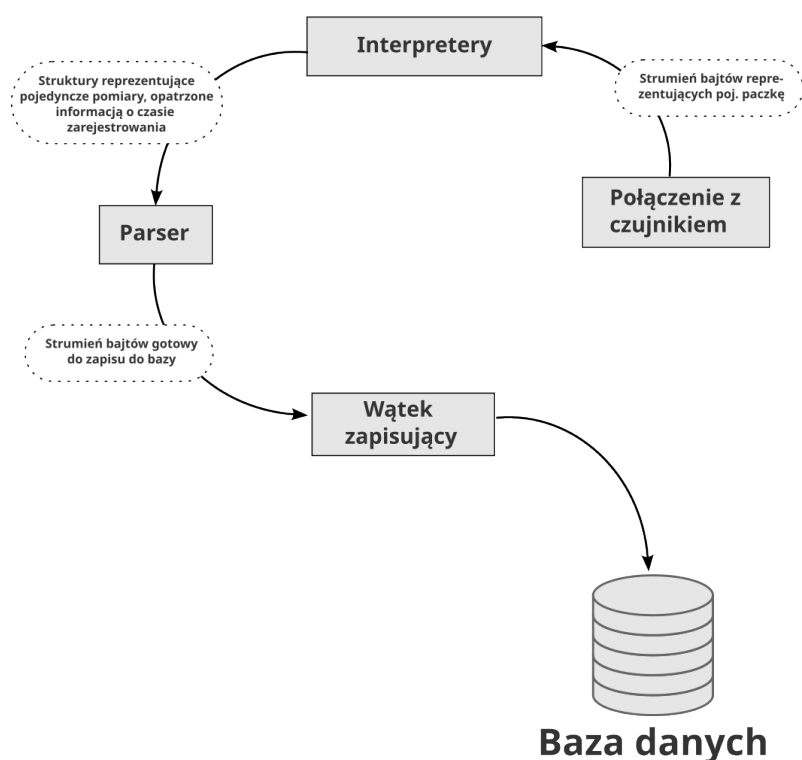


Rysunek 5.7: Schemat części aplikacji obsługującej serwer HTTP

5.1.3 Przepływ informacji i zapis danych

W procesie tworzenia aplikacji wypróbowano wiele metod zapisu danych z czujników, spośród których najwydajniejszy okazał się zapis binarny do plików. Baza danych, do której następują odwołania w pracy to po prostu system plików serwera. Odczyty z

czujników są zapisywane do plików binarnych. Taki porządek rzeczy jest podyktowany stosunkowo niskim kosztem zapisu danych w tym formacie w porównaniu do innych, wypróbowanych rozwiązań, jednak taki zapis wprowadza konieczność późniejszego (po zakończonym pomiarze i zapisie) przetworzenia plików na format zdatny do analizy. Proces ten został opisany w podrozdziale 7.2.2. Na wcześniejszym etapie rozwoju projektu została zastosowana baza relacyjna MySQL, a po niej zapis bezpośrednio do plików CSV, jednak te metody okazały się zbyt nieprzystosowane do zapisywania tak dużej liczby danych w tak małym czasie, w efekcie czego następowało gubienie pomiarów oraz wysokie zużycie zasobów serwera.



Rysunek 5.8: Schemat przepływu informacji zapisywanych do bazy danych

Schemat przepływu informacji został zaprezentowany na rys. 5.8. Żaden wątek obsługujący połączenie z węzłem nie dokonuje zapisu do bazy bezpośrednio. Ciąg bajtów reprezentujący jeden pakiet pomiarów jest przekazywany przez wątek obsługujący połączenie z węzłem do kolejki interpretatorów, która jest obsługiwana przez pulę wątków interpretujących wiadomości. To zachowanie jest widoczne w kodzie obsługującym nasłuch, załączonym jako B.8, a kod wątków interpretujących został załączony jako B.3. Ich zadaniem jest na podstawie otrzymanych w nagłówku pakietu informacji podzielenie ciągu na pojedyncze pomiary oraz określenie czasu ich zarejestrowania. Tak

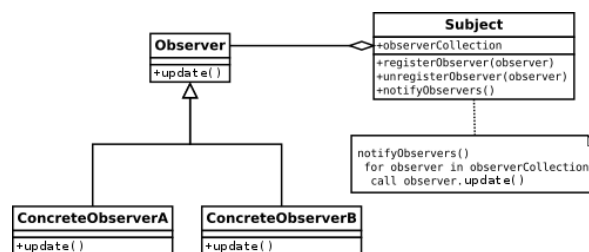
przygotowane struktury są wysyłane do kolejki parserów, gdzie są zamieniane na ciągi bajtów w takiej formie, w jakiej mają trafić do pliku. Następnie te dane są kierowane do kolejki zapisów, gdzie wątek zapisujący dane zajmuje się zapisem danych do bazy. Kod odpowiedzialny za zapis danych oraz parsowanie struktur załączono jako B.6.

Informacja o węzłach w aplikacji sprowadza się tylko do listy ID węzłów, które zostały wydane oraz listy ID, które są obecnie połączone. Dane te są ulotne, trzymane w pamięci programu. Dobrym kierunkiem dalszego rozwoju jest umieszczenie ich w bazie danych, a także zbieranie dodatkowych informacji jak np. unikalny identyfikator (MAC) urządzenia do którego ID należy, obecnie aktywne ustawienia etc.

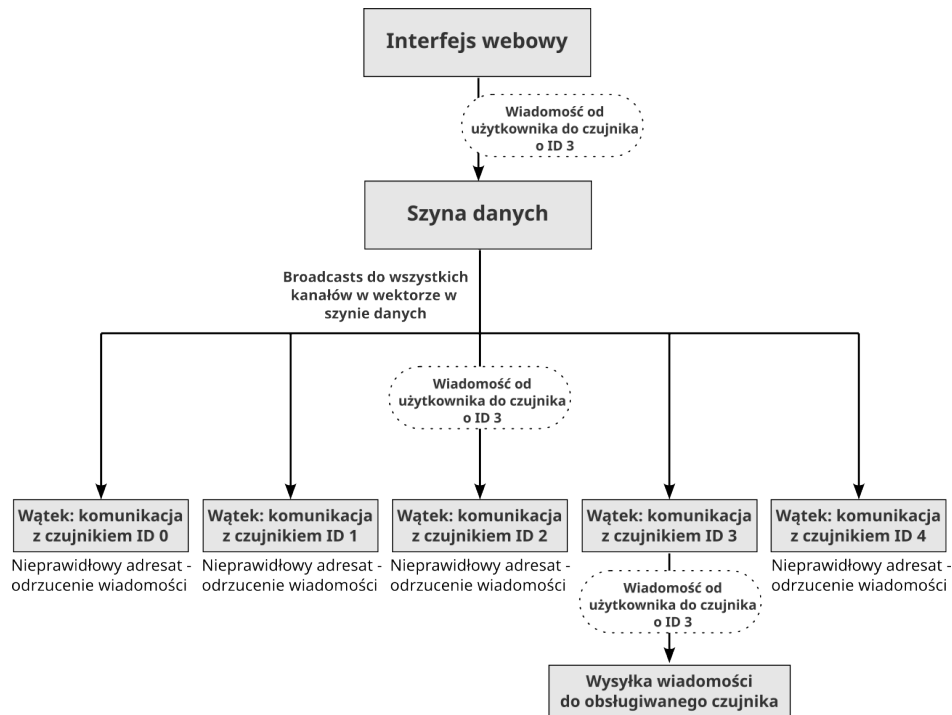
5.1.4 Przekazywanie poleceń od użytkownika do węzła

Wewnętrzna szyna danych jest oparta na wzorcu projektowym obserwator, którego ogólny schemat w notacji UML został przedstawiony na rys 5.9 [4]. Oprócz tego, opiera się ona również na wbudowanych w język strukturach danych, kanałach (ang. channel), będących implementacją kolejek, bezpiecznych do używania w środowisku wielowątkowym [3]. Jej zadaniem jest przekazywanie poleceń od użytkownika, wprowadzonych za pomocą interfejsu webowego do konkretnych węzłów, których te polecenia dotyczą.

W głównym wątku programu jest inicjalizowana struktura - szyna danych, zawierająca odniesienie to wektora kanałów. Istnieje metoda, pozwalająca na dodanie do wektora nowego kanału oraz zwracająca wskaźnik do niego. Za każdym razem, kiedy zostanie wysłana wiadomość z poleceniem na szynę danych, jest ona przekazywana dalej do każdego z kanałów w tym wektorze, jest to tzw. broadcast. Odbiorcy tych kanałów, w domyśle wątki obsługujące komunikację z węzłem już we własnym zakresie decydują, czy otrzymana wiadomość powinna zostać wysłana do węzła na podstawie załączonego do wiadomości ID. Działanie tego systemu zostało zobrazowane na rys. 5.10. Kod szyny został załączony do pracy jako B.1



Rysunek 5.9: Ogólny schemat wzorca obserwator w graficznej reprezentacji notacji UML



Rysunek 5.10: Schemat działania szyny danych

5.2 Wydajność i ograniczenia

Liczba wątków interpretujących jest konfigurowalna, może zostać dostosowana do bieżących potrzeb. Aplikacja po drobnych modyfikacjach nadaje się do skalowania horyzontalnego, czyli zwiększania wydajności programu poprzez uruchamianie jej na dodatkowych maszynach. Wymagany byłby do tego load balancer [62]. Testy wydajności oraz badanie ograniczeń aplikacji przeprowadzono dla jednej maszyny, z węzłami połączonymi w sieci lokalnej. Specyfikacja maszyny, zawierająca najważniejsze informacje, została przedstawiona w tabeli 5.1 [63]. Pozostałe elementy użyte w testach są tymi samymi, które wymieniono w rozdziale poświęconym użytym elementom.

Testy rozpatrują ograniczenia na poszczególnych etapach komunikacji. Dla danej konfiguracji, maksymalna wydajność całego systemu ograniczona jest przez ten element, którego możliwości przetwarzania lub przesyłania danych będą najniższe, tzw. "wąskie gardło".

5.2.1 Wydajność węzła i specyfikacja danych wejściowych

Karta sieciowa węzła nie pozwala na wysłanie większej ilości danych niż 1 MB/s. Węzeł nie jest w stanie wygenerować tyle danych, aby ten bufor zapełnić. Doświadczalnie wyznaczono, że przy największej intensywności pracy (mikrofon, 44100 Hz) pojedynczy węzeł przesyła dane z prędkością nie większą niż 100 kB/s.

Element systemu	Specyfikacja
Procesor	Intel(R) Core(TM) i5-7300HQ CPU @ 2.50GHz, 4 rdzenie
Karta sieciowa	Realtek Semiconductor Co., Ltd. RTL8111/8168/8411 PCI Express
Pamięć RAM	16 GB DDR4
Dysk twardy	Samsung SSD 970 EVO 250GB

Tabela 5.1: Specyfikacja maszyny, użytej do testów wydajności i ograniczeń

5.2.2 Wydajność routera

W zależności od użytego routera, jego wydajność będzie się zmieniać. Użyty do testów router to TpLink Archer C6, którego złącza przewodowe pracują z prędkością do 125 MB/s oraz który ma możliwość pracy w standardzie Wi-Fi 802.11ac. Maksymalna teoretyczna przepustowość dla częstotliwości 2,4 GHz, w której pracują karty sieciowe węzłów wynosi 37,5 MB/s. Przepustowość sieci bezprzewodowej jest dzielona pomiędzy podłączone urządzenia. Jeżeli przyjmiemy, że jeden czujnik potrzebuje przepustowości 100 kB/s, to z użyciem tego urządzenia możliwe jest efektywne wykorzystanie do 375 czujników.

5.2.3 Wydajność karty sieciowej serwera

Karta sieciowa na serwerze obsługuje do 125 MB/s na połączeniu kablowym. Ta wartość jest podobna do obsługiwaną przez router, co sprawia, że karta sieciowa używana w testowanej sieci nie będzie ograniczeniem - nie będzie zmuszona do przetwarzania większej ilości danych, niż te dostarczone przez router.

5.2.4 Wydajność aplikacji serwerowej

Aplikacja serwerowa została przetestowana pod względem wykorzystywanych zasobów serwera z każdą dostępną ilością czujników, z każdym rodzajem czujnika. Pomiaru dokonano za pomocą narzędzia 'docker stats', na uruchomionej, skonteneryzowanej aplikacji z udostępnionym odpowiednim portem. Zużycie CPU jest mierzone w procentach czasu obliczeniowego oferowanego przez jeden rdzeń procesora, co oznacza, że wyniki powyżej 100% są możliwe, gdyż oznacza to wykorzystanie czasu obliczeniowego więcej niż jednego rdzenia. W stanie bezczynności, tj. żadne pomiary nie są wykonywane, aplikacja serwerowa zajmuje 0% CPU oraz 6,5 MB RAMu. Pomiary zużycia zasobów zostały zaprezentowane w tab. 5.2. Zauważyć można, że wzrost zużycia pamięci oraz czasu obliczeniowego procesora jest liniowy, co jest spodziewanym zjawiskiem - każdy czujnik dodaje dwa wątki obsługujące połączenie (wysyłanie komend, odbieranie danych) oraz dodaje stałą liczbę pakietów odbieranych przez serwer.

Konfiguracja	użycie CPU [%±0,1%]	Użycie RAM [MB±0,1MB]
Jeden czujnik IMU	1,2	15,4
Dwa czujniki IMU	1,6	19,1
Trzy czujniki IMU	3,1	25
Cztery czujniki IMU	4,3	31,2
Jeden czujnik MIC	13	25
Dwa czujniki MIC	20,1	28,9
Trzy czujniki MIC	35	36,1
Cztery czujniki MIC	44,8	49,3

Tabela 5.2: Zestawienie zużycia zasobów przez aplikację działającą w pełnej wersji

5.3 Interfejs webowy

Spośród różnych możliwości, jak np. interakcja przez konsolę czy natywna aplikacja okienkowa na komputery, wybrany został interfejs webowy, dostępny przez przeglądarkę. Wybór taki jest najlepszy z podanych, ponieważ zapewnia łatwość i prostotę implementacji i użytkowania. Nie wymaga instalowania dedykowanego oprogramowania a jedynie posiadania przeglądarki z dostępem do odpowiedniego portu serwera aplikacji. W niczym również nie ustępuje alternatywnym rozwiązaniom.

5.3.1 REST API

REST API zapewnia interfejsowi webowemu oraz programistom dalej rozbudowującym komunikację z aplikacją zestaw narzędzi, przeznaczonych do komunikacji z aplikacją [52]. API opiera się na formacie danych JSON, który stanowi najlepszy wybór do tego typu komunikacji ze względu na swoją prostotę i klarowność. Obecnie REST API jest ograniczone do endpointu POST */command* (np. <http://localhost:8081/command>), czyli adresu, na który należy wysłać żądanie HTTP, który przyjmuje dane w metodzie POST i pozwala na wysyłanie poleceń do węzłów. Użytkownik nie musi robić tego ręcznie, jest to automatycznie robione przez skrypty interfejsu użytkownika. W przyszłości można rozbudować API o endpointy pozwalające na pobranie danych nt węzłów czy stanu serwera.

5.3.2 Interfejs HTML i JavaScript

Interfejs webowy, napisany przy użyciu HTML, CSS i JavaScript oraz odpowiednich dla tych języków frameworków stanowi przykład standardowego panelu kontrolnego, dostępnego przez internet. Interfejs zaprezentowany został na rys. 5.11. Jest to prosty zestaw pól typu select oraz różnych przycisków, pozwalających na wysłanie ustawionych danych do serwera a także lista komunikatów walidacyjnych. Przyciski oskryptowane są w taki sposób, że na naciśnięcie wysyłają do serwera polecenie, w którym są wyspecjalizowane, zbierając przy tym wartości wskazane w panelu. Działanie skryptu zostało zaprezentowane na rys. 5.12

Panel kontrolny

Ustawienia czujnika

ID czujnika

1 ▾

Opcje IMU

2G ▾

250 DPS ▾

Wyślij ustawienia IMU

Opcje Mic

44100 Hz ▾

Wyślij ustawienia MIC

Status czujnika

Imu ▾

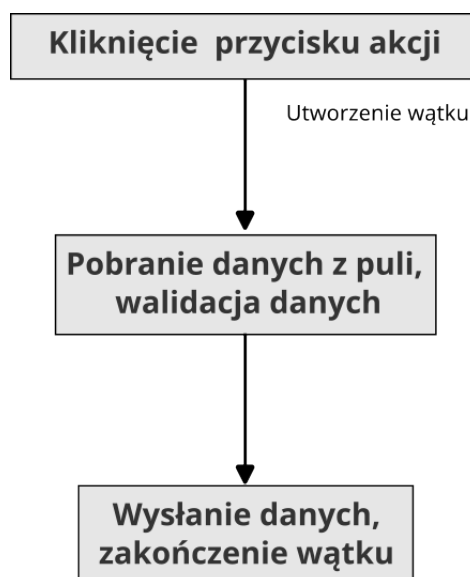
Włącz czujnik

Wyłącz czujnik

Komunikaty

19:58:36 ok

Rysunek 5.11: Interfejs webowy, panel kontrolny systemu



Rysunek 5.12: Schemat przedstawiający działanie skryptu w interfejsie webowym

Rozdział 6

Aplikacja węzłów

6.1 Architektura aplikacji

Architektura aplikacji węzłów została zaprezentowana na rys.6.2. Aplikację można podzielić na 2 główne części. Część główna aplikacji, odpowiada za:

- logowanie się do sieci Wi-Fi
- nawiązywanie/odawianie połączenia z aplikacją serwerową i wysyłanie danych
- odbieranie komend od aplikacji serwerowej i ich wykonanie ich

Część pomiarowa aplikacji, składa się z modułów, po jednym dla każdego czujnika. Każdy moduł jest samodzielną częścią aplikacji, która odpowiada za:

- odczyt danych z czujnika
- przetworzenie danych
- wysłanie danych do głównej części aplikacji

W celu zapewnienia stabilności czasowej pomiarów, przyjęto że część główna aplikacji działa na innym rdzeniu niż część pomiarowa. Zapewnia to niezakłócone pomiary, ze stałą częstotliwością. Niezależnie od stanu głównej aplikacji. Dodatkowym atutem takiego podziału jest możliwość wysyłania danych do aplikacji serwerowej równole do odczytu kolejnych danych z czujników.

Interakcja pomiędzy obiema częściami aplikacji odbywa się za pomocą kolejek danych [20]. oraz kilku funkcji udostępnionych przez moduły czujników. Za pomocą kolejek danych przesyłane są dane z czujników do głównej części aplikacji, która następnie przesyła je do aplikacji serwerowej. Każdy moduł czujnika do poprawnej integracji z główną częścią aplikacji musi udostępnić funkcje:

- inicjalizacji wątku pomiarowego - przyjmuje jako argument wskaźnik na obiekt komunikacyjny, który pozwala dodać wiadomość do kolejki danych
- zakończenia wątku pomiarowego - brak argumentów

- zmiany parametrów pomiaru - przyjmuje ciąg bajtów, który zawiera parametry pomiaru

Natomiast główna część aplikacji udostępnia części pomiarowej funkcję przekazującą dane do kolejki. Z której następnie zostaną pobrane przez główną część aplikacji i przesłane do aplikacji serwerowej.

Przyjęto konwencję że każdy moduł czujnika przesyła pakiet danych pomiarowych po przez kolejkę w formie struktury Msg 6.1 zdefiniowanej globalnie, która zawiera:

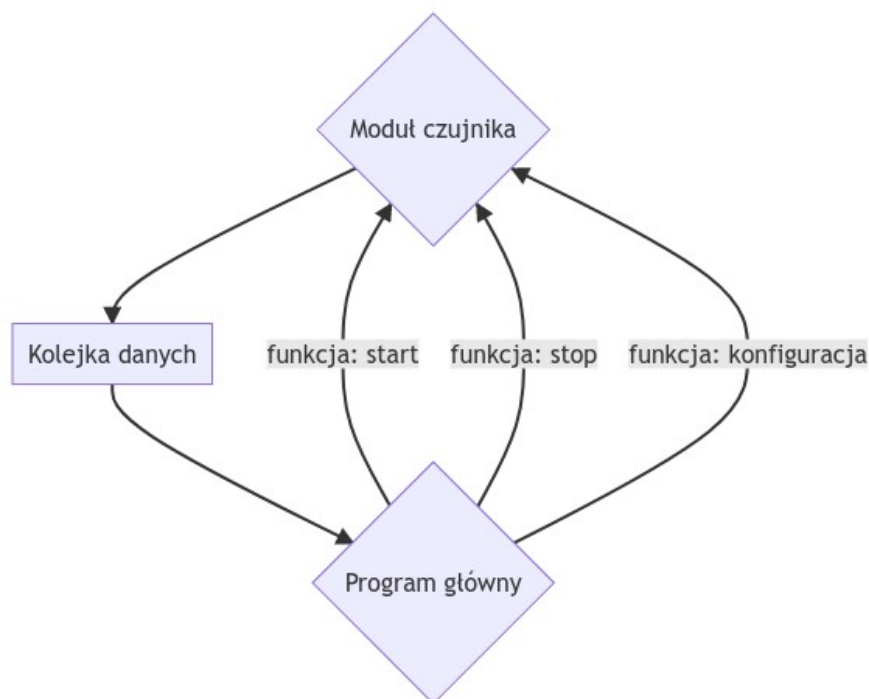
- wskaźnik na ciąg bajtów zawierający dane
- liczbę wysyłanych bajtów
- wskaźnik na mutex który jest zablokowany i zostanie odblokowany po zakończeniu przetwarzania danych

Taka architektura pozwala na łatwe dodawanie nowych czujników, bez konieczności modyfikacji głównej części aplikacji.

Kod źródłowy aplikacji znajduje się w Dodatku A.

```
struct Msg
{
    byte *data;
    int len;
    SemaphoreHandle_t* mutex;
};
```

Rysunek 6.1: Struktura Msg

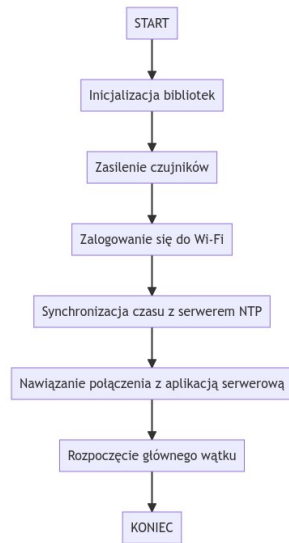


Rysunek 6.2: Architektura aplikacji węzłów

6.2 Użyte biblioteki

- **Arduino**[38] - biblioteka zmienia nasz projekt, dodając nam możliwość budowania go analogicznie jak w środowisku Arduino IDE [37]. Dzięki temu dostajemy dostęp do wielu funkcji, które są dostępne w tym środowisku.
- **WiFi**[36] - biblioteka umożliwiająca połączenie z siecią Wi-Fi. Dostępna jedynie w połączeniu z biblioteką Arduino.
- **EEPROM**[35] - biblioteka używa pamięci Flash do emulacji pamięci EEPROM. Dostępna jedynie w połączeniu z biblioteką Arduino. Używamy jej do zapisywania ID otrzymanego od serwera w pamięci nieulotnej.
- **M5StickCPlus**[34] - biblioteka stworzona przez twórców modułu M5StickC PLUS. Dostępna jedynie w połączeniu z biblioteką Arduino. Zawiera biblioteki potrzebne do obsługi urządzeń znajdujących się w module. Potrzebujemy jedynie części odpowiedzialnej za obsługę modułu zasilającego te urządzenia.
- **Wire**[32] - biblioteka odpowiedzialna za łączenie się po przez protokół I2C. Dostępna jedynie w połączeniu z biblioteką Arduino. Używamy jej do komunikacji z 6-cio osiowym IMU.
- **driver/i2s**[33] - biblioteka odpowiedzialna za łączenie się po przez protokół I2S. Używamy jej do komunikacji z mikrofonem cyfrowym.

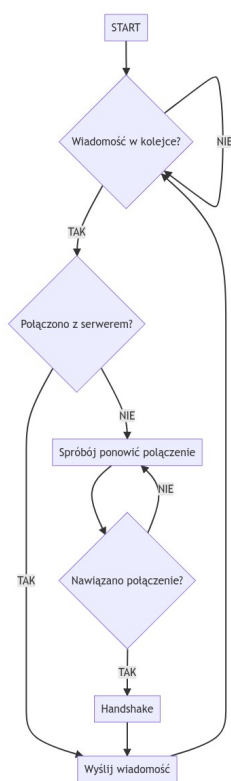
6.3 Schematy działania aplikacji

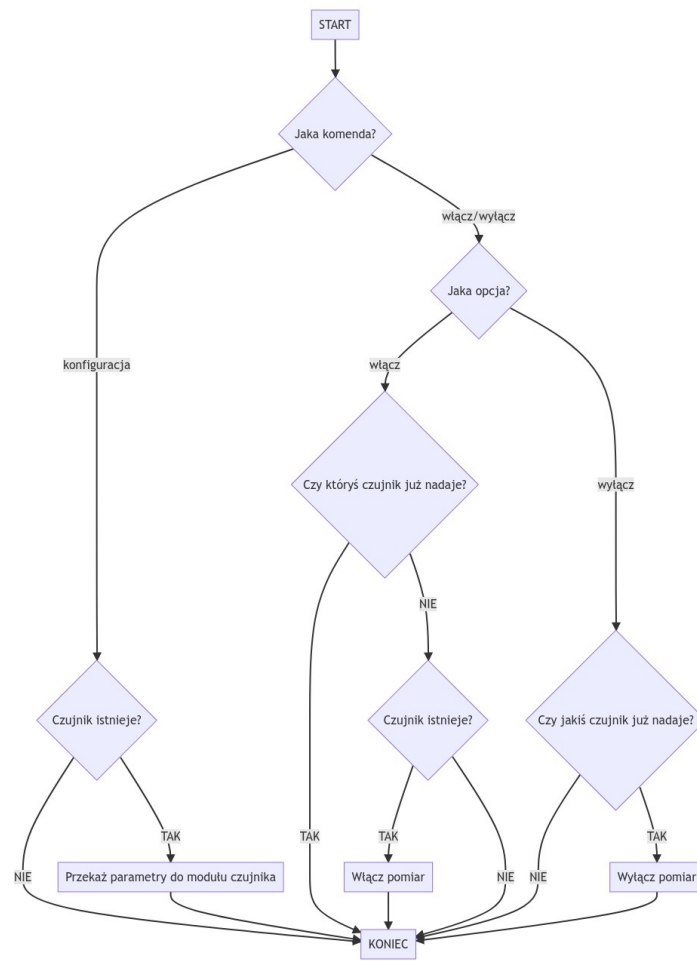


Rysunek 6.3: Inicjalizacja aplikacji - Dodatek A.1

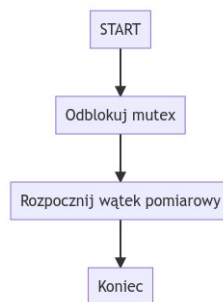


Rysunek 6.4: Główny wątek aplikacji - Dodatek A.1

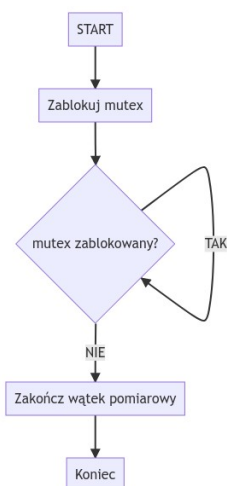
Rysunek 6.5: Wątek `led_task` - A.1Rysunek 6.6: Wątek `send_task` - A.1



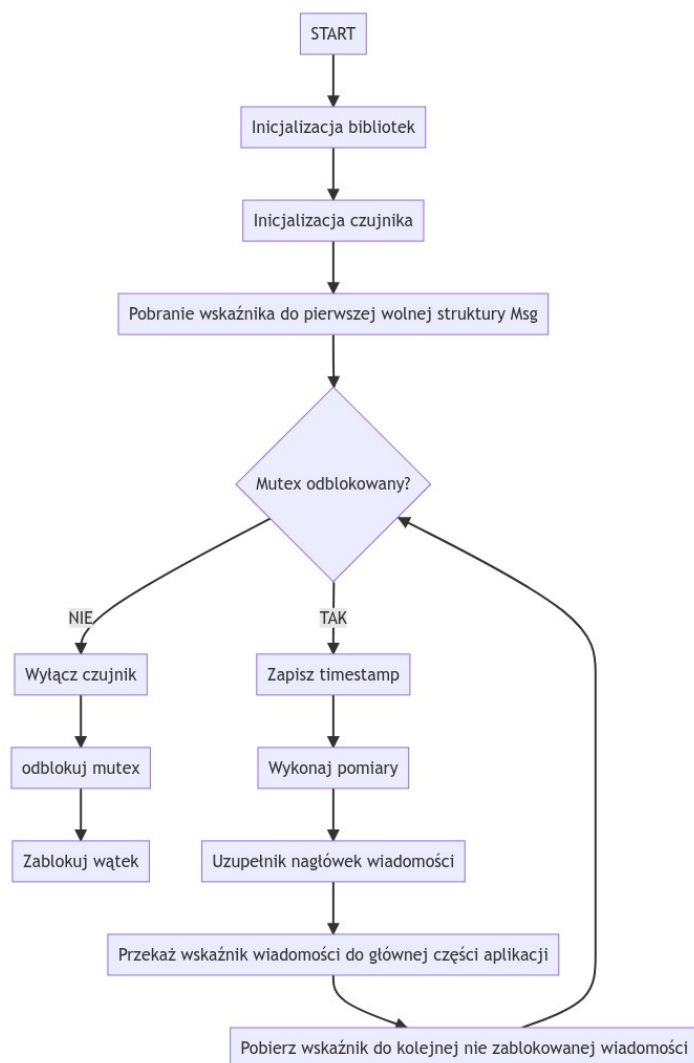
Rysunek 6.7: Funkcja przetwarzająca komendy - A.1



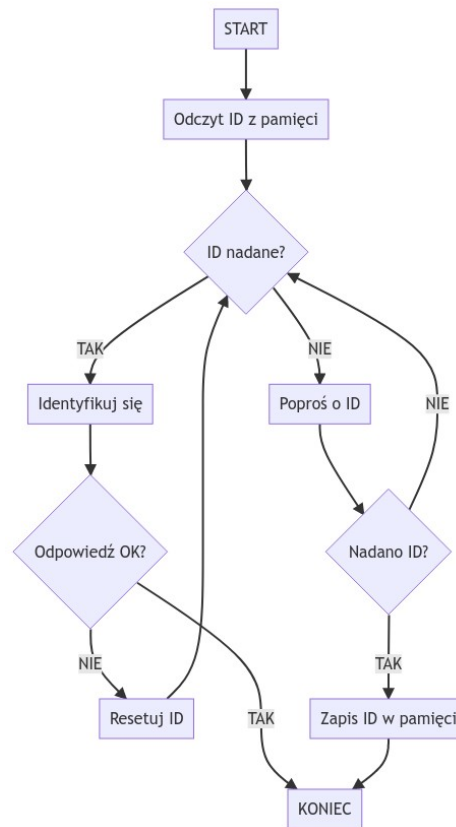
Rysunek 6.8: Funkcja startująca wątek pomiarowy - A.6 i A.8



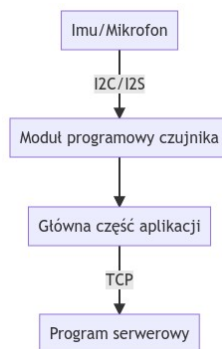
Rysunek 6.9: Funkcja kończąca wątek pomiarowy - A.6 i A.8



Rysunek 6.10: Wątek pomiarowy - A.6 i A.8



Rysunek 6.11: Handshake - A.4



Rysunek 6.12: Przepływ danych

6.4 Debugowanie

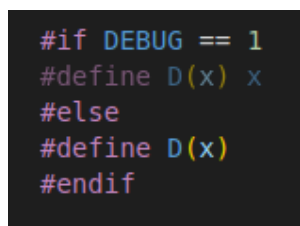
6.4.1 Separacja kodu debugowego od kodu produkcyjnego

Aby umożliwić łatwe przełączanie się pomiędzy kodem produkcyjnym a kodem debugowym. Zostało stworzone makro 6.13 oraz zmienna globalna "DEBUG". Jeżeli zmienna ta ma wartość 0 to kod wewnątrz makra nie jest kompilowany. Jeżeli zmienna ma wartość 1 to kod wewnątrz makra jest kompilowany. W ten sposób można łatwo

przełączać się pomiędzy kodem produkcyjnym a kodem debugowym. W trybie debugowym można podglądać logi poprzez REPL 6.15 po kliknięciu w Visual Studio Code 6.16. Było to pomocne głównie przy sprawdzaniu poprawności działania funkcji, czy logiki aplikacji. Zaimplementowany kod debugowy pozwala:

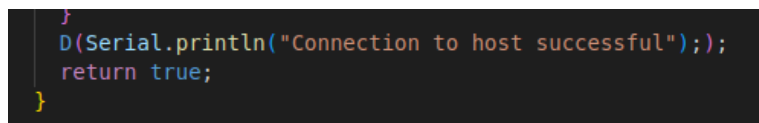
- Sprawdzić IP nadane przez router.
- Status połączenia z siecią Wi-Fi.
- Status połączenia z serwerem.
- Status synchronizacji czasu z serwerem NTP.
- Podejrzec rzebieg komunikacji z serwerem.
- Zobaczyć czy handshake przebiegł poprawnie i z jakim ID pracuje węzeł.
- Zobaczyć stan aplikacji (Stan jałowy/Stan pomiarowy) oraz który czujnik jest aktywny.

Dodatkowo REPL wyświetla informacje o błędach, jeżeli wystąpią.



```
#if DEBUG == 1
#define D(x) x
#else
#define D(x)
#endif
```

Rysunek 6.13: Makro debugowe.

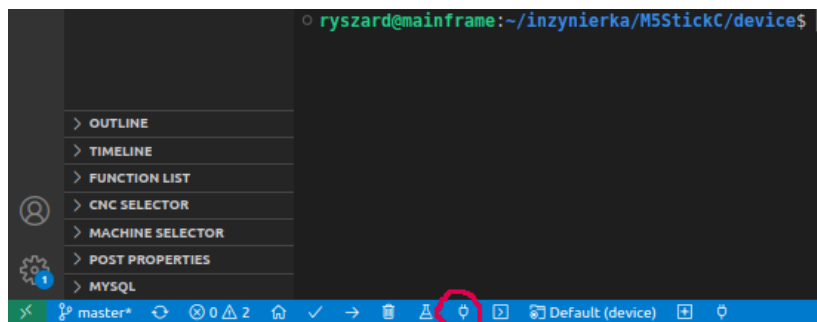


```
}
D(Serial.println("Connection to host successful"));
return true;
}
```

Rysunek 6.14: Sposób użycia makra debugowego. Tekst zostanie wydrukowany tylko wtedy gdy zmienna DEBUG ma wartość 1.

```
---START---
Connecting to Wi-Fi
....
WiFi connected with IP: 192.168.8.107
Trying to get time info from: time.google.com
It could take a while.
Got current time
Connection to host successful
Sending ID: 0h
Received: IdIsAlreadyConnected
Asking for ID
1h
GOT statusOk with ID: ID stored in Flash memory
Send task
run_command
Device parameters
Set device parameters
Setting MPU parameters
run_command
Device parameters
Set device parameters
Setting MIKE parameters
run_command
on_off
Turning on
Device on
MPU on
Starting MPU6886 task
Starting MPU6886 task complete
Initializing MPU
Initializing MPU complete
Initializing I2C
Initializing I2C complete
Resetting MPU complete
Initializing MPU complete
39721444
39721444
Measuring MPU frequency complete
run_command
on_off
Turning off
Device off
MPU6886 off
Stopping MPU6886 task
Sending stop signal to MPU6886
```

Rysunek 6.15: Logi debugowe widoczne poprzez REPL



Rysunek 6.16: Przycisk uruchamiający REPL. Widoczny na dolnym pasku narzędzi w Visual Studio Code.

6.4.2 Debugowanie z użyciem Pythona

W celu debugowania aplikacji został stworzony skrypt w języku Python. Skrypt ten pozwala na wysyłanie komend do aplikacji. Pozwala on również na wyświetlenie danych pomiarowych na animowanych wykresach, w czasie rzeczywistym. Ponieważ aplikacja węzłów powstawała równolegle do aplikacji serwerowej, było to istotne narzędzie, pozwalające sprawdzać poprawność działania aplikacji węzłów niezależnie od statusu prac nad aplikacją serwerową. Na poniższych rysunkach przedstawiono screen z działania

skryptu.

Środowisko na którym pracowano:

- Komputer z systemem Linux Ubuntu 22.04
- Zainstalowany Python w wersji 3.9.14
- Dostęp do sieci internet (do pobrania bibliotek)

Aby skonfigurować środowisko debugowe należy otworzyć terminal w folderze "M5StickC/device/test".

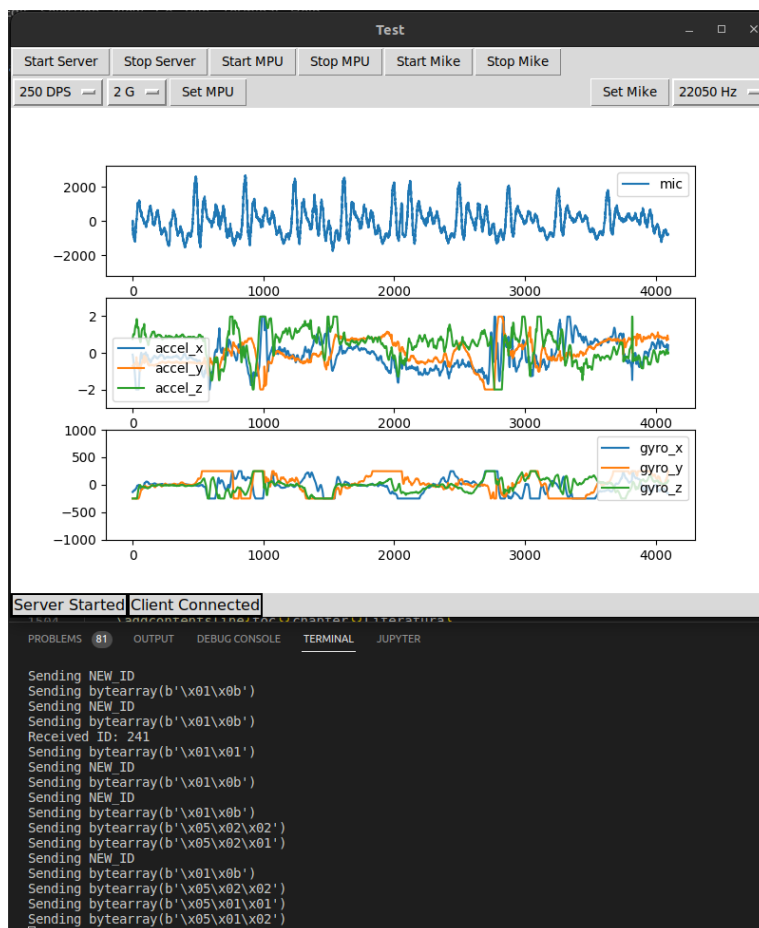
A następnie uruchomić poniższe komendy:

- `sudo apt install python3-tk` (środowisko graficzne, wymagane do wyświetlania wykresów)
- `python3.9 -m venv venv` (środowisko wirtualne)
- `source venv/bin/activate` (aktywacja środowiska wirtualnego)
- `pip install -U pip` (aktualizacja pip)
- `pip install -r requirements.txt` (instalacja bibliotek)

Aby uruchomić środowisko debugowe należy otworzyć terminal w folderze "data_test".

A następnie uruchomić poniższą komendy:

- `source venv/bin/activate`
- `python test_tool.py`



Rysunek 6.17: Na górze - prezentacja pomiarów z mikrofonu, żyroskopu, oraz akcelero-
metru. Na dole - logi debugowe.

6.4.3 Debugowanie z użyciem Wiresharka

Podczas integracji systemu z serwerem, zastosowano narzędzie Wireshark do debugowania komunikacji. Wireshark to wiodąca na świecie aplikacja pozwalająca na podglądanie protokołów sieciowych na niskim poziomie. Jest to projekt zapoczątkowany przez Geralda Combsa w 1998 roku i do tej pory kontynuowany przez wielu ekspertów z całego świata. [47]

Wireshark pozwala na podglądanie komunikacji w czasie rzeczywistym. Dzięki temu mogliśmy zobaczyć czy pakiety są gubione, czy są wysyłane w odpowiedniej kolejności i formacie. Pozwoliło nam to również znaleźć i usunąć błąd w działaniu systemu.

Błędy "TCP window full" oraz "TCP ZeroWindow" 6.20 oznaczają że bufor TCP odbiorcy został zapelniony oraz długość okna TCP zostaje ustawiona na zero. Z reguły dzieje się tak w przypadku, gdy odbiorca nie jest w stanie odebrać danych w odpowiednim tempie [48]. W naszym przypadku również tak było. Rozwiązaniem problemu było zmniejszenie architektury programu serwera z synchronicznej na asynchroniczną, oraz zmiana formatu zapisu danych z MySQL na pliki .csv.



Rysunek 6.18: Logo aplikacji Wireshark

Rysunek 6.19: Ramka danych pomiarowych z IMU widoczna w aplikacji Wireshark. Aby uzyskać logi należy uruchomić aplikację, wybrać sieć w której pracują czujniki i kliknąć interesujący nas pakiet. Dodatkowo możemy przefiltrować pakiety, np po adresie IP, wpisując w polu filtra (zielony pasek) "ip.address == adres IP serwera, lub czujnika".

192.168.8.100	192.168.8.112	TCP	1490	62697 → 2137 [PSH, ACK] Seq=990845 Ack=1174 Win=4571 Len=1436
192.168.8.100	192.168.8.112	TCP	1490	[TCP Window Full] 62697 → 2137 [PSH, ACK] Seq=992281 Ack=1174 Win=4571 Len=1436
192.168.8.112	192.168.8.100	TCP	54	[TCP ZeroWindow] 2137 → 62697 [ACK] Seq=1176 Ack=993717 Win=0 Len=0
192.168.8.100	192.168.8.112	TCP	55	[TCP ZeroWindowProbe] 62697 → 2137 [ACK] Seq=993717 Ack=1174 Win=4571 Len=1
192.168.8.112	192.168.8.100	TCP	54	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 2137 → 62697 [ACK] Seq=1176 Ack=993717 Win=0 Len=0
192.168.8.100	192.168.8.112	TCP	55	[TCP ZeroWindowProbe] 62697 → 2137 [ACK] Seq=993717 Ack=1174 Win=4571 Len=1
192.168.8.112	192.168.8.100	TCP	54	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 2137 → 62697 [ACK] Seq=1176 Ack=993717 Win=0 Len=0
192.168.8.112	192.168.8.100	TCP	56	[TCP ZeroWindow] [TCP Retransmission] 2137 → 62697 [PSH, ACK] Seq=1174 Ack=993717 Win=0 Len=2
192.168.8.100	192.168.8.112	TCP	55	[TCP ZeroWindowProbe] 62697 → 2137 [ACK] Seq=993717 Ack=1174 Win=4571 Len=1
192.168.8.112	192.168.8.100	TCP	54	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 2137 → 62697 [ACK] Seq=1176 Ack=993717 Win=0 Len=0

Rysunek 6.20: Debugowanie błędów związanych z wydajnością systemu we wczesnych etapach projektu, za pomocą aplikacji Wireshark.

6.5 Modułowość i skalowalność

Jednym z założeń przyświecających projektowi było zapewnienie modułowości i skalowalności systemu. Aby to osiągnąć jako część sprzętową wykorzystano moduł pozwalający na wygodne podłączanie kolejnych czujników poprzez wbudowane złącze 2.54mm. Mnogość dostępnych protokołów komunikacyjnych w ESP32 pozwala na łatwe dopasowanie się do potrzeb każdego projektu. Ciekawym rozwiązaniem jest możliwość podłączenia modułu do sieci CAN [29], dzięki czemu można wykorzystać istniejące rozwiązania komunikacyjne w systemach pojazdów.

Od strony programowej, aby dołożyć do projektu nowy czujnik, wystarczy stworzyć moduł programowy który spełnia wymagania określone w wypracowanym w ramach projektu interfejsie. Dodać numer czujnika do struktury "Devices" w pliku "globals.hpp" (Dodatek A.3) oraz zarejestrować w głównej części aplikacji w pliku "main.cpp" (Dodatek A.1) Funkcję startującą wątek pomiarowy 6.22, zatrzymującą wątek pomiarowy 6.23 oraz ustawiającą parametry modułu 6.24 według zamieszczonych w kodzie przykładów. Aplikacja główna wraz z dwoma czujnikami użytymi w projekcie zajmuje 12.2% pamięci RAM oraz 53.5% pamięci FLASH 6.21. Co jest zadowalającym wynikiem, ponieważ pozostawia dużo miejsca na nowe funkcjonalności.

```
RAM:  [=      ] 12.2% (used 40124 bytes from 327680 bytes)
Flash: [=====] 53.5% (used 701357 bytes from 1310720 bytes)
```

Rysunek 6.21: Część logu kompilacji mówiąca o zajętości pamięci.

```
bool device_on(byte device, byte *response)
{
    D(Serial.println("Device on"));
    response[1] = Responses::COMMAND_OK;
    switch(device)
    {
        case Devices::MPU6886:
            D(Serial.println("MPU on"));
            return start_MPU6886_task(&conn);
        case Devices::MIKE:
            D(Serial.println("Device on: Mike"));
            return start_Mike_task(&conn);
        // Here add other devices
        // case Devices::Example:
        //     D(Serial.println("Device on: Example"));
        //     return start_Example_task(&conn);
        default:
            response[1] = Responses::COMMAND_UNKNOWN;
            return false;
    }
}
```

Rysunek 6.22: Rejestracja funkcji uruchamiających wątki pomiarowe. A.1

```
bool device_off(byte device, byte *response)
{
    D(Serial.println("Device off"));
    response[1] = Responses::COMMAND_OK;
    switch(device)
    {
        case Devices::MPU6886:
            D(Serial.println("MPU6886 off"));
            return stop_MPU6886_task();
        case Devices::MIKE:
            D(Serial.println("MIKE off"));
            return stop_Mike_task();
        // Here add other devices
        // case Devices::Example:
        //     D(Serial.println("Example off"));
        //     return stop_Example_task();
        default:
            response[1] = Responses::COMMAND_UNKNOWN;
            return false;
    }
}
```

Rysunek 6.23: Rejestracja funkcji kończących wątki pomiarowe. A.1

```
bool set_device_parameters(byte device, byte *params, int *param_len)
{
    D(Serial.println("Set device parameters"));
    switch (device) {
        case Devices::MPU6886:
            D(Serial.println("Setting MPU parameters"));
            return MPU::set_parameters(params, param_len);
        case Devices::MIKE:
            D(Serial.println("Setting MIKE parameters"));
            return Mike::set_parameters(params, param_len);
        // Here add other devices
        // case Devices::Example:
        //     D(Serial.println("Setting Example parameters"));
        //     return Example::set_parameters(params, param_len);
        default:
            D(Serial.println("Command: 0x?? - (Unknown)"));
            return false;
    }
}
```

Rysunek 6.24: Rejestracja funkcji ustawiających parametry modułów pomiarowych. A.1

Rozdział 7

Testy systemu pomiarowego

7.1 Proces zbierania danych

Do testów IMU użyliśmy czterech połączonych ze sobą mechanicznie węzłów. Pierwszy test polegał na umieszczeniu czujników wewnątrz rolki, która została potoczona, po czasie gwałtownie zatrzymana i potoczona w przeciwnym kierunku. Doświadczenie pokazano na rysunku 7.3,

Do testów mikrofonu użyto czterech węzłów ustawionych jeden na drugim, co widać na rys. 7.1, 7.2. Aby zmierzyć synchronizację odczytów mikrofonu, nagrano serię klaśnień.

Każdy z pomiarów przebiegał podobnie. Węzły zostały ustawione w odpowiednim miejscu, a następnie rozpoczęto nagrywanie danych. Pomiary były wykonywane w odległości około trzech metrów od punktu dostępowego Wi-Fi, bez żadnych zakłócających sygnał przeszkód fizycznych i w środowisku mieszkalnym, gdzie występują inne sygnały sieci Wi-Fi.



Rysunek 7.1: Urządzenia świecą światłem ciągłym, sygnalizując, że połączenie z serwerem nie zostało jeszcze nawiązane.



Rysunek 7.2: Pomiar dźwięku kłaśnięcia



Rysunek 7.3: Pomiar prędkości kątowej podczas toczenia rolki.

7.2 Przygotowania do pomiarów

Zebranie pomiarów wymaga uruchomienia węzłów oraz aplikacji serwerowej. Węzły przed uruchomieniem muszą znać adres IP serwera, nazwę sieci oraz hasło do niej. Aby wgnać te informacje, należy przekompilować działający na nich program. Uruchomienie serwera odbywa się za pomocą aplikacji docker-compose, która eliminuje konieczność instalowania bibliotek bezpośrednio na komputerze. Docker-compose tworzy kontenery, na których instaluje się odpowiednie narzędzia oraz na ich podstawie buduje się projekt.

7.2.1 Konfiguracja i kompilacja aplikacji węzłów

Środowisko programistyczne

Podczas pracy nad projektem używaliśmy środowiska programistycznego:

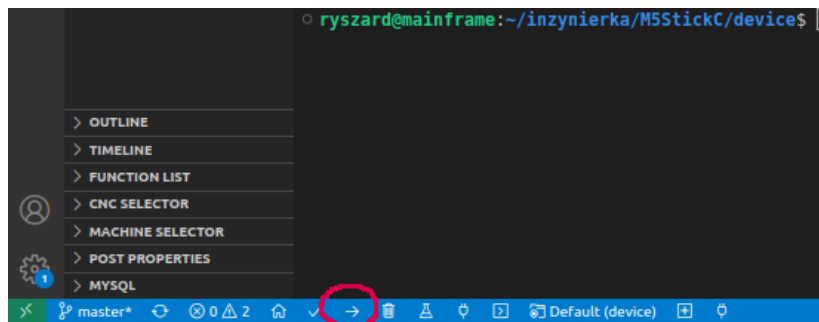
- Komputer z systemem Linux Ubuntu 22.04
- Zainstalowany Python 3.9.14
- Dostęp do sieci internet (do pobrania bibliotek)

- Kompilator gcc 11.2.0
- Visual Studio Code 1.70.20 z zainstalowanym dodatkiem PlatformIO
- Kabel USB-C do programowania węzłów

Proces kompilacji

- Należy otworzyć Visual Studio Code w folderze "M5StickC/device"
- Podłączyć węzeł do komputera (Port USB jest wykrywany automatycznie)
- Kliknąć przycisk "PlatformIO: Upload" 7.4 w lewym dolnym rogu

Wywołanie komendy "PlatformIO: Upload" spowoduje pobranie zależności projektu na bazie pliku konfiguracyjnego A.10, a następnie skompilowanie i wgranie programu na węzeł. 7.5



Rysunek 7.4: Przycisk uruchamiający REPL

[illegible]

Rysunek 7.5: Log kompilacji aplikacji węzłów

Uruchomienie urządzeń

Po wgraniu programu na węzeł, należy go uruchomić. Uruchomienie odbywa się za pomocą przycisku na boku obudowy (rys. 7.6). Należy przytrzymać przycisk wciśnięty przez 2 sekundy aby się uruchomił, przez 8 sekund aby się wyłączył. Po uruchomieniu węzeł będzie świecił światłem ciągłym aż do momentu połączenia z serwerem (rys. 7.1). Następnie przejdą w stan jałowy, sygnalizując to pojedynczymi mrugnięciami diody. W tym stanie węzeł czeka na polecenia od serwera. Jeżeli serwer wyda komendę "start", węzeł rozpocznie pomiar i będzie sygnalizował ten stan za pomocą podwójnych mrugnięć.



Rysunek 7.6: Przycisk zasilania węzła

7.2.2 Konfiguracja i kompilacja aplikacji serwera

Testy przeprowadzono z użyciem aplikacji serwera z następującymi wersjami narzędzi:

- Go v1.18 (w kontenerze)
- Chrony v1.18 (w kontenerze)
- Vue v3.2.39 (panel użytkownika, plik ze skryptem)
- axios.js v0.27.2 (panel użytkownika, plik ze skryptem)
- Gin v1.7.7 (w kontenerze)
- Godotenv v1.4.0 (w kontenerze)
- logrus v1.8.1 (w kontenerze)

- `docker-compose 2.10.2` (na systemie hosta)

Wszystkie podane wyżej narzędzia oprócz `docker-compose` są zapisane przez autorów pracy w plikach konfiguracyjnych projektu, dlatego nie występuje potrzeba aby pobierać je manualnie, gdy projekt zostanie uruchomiony zgodnie z zaleceniami. `Docker-compose` jest natomiast niezbędny do uaktywnienia aplikacji. Oprogramowanie serwerowe uruchomiono na komputerze z zainstalowanym systemem operacyjnym GNU/Linux Ubuntu 22. Należy podkreślić, że gwarantuje się, iż program będzie działał w systemach GNU/Linux z zainstalowanymi wyżej wymienionymi narzędziami, jednak nie ma gwarancji, iż będzie działał bezproblemowo na systemach z rodziny Windows czy MacOS. Dostęp do sieci internet jest niezbędny do poprawnego uruchomienia projektu po raz pierwszy.

Aby uruchomić aplikację serwerową należy w katalogu serwera uruchomić polecenie `docker-compose up`. W efekcie tego polecenia zostaną utworzone kontenery, a na nich pobrane i zbudowane odpowiednie zależności, zostanie uruchomione oprogramowanie serwerowe oraz serwer NTP. Całość może zająć od jednej do trzech minut, w zależności od mocy komputera. Pomyślne uruchomienie aplikacji sygnalizuje pojawienie się logów aplikacji na ekranie terminala. Proces uruchomienia przedstawiono na rys. 7.7. Od tej pory w tym terminalu będą się pojawiały logi związane z działaniem aplikacji: połączenia HTTP, wysyłanie komend, rozpoczęcie i zakończenie pomiarów, połączenie i zerwanie połączenia z węzłami wraz z ich identyfikatorami.

```
[david@david-pc server]$ docker-compose up
[*] Running 4/4
# ntp Pulled
# 59b1c3509f3 Pull complete
# 7b50b2aedb4 Pull complete
# 61038a069dce Pull complete
[*] Building 36.6s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 312B
=> [internal] load .dockerignore
=> => transferring context: 46B
=> [internal] load metadata for docker.io/library/golang:1.18
=> [1/6] FROM docker.io/library/golang:1.18sha256:040bc098888e2df09499c73156f3f869d843b7033d11b465fee7f1a358a494a
=> => resolve docker.io/library/golang:1.18sha256:040bc098888e2df09499c73156f3f869d843b7033d11b465fee7f1a358a494a
=> => sha256:040bc098888e2df09499c73156f3f869d843b7033d11b465fee7f1a358a494a 2.36kB / 2.36kB
=> => sha256:ebd59a91af0c61cf28a22a9651ff45612848bdcc1a43c2c3063825ba2e613fc 1.80kB / 1.80kB
=> => sha256:3261452ade5c33897eba4ba88a24bd77a93a3d9944d4c39b936482655e664857 5.16MB / 5.16MB
=> => sha256:a42021c14fb31c4aa253283e7f8e34fc3b15069ce370f1223fbb4252a64202 10.80MB / 10.80MB
=> => sha256:f81baf819d5b520ab919b42e5925c4b32b62241fcd036036d9bb5aa2f522d1 7.10kB / 7.10kB
=> => sha256:23858da423a6737f0467fab0014e5b53009ea7405d575636af0c3f100bb2f10 55.03MB / 55.03MB
=> => sha256:8471b75885efc7790a16be5328e3b368567b76a60fc3feabd6869c15e175ee05 54.58MB / 54.58MB
=> => sha256:b7aa120d0d29fa75bb50861103f7837514028eaf06e3e821b8c47c2c10d386 85.96MB / 85.96MB
=> => sha256:5cc67fe144b8a2d33963edb62297bdaa3f968746861866b48db3f3125e68734 141.88MB / 141.88MB
=> => sha256:3261452ade5c33897eba4ba88a24bd77a93a3d9944d4c39b936482655e664857 5.16MB / 5.16MB
=> => sha256:a42021c14fb31c4aa253283e7f8e34fc3b15069ce370f1223fbb4252a64202 10.80MB / 10.80MB
=> => sha256:ebd59a91af0c61cf28a22a9651ff45612848bdcc1a43c2c3063825ba2e613fc 1.80kB / 1.80kB
=> => sha256:3261452ade5c33897eba4ba88a24bd77a93a3d9944d4c39b936482655e664857 5.16MB / 5.16MB
=> => sha256:b7aa120d0d29fa75bb50861103f7837514028eaf06e3e821b8c47c2c10d386 85.96MB / 85.96MB
=> => sha256:5cc67fe144b8a2d33963edb62297bdaa3f968746861866b48db3f3125e68734 141.88MB / 141.88MB
=> => sha256:cd3416ecab879296d41843948ae859cadb4bd4300a60b23abef443645845ab4a 156B / 156B
=> => sha256:cd3416ecab879296d41843948ae859cadb4bd4300a60b23abef443645845ab4a 156B / 156B
=> => sha256:b7aa120d0d29fa75bb50861103f7837514028eaf06e3e821b8c47c2c10d386 85.96MB / 85.96MB
=> => sha256:5cc67fe144b8a2d33963edb62297bdaa3f968746861866b48db3f3125e68734 141.88MB / 141.88MB
=> => sha256:cd3416ecab879296d41843948ae859cadb4bd4300a60b23abef443645845ab4a 156B / 156B
=> [internal] load build context
=> => transferring context: 2.10MB
=> [2/6] WORKDIR /app
=> [3/6] COPY go.mod go.sum ./
=> [4/6] RUN go mod download && go mod verify
=> [5/6] COPY . .
=> [6/6] RUN go build -v -o /usr/local/bin/app .
=> exporting to image
=> exporting layers
=> writing image sha256:a03c7cd1c8e0b73762b8ef980ca53f126e415546b9504538da29e200a96eb6
=> naming to docker.io/library/server-app
[*] Running 3/3
# Network server_default Created
# Container server-app-1 Created
# Container server-ntp-1 Created
Attaching to server-app-1, server-ntp-1
server-ntp-1 | 2022-10-01T13:51:13Z chronyd version 4.1 starting (+CMDMON +NTP +REFCLOCK +RTC +PRIVDROP +SCFILTER +SIGND +ASYNCDNS +NTS +SECHASH +IPV6 -DEBUG)
server-ntp-1 | 2022-10-01T13:51:13Z Disabled control of system clock
server-ntp-1 | 2022-10-01T13:51:13Z Could not read valid frequency and skew from driftfile /var/lib/chrony/chrony.drift
server-app-1 | time="13:51:13.714" level=info msg="[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached."
server-app-1 | time="13:51:13.714" level=info msg="[GIN-debug] [WARNING] Running in \"debug\" mode. Switch to \"release\" mode in production."
server-app-1 | time="13:51:13.714" level=info msg=" - using env:VTEXPORT GIN_MODE=release"
server-app-1 | time="13:51:13.714" level=info msg=" - using code:\\tgin.SetMode(gin.ReleaseMode)"
server-app-1 | time="13:51:13.714" level=info msg="[GIN-debug] GET /assets/*filepath --> github.com/gin-gonic/gin.(*RouterGroup).createStaticHandler.func1 (3 handlers)"
server-app-1 | time="13:51:13.714" level=info msg="[GIN-debug] HEAD /assets/*filepath --> github.com/gin-gonic/gin.(*RouterGroup).createStaticHandler.func1 (3 handlers)"
server-app-1 | time="13:51:13.714" level=debug msg="Created TCP server" port=7122
server-app-1 | time="13:51:13.715" level=info msg="[GIN-debug] [WARNING] Since SetHTMLTemplate() is NOT thread-safe. It should only be called"
server-app-1 | time="13:51:13.715" level=info msg="at initialization, ie. before any route is registered or the router is listening in a socket:"
server-app-1 | time="13:51:13.715" level=info msg="\\trouter := gin.Default()"
server-app-1 | time="13:51:13.715" level=info msg="\\trouter.SetHTMLTemplate(template) // << good place"
server-app-1 | time="13:51:13.715" level=info msg="[GIN-debug] GET / --> server/serving.Gut (3 handlers)"
server-app-1 | time="13:51:13.715" level=info msg="[GIN-debug] GET /sensors --> server/serving.Sensors (3 handlers)"
server-app-1 | time="13:51:13.715" level=info msg="[GIN-debug] POST /command --> server/serving.Command (3 handlers)"
server-app-1 | time="13:51:13.715" level=info msg="[GIN-debug] Listening and serving HTTP on :8001"
server-ntp-1 | 2022-10-01T13:51:20Z Selected source 162.159.200.123 (time.cloudflare.com)
```

Rysunek 7.7: Pomyślne uruchomienie aplikacji serwera

Aplikacja zapisuje pomiary do pliku binarnego w katalogu `data/` w katalogu serwera. Aplikacja zapisuje dane do czasu, aż rozmiar pliku przekroczy 2 GB - wtedy tworzy kolejny, inkrementując liczbę w nazwie pliku.

Kontrola czujników odbywa się za pomocą panelu użytkownika, dostępnego po uruchomieniu aplikacji pod adresem `http://localhost:8081/`. Należy wybrać ID węzła, po czym można dobrać konfigurację czujników w tym węźle i wybrać status dla każdego czujnika - włączony lub wyłączony. Nie jest możliwe włączenie dwóch czujników na raz, należy wybrać tylko jeden z nich.

Dodatkowo, w katalogu `monitoring/` znajduje się aktualizowany co sekundę podczas działania serwera plik `report`, zawierający dane na temat przepływu informacji wewnątrz aplikacji. Po zakończeniu pomiarów warto zajrzeć do tego pliku aby mieć pewność, że żadne informacje nie zostały zgubione czy pominięte przez aplikację serwerową. Najlepszym wskaźnikiem jest porównanie dwóch wielkości: "Bytes written" oraz "Expected bytes written, based on receiver data", z czego ta pierwsza to licznik bajtów zapisanych do pliku, a druga to suma estymat rozmiaru danych zdekodowanych pomiarów, dokonywanych zaraz po otrzymaniu danych od czujnika. Jeżeli obie wielkości są jednakowe to serwer zapisał taką liczbę danych, jaką oczekiwano i żadna informacja nie została zagubiona. Przykładowy raport z pliku został przedstawiony na rys. 7.8.

```

Co 1,0s: cat server/monitoring/report

Monitoring status for Oct  1 13:37:08.671
=====
Parse queue len:      0
Interpreter queue len: 0
Monitoring queue len: 0

Avg time spent on receiving: 29.421µs
Avg time spent on interpreting: 604.105µs
Avg time spent on parsing: 364.966µs
Avg time spent on saving: 0s

Buffered bytes:      8028168

Bytes received:      3723536
  Since last measure: 103352
Bytes received by interpreter: 3723536
Bytes written:      16064076
  Expected bytes written, based on receiver data: 20810712

Data structures created by interpreter: 1606078
  Of which:
    MIC: 1584720
    IMU: 21358
Data structures expected to be created based on receiver data: 1606078
  Of which:
    MIC: 1584720
    IMU: 21358
Data structures sent by interpreter: 1606078
  Of which:
    MIC: 1584720
    IMU: 21358
Data structures received by file saver: 1606078
  Of which:
    MIC: 1584720
    IMU: 21358

```

Rysunek 7.8: Raport z działania aplikacji serwera, odczytany podczas zbierania odczytów z czujników.

Narzędzie do parsowania plików binarnych

Pliki zapisywane przez serwer w katalogu `server/data` są zapisywane w formacie binarnym. Aby można było je przetwarzać i analizować, należy najpierw przekonwertować je na format CSV. Format CSV prezentuje dane w formie tabelarycznej, oddzielone przecinkami [66].

Do projektu został załączony kod programu konwertera. Należy skompilować go używając wersji Go v1.18 (w tym przypadku bez użycia kontenera, należy zainstalować kompilator na komputerze ręcznie[67]) i zbudować program przez polecenie: `go build -o decoder main.go`, uruchomione w katalogu `file_decoder`, gdzie znajduje się kod programu. W efekcie w tym katalogu pojawi się plik wykonywalny `decoder`, którego można użyć do przetworzenia pliku binarnego na CSV przez polecenie: `./decoder < plik_binarny_z_serwera > plik_wyjsciowy.csv`. Plik binarny z serwera podany na wejście programu zostanie przekonwertowany i zapisany jako `plik_wyjsciowy.csv`, który można poddać dalszej analizie.

7.3 Uruchomienie środowiska do analizy danych

Środowisko na którym pracowano:

- Komputer z systemem Linux Ubuntu 22.04

- Zainstalowany Python w wersji 3.9.14
- Dostęp do sieci internet

7.3.1 Konfiguracja

Aby skonfigurować środowisko do analizy danych należy otworzyć terminal w folderze "data_tests". A następnie uruchomić poniższe komendy:

- `sudo apt install python3-tk`
- `python3.9 -m venv venv`
- `source venv/bin/activate`
- `pip install -U pip`
- `pip install -r requirements.txt`

7.3.2 Uruchomienie

Aby uruchomić środowisko do analizy danych należy otworzyć terminal w folderze "data_test". A następnie uruchomić poniższą komendę:

```
source venv/bin/activate
```

7.4 Analiza danych

W analizie danych skupiono się przede wszystkim na aspektach synchronizacji czasowej odczytów z czujników. Nie dokonano analizy poprawności danych zebranych przez czujniki ponieważ nie mieści się to w zakresie tej pracy. Poprawność danych określono jedynie wizualnie, na podstawie wygenerowanych przez skrypty wykresów przebiegów oraz poprzez przekonwertowanie pomiarów z mikrofonu do pliku dźwiękowego.

7.4.1 Skrypty do wstępnego przetwarzania danych

Aby usprawnić proces analizy danych zostały stworzone skrypty do wstępnego przetwarzania danych, których instrukcje wywołania podano w tab. 7.1. Te oraz pozostałe użyte przez nas skrypty zostały umieszczone w dodatku C.

splitter.py

Skrypt dzieli dane z pliku .csv na pomniejszych pliki .csv zawierające odczyty z pojedynczych węzłów oraz według ustawień w pliku "settings.py". Aby dodać nowy typ danych w zestawie testowym, w ustawieniach należy dodać klasy typu T reprezentujące typ danych, jak przedstawiono na rys. 7.10. Następnie należy dodać je do listy.

Komenda	Pierwszy argument	Drugi argument	Trzeci Argument
python splitter.py	nazwa folderu	nazwa folderu	-
python union.py	ścieżka pliku	ścieżka pliku	ścieżka pliku
python cut_by_time.py	nazwa pliku usatawień	-	-
python printer.py	nazwa folderu	nazwa folderu	-

Tabela 7.1: Zestawienie komend wykonujących skrypty

Możliwe jest utworzenie większej ilości zestawów danych i modyfikowanie jedynie listy w ustawieniach według potrzeb, co przedstawiono na rys. 7.9.

Dla każdego pliku w folderze z argumentu pierwszego, typu danych T, oraz każdego węzła N. Zostanie utworzony plik "T_N_nazwa_oryginalna_pliku.csv" w folderze z argumentu drugiego. Plik ten będzie zawierał przefiltrowane dane z pliku oryginalnego. C.1

```
SENSORS:List[T] = [Imu, Mic]
```

Rysunek 7.9: Lista używanych obecnie klas ustawień.

```
class T(str, Enum):
    @classmethod
    def as_list(cls) -> List[str]:
        return [t.value for t in cls]

class Imu(T):
    x_dps = "x-dps"
    y_dps = "y-dps"
    z_dps = "z-dps"
    x_g = "x-g"
    y_g = "y-g"
    z_g = "z-g"

class Mic(T):
    volume = "volume"
```

Rysunek 7.10: Klasy typu T.

union.py

Skrypt łączy dane z plików .csv z dwóch pierwszych argumentów i zapisuje je w pliku z trzeciego argumentu. Skrypt załączony jako C.2.

cut_by_time.py

Skrypt wycina dane w wyznaczonych przedziałach czasowych z wybranych plików i zapisuje je pod nowymi nazwami. Określenie parametrów transformacji odbywa się poprzez podanie jako argument pliku "*.py" z ustawieniami. Każdy taki plik powinien

zawierać listę "FILES" zawierającą znaczniki czasowe według których plik ma być przycięty, nazwę pliku wejściowego, folder wyjściowy oraz nazwę pliku wyjściowego. Dodatkowo musi być zdefiniowany string "time_format" definiujący użyty format znaczników czasowych. Skrypt załączono jako C.4

printer.py

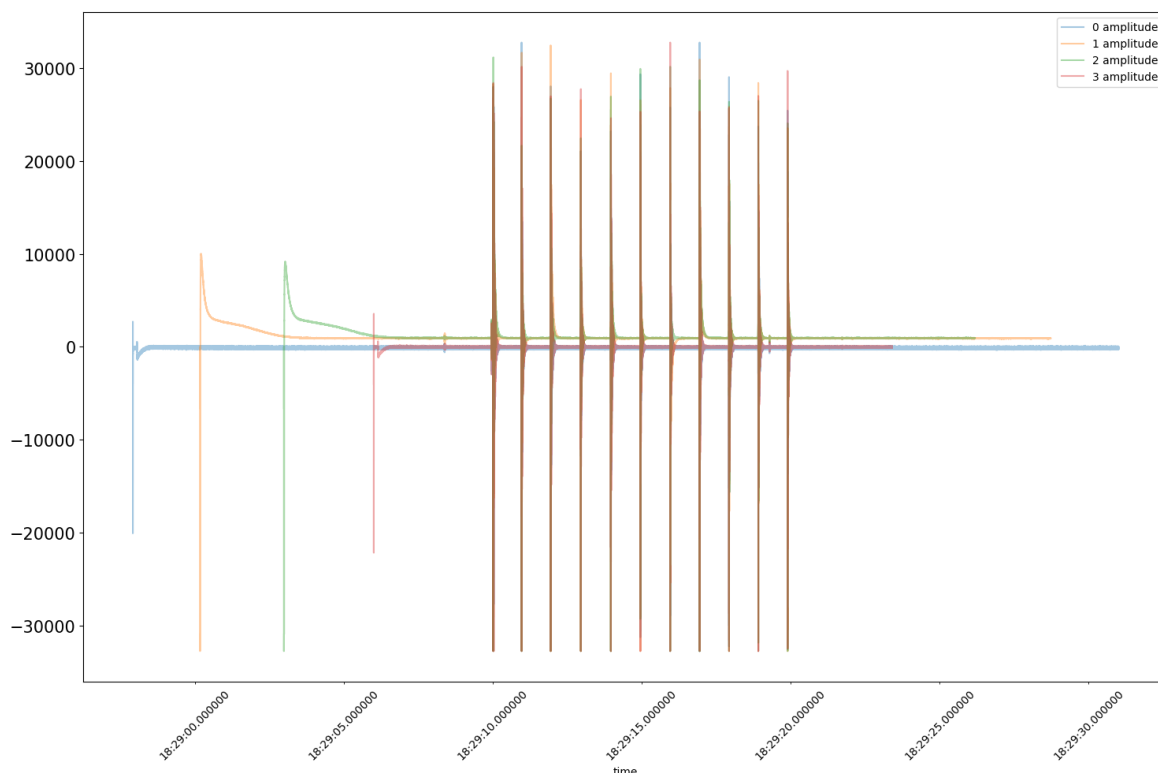
Skrypt drukuje dane z plików .csv z folderu z argumentu pierwszego do folderu z argumentu drugiego. Dane są drukowane w formacie .png, nazwa pliku jest taka sama jak nazwa pliku z danymi. Skrypt załączono jako C.3

7.4.2 Wstępne przetworzenie danych

W celu wstępnego przetworzenia danych, wykonano następujące kroki:

- Pliki z danymi pomiarowymi zostały umieszczone w folderze "data_tests".
- Następnie utworzono wykresy poglądowe dla każdego pliku z danymi komendą "python printer.py input init_images". Wykresy te są widoczne na rys. 7.11 i 7.13. Pozwiliło nam to na określenie jakie dane są nam potrzebne oraz jakie dane są zbędne.
- Na podstawie uzyskanych wykresów utworzono plik ustawień "time_cut_receipt.py". Plik załączono jako C.5.
- Wykonano komendę "python cut_by_time.py time_cut_receipt", co wycięło z plików jedynie przydatne fragmenty i zapisało je pod odpowiednimi nazwami.
- Wykonano komendę "python splitter.py cutted splitted", co rozdzieliło dane z plików na pojedyncze węzły.

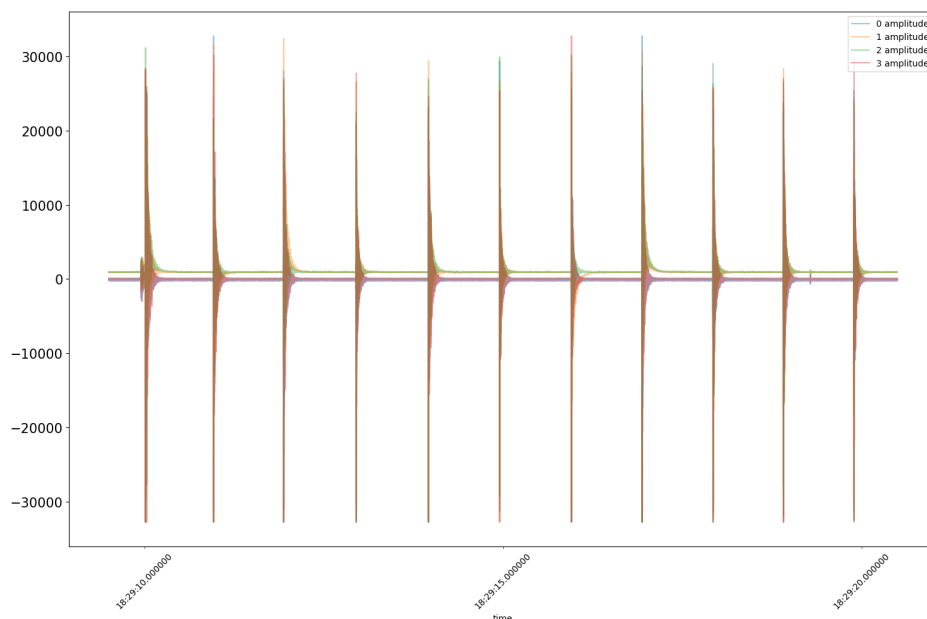
Test mikrofonu - impulsy



Rysunek 7.11: Wizualizacja danych z mikrofonu. Każdy kolor oznacza inny mikrofon.

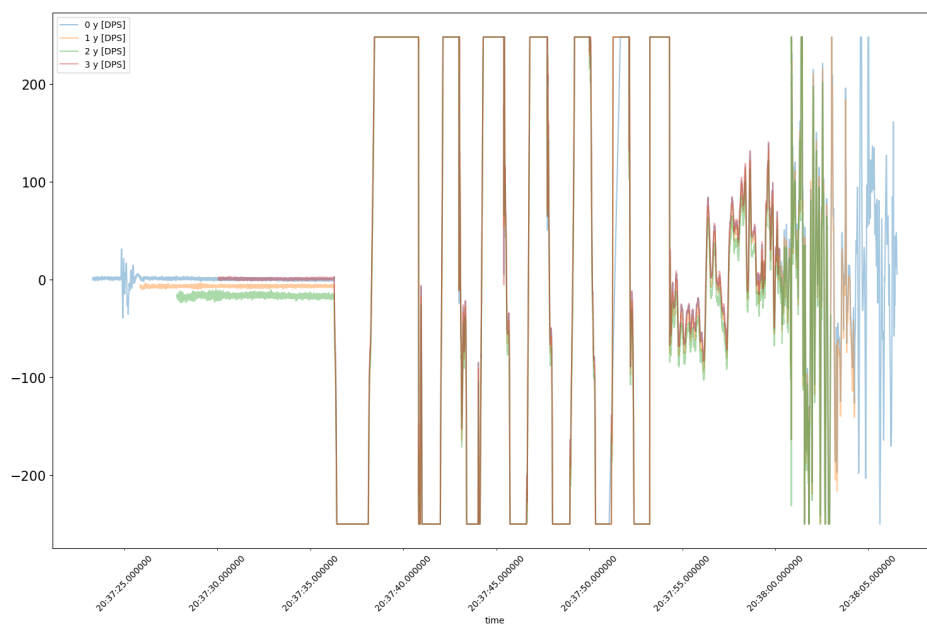
Na rys. 7.11 przedstawiono wykres sporządzony na podstawie pomiarów mikrofonu. Wartości na osi Y tego i wszystkich kolejnych wykresów z pomiarami mikrofonu nie reprezentują żadnych jednostek - są 16-bitową reprezentacją danych wysłanych przez czujnik i stanowią podstawę do cyfrowego odwzorowywania dźwięku zarejestrowanego przez ten czujnik. Na wykresach można zaobserwować "ścięte" wierzchołki amplitud. Jest to bezpośrednia konsekwencja 16-bitowej reprezentacji danych, w której można zapisać wartości liczb całkowitych od -32768 do 32767 i informuje o nasyceniu czujnika. Można zauważyć, że odczyty rozpoczynają się w różnych momentach - to dlatego, że czujniki były włączane po kolei. Przez kilka następnych sekund następuje "cisza" - okres, w którym jedynym słyszalnym dźwiękiem jest szum tła w niewyciszonym otoczeniu. Po ciszy następuje wyraźny impuls. Rozpoczyna on serię, złożoną z jedenastu impulsów. Impulsy są w rzeczywistości zarejestrowanymi kłasnieniami w dłoń. Dalsze stuknięcia i szumy są nieistotne dla tego pomiaru.

Na rys. 7.12 przedstawiono zbliżenie na impulsy. Widać na nim, że każdy czujnik ma różne wartości "zerowe". To zjawisko jest spowodowane niską jakością czujników, a co za tym idzie dużą rozbieżnością w ich wykonaniu i dużą wartością stałą, dodawaną do pomiarów.



Rysunek 7.12: Zbliżenie na serię kłaśnień z pomiarów mikrofonu.

Test IMU - rolka



Rysunek 7.13: Odczyty DPS osi Y w doświadczeniu z rolką.

Na rys. 7.13 przedstawiono odczyty z IMU podczas toczenia. Na wykresie przedstawia się jedynie odczyty z czujników DPS osi Y wokół której mierzono prędkość obrotową, ze względu na to, że ukazanie wszystkich danych zaciemniłoby obraz. Nieuwzględnienie wartości przyspieszeń i DPS dla pozostałych osi nie jest istotne dla badania synchronizacji danych, ponieważ mamy pewność, że wszystkie te pomiary wy-

stępują razem - są przekazywane w jednej paczce, w ramach jednego odczytu IMU i są opatrzone jednakowym znacznikiem czasowym. Na rysunku widać moment rozpoczęcia toczenia, który charakteryzuje się rozpoczęciem przebiegu prostokątnego. Efekt ten jest zamierzony, podczas zmiany kierunku toczenia otrzymujemy wygodny do analizy synchronizacji punkt wejścia w stan nasycenia, który jest użyty do analizy synchronizacji pomiarów. Szum występujący na końcu wykresów wynika z zakończenia doświadczenia i jest nieistotny dla tego pomiaru.

7.4.3 Przetworzenie danych

Do celów przetworzenia danych wybrano odpowiednie dane otrzymane w poprzednim etapie. Następnie napisano i wykonano skrypty przetwarzające dane.

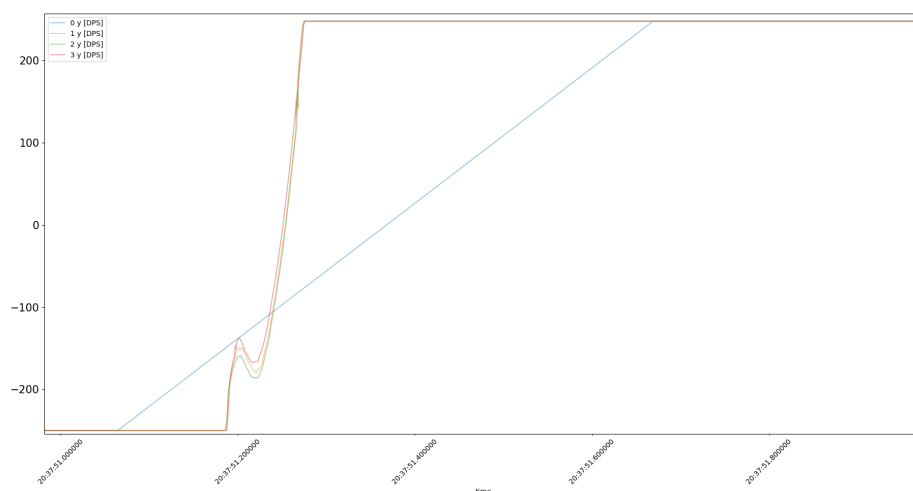
Analiza stabilności pomiarów w czasie

Aby mieć pewność, że dane w czujnikach są wysyłane w sposób stabilny, tj. odstępy czasowe pomiędzy konkretnymi pomiarami nie zmieniają się, a co za tym idzie częstotliwość próbkowania pozostaje stabilna, dokonano analizy stabilności pomiarów w czasie. Wybrano chwile czasu w których różna liczba węzłów wysyła dane jednocześnie, gdy są uruchamiane jeden po drugim na początku pomiaru. Dane zostały uzyskane za pomocą skryptu C.6

Czujnik	IMU				Mikrofon			
Ilość węzłów	1	2	3	4	1	2	3	4
Ilość próbek	25546	48555	69617	87729	1459759	2719298	3742407	4509916
Średnia różnic czasu pomiędzy próbkami [μ s]	1693	1682	1678	1675	22,67	22,67	22,67	22,67
odchylenie standardowe różnic czasu [μ s]	3780	2747	2297	2049	0,46	0,46	0,46	0,46
25% różnic czasu jest w przedziale [μ s]	1718	1718	1718	1718	22	22	22	22
50% różnic czasu jest w przedziale [μ s]	1718	1718	1718	1718	23	23	23	23
75% różnic czasu jest w przedziale [μ s]	1718	1718	1718	1718	23	23	23	23
największy czas pomiędzy próbkami [μ s]	604795	604795	604795	604795	23	23	23	23

Tabela 7.2: Zestawienie wyników pomiarów stabilności czasowej. Częstotliwości próbkowania zostały wyliczone na podstawie średnich różnic czasu pomiędzy próbkami.

Wyniki z tab. 7.2 pokazują, że dla wszystkich czujników różnica czasu pomiędzy kolejnymi próbkami jest w ogólności stała. Pojawia się jednak jedna anomalia w przypadku czujników IMU i wstępuje ona dla jednego z czujników. W pewnym fragmencie pomiarów występuje przerwa, której czas trwania jest zaprezentowany jako największy czas pomiędzy próbkami. Przerwa ta jest widoczna na rysunku 7.13 w szóstym zboczach narastającym - pomiar DPS Y 0 jest w tym miejscu reprezentowany przez idealnie prostą linię, co oznacza brak danych pomiędzy dwoma punktami, w których linia jest zawarta. Zbliżenie na zbocze ukazano na rys 7.14. W przypadku mikrofonu nie zarejestrowano niespodziewanych odchyłeń i długich przerw pomiędzy czasami zarejestrowania próbek co pozwala wnioskować, że nie zawierają one błędów.



Rysunek 7.14: Zbliżenie na zbocze z niepełnymi danymi dla czujnika 0

Analiza dokładności synchronizacji odczytów mikrofonu

Do analizy dokładności synchronizacji odczytów mikrofonu wykorzystano zestaw danych pomiarowych 7.12 oraz skrypt załączony jako C.8. W nagraniu występuje jedenaście kłaśnieć. Synchronizację zmierzono przez określenie czasu pierwszego z serii odczytów przekraczających wartość zarejestrowanej ciszy. Te czasy zestawiono w tabeli 7.3. Ukazany w tabeli czas to ilość sekund po godzinie 18:29, o której rozpoczęto pomiary. Błąd pomiarowy dla zarejestrowanych czasów to $\pm 1\mu s$. Pomiary są zbierane przez mikrofony z szybkością 44100 próbek na sekundę, czyli co około $22,7\mu s$. Na nasze potrzeby wartość została zaokrąglona do $23\mu s$. Jest więc możliwe, że mikrofony rejestrują dźwięk w innym czasie: mikrofon A może zarejestrować próbkę dźwięku pomiędzy dwoma odczytami mikrofonu B. Oczekuje się więc, że przy doskonałym wyeliminowaniu opóźnień komunikacji bezprzewodowej, błąd może wynosić $\pm 1\mu s$ dla odczytu z jednego czujnika, $\pm 1\mu s$ dla odczytu z drugiego czujnika i połowę okresu próbkowania dla różnicy pomiędzy nimi, czyli $\pm 11,5\mu s$. Po zsumowaniu wartości oczekiwany błąd przy idealnym wyeliminowaniu opóźnień komunikacji bezprzewodowej wynosi w zaokrągleniu $\pm 14\mu s$.

Czujnik	Czujnik 0 [s]	Czujnik 1 [s]	Czujnik 2 [s]	Czujnik 3 [s]	Największa różnica [μs]	Najmniejsza różnica [μs]
Impuls 1	10.010931	10.011921	10.012406	10.012749	1818	485
Impuls 2	10.963447	10.964437	10.964923	10.965266	1819	486
Impuls 3	11.940316	11.941352	11.941837	11.942135	1819	485
Impuls 4	12.952014	12.953004	12.953489	12.953788	1774	485
Impuls 5	13.960310	13.961346	13.961786	13.962129	1819	440
Impuls 6	14.950807	14.952182	14.952713	14.952603	1906	421
Impuls 7	15.952595	15.953585	15.954071	15.954369	1774	486
Impuls 8	16.941391	16.942427	16.942912	16.943074	1683	485
Impuls 9	17.928555	17.929069	17.930030	17.930419	1864	961
Impuls 10	18.911274	18.912332	18.912817	18.913138	1864	485
Impuls 11	19.895036	19.896548	19.896965	19.896900	1929	352

Tabela 7.3: Zestawienie czasów zarejestrowania pierwszych próbek rejestrujących impuls

Najmniejsze różnice pomiędzy zarejestrowaniem tego samego wydarzenia przez różne czujniki wynoszą setki mikrosekund, podczas gdy największe nie więcej niż dwa tysiące, czyli 2 ms. Jest to wynik spodziewany: użyta technologia IP, a co za tym idzie protokół NTP nie daje gwarancji uzyskania precyzji większej niż pojedyncze milisekundy. Dla niektórych zastosowań jest to czas zadowalający, jednak przed zastosowaniem projektu jako narzędzia w badaniach w przyszłości należy określić, jaka precyzja synchronizacji jest wymagana.

Analiza plików audio

Na podstawie danych z mikrofonu wygenerowano pliki .wav, aby stwierdzić w sposób empiryczny, czy dane mikrofonowe docierają do serwera w formie niezmienionej i czy fragmenty danych nie są gubione w procesie. Wykorzystano do tego skrypt załączony jako C.7. Nagrany dźwięk dobrze odwzorowywał nawet ciche kliknięcia myszki, jednak przez cały czas nagrania słychać było wyraźny szum. Szum jest prawdopodobnie spowodowany niską jakością wbudowanego w węzeł mikrofonu. Należy pamiętać również, że mikrofon znajduje się wewnątrz urządzenia, więc szum może wynikać również z rejestrowania dźwięku wydawanego przez obwody w urządzeniu. Dźwięk zawierał głośne "kliknięcie" na początku nagrania, co można tłumaczyć zachowaniem kondensatorów w niskiej jakości wbudowanym w węzeł mikrofonie pojemnościowym. W całym nagraniu nie było słychać trzasków ani innych zakłóceń mogących wskazywać na utratę pakietów.

Analiza synchronizacji odczytów IMU

Do analizy dokładności synchronizacji odczytów IMU wykorzystano zestaw danych pomiarowych z rys. 7.13 oraz skrypt załączony jako C.9. Podobnie jak w przypadku mikrofonu, do analizy synchronizacji odczytów IMU wykorzystano przedstawione pomiary, aby określić czas zarejestrowania tych samych zmian w układzie, w tym przypadku czasy wejścia czujników w stan nasycenia pod koniec zbocza narastającego bądź opadającego. Analizę przeprowadzono dla doświadczenia z rolką.

Próbkowanie wielkości w IMU odbywa się z częstotliwością około 600 Hz, czyli w odstępach czasowych $1667 \mu\text{s}$. Na nasze potrzeby wartość została zaokrąglona do $1700 \mu\text{s}$. Błąd pomiarowy dla zarejestrowanych czasów to $\pm 1 \mu\text{s}$. Jest więc możliwe, że mikrofony rejestrują dźwięk w innym czasie: czujnik A może zarejestrować próbkę dźwięku pomiędzy dwoma odczytami czujnika B. Oczekuje się więc, że przy doskonałym wyeliminowaniu opóźnień komunikacji bezprzewodowej, błąd może wynosić $\pm 1 \mu\text{s}$ dla odczytu z jednego czujnika, $\pm 1 \mu\text{s}$ dla odczytu z drugiego czujnika i połowę okresu próbkowania dla różnicy pomiędzy nimi, czyli $\pm 850 \mu\text{s}$. Po zsumowaniu wartości oczekiwany błąd przy idealnym wyeliminowaniu opóźnień komunikacji bezprzewodowej wynosi w zaokrągleniu $\pm 850 \mu\text{s}$.

Wyniki analizy synchronizacji pomiarów IMU zostały podane w tab. 7.4. Wynika z nich, że najmniejsze różnice w czasie nasycenia czujników wynoszą dziesiątki mikrosekund, podczas gdy największe wynoszą nawet do ponad 7 ms. Jest to wynik spodziewany ze względu na dwie cechy czujników: po pierwsze, jakość użytych czujników nie jest wysoka, stąd każdy czujnik może dodawać swoją, inna wartość stałą do pomiarów,

Czujnik	Czujnik 0 [s]	Czujnik 1 [s]	Czujnik 2 [s]	Czujnik 3 [s]	Największa różnica[μ s]	Najmniejsza różnica[μ s]
Zbocze 1	36.418464	36.417359	36.413652	36.420849	7197	1105
Zbocze 2	38.444572	38.445697	38.449489	38.442675	6814	1125
Zbocze 3	40.828292	40.827104	40.828165	40.829431	2327	127
Zbocze 4	42.128230	42.127222	42.129376	42.128510	2154	280
Zbocze 5	43.400019	43.398503	43.397670	43.401829	4159	1516
Zbocze 6	44.295966	44.295223	44.299863	44.294660	5203	743
Zbocze 7	45.734697	45.732112	45.731082	45.733147	3615	1035
Zbocze 8	46.799186	46.800955	46.802505	46.796137	6368	1769
Zbocze 9	48.019619	48.019539	48.020906	48.022362	2823	1287
Zbocze 10	49.200261	49.201848	49.203555	49.201761	3294	87
Zbocze 11	50.334687	50.333604	50.329969	50.332943	4718	661

Tabela 7.4: Zestawienie czasów wejścia czujników DPS w stan nasycenia

przez co dwa czujniki mogą wchodzić w stan nasycenia w różnych chwilach pomiaru tej samej, zmieniającej się wartości. Ten problem występowałby niezależnie od obranego punktu referencyjnego dla którego sprawdzamy synchronizację i wpływałby na wynik w taki sam sposób, jeżeli przyjęlibyśmy za punkt odniesienia wartość zerową w czujnikach zamiast wejścia w stan nasycenia. Po drugie częstotliwość próbkowania jest niższa niż w przypadku mikrofonu, w konsekwencji czego oczekuje się błędu rzędu setek mikrosekund, jak wyjaśniono wyżej.

7.4.4 Wnioski

Na podstawie przeprowadzonych doświadczeń oraz analizy wyników można wnioskować, że zbudowany system spełnia swoją rolę. Synchronizacja czasu węzłów jest obarczona doświadczalnie wykazanym błędem rzędu pojedynczych milisekund. Taka precyzja spowodowana jest użytym protokołem NTP, bazującej na technologii IP, która nie jest przeznaczona do synchronizowania informacji z większą precyzją. Dane dostarczone do serwera są w dobrym stanie i w ogólności pozbawione defektów, możliwe jest wnioskowanie o wynikach badań na podstawie tych danych.

Rozdział 8

Podsumowanie

Celem pracy było zaprojektowanie i zbudowanie bezprzewodowej sieci kontrolno-pomiarowej, komunikującej się przez Wi-Fi ze szczególnym uwzględnieniem synchronizacji odczytów z czujników. Cel został zrealizowany poprzez opracowanie aplikacji serwerowej oraz węzłowej, oraz z użyciem bezprzewodowej łączności w lokalnej sieci komputerowej. Ustalono, iż zaprojektowany system jest w stanie zapisywać zebrane z czujników dane w postaci w ogólności wiernie odwzorowującej oryginalne dane.

WSN są dziedziną inżynierii, która prężnie rozwija się i w najbliższych latach liczba jej zastosowań będzie się stale zwiększać. To istotne, aby pracować nad nowymi technologiami pozwalającymi na bezawaryjną pracę takich sieci. Technologia IP choć uniwersalna, nie jest najlepszym wyborem w budowie WSN ze względu na spory koszt routingu pakietów, a co za tym idzie stosunkowo wysokie opóźnienie w przekazywaniu informacji od węzła do serwera. Duża precyzja synchronizacji czasowej jest niezbędna w niektórych zastosowaniach WSN, takich jak macierz mikrofonów, pozwalająca określić kierunek nadejścia dźwięku czy systemy wykrywające zderzenia, ich siłę oraz kierunek i podejmujące błyskawiczne działania na podstawie tych danych. Niemniej jednak, zastosowanie opracowanej w niniejszej pracy bezprzewodowej sieci czujników jest wskazane, gdy błąd synchronizacji rzędu pojedynczych milisekund nie stanowi dużego problemu, a koszt budowy sieci jest czynnikiem kluczowym.

Literatura

- [1] Performance Analysis of SMAC Protocol in Wireless Sensor Networks Using Network Simulator (Ns-2) by Gayatri Sakya and Vidushi Sharma; JSS Academy of Technical Education, Noida Gautam Buddha University, Greater Noida
- [2] Computer Networking: A Top-Down Approach (7th edition); by K.W. Ross, J.F. Kurose; published by Pearson Education, Inc.; 2017
- [3] The Go Programming Language by Alan A. A. Donovan, Brian W. Keringhan; published by Pearson Education, Inc.; 2016
- [4] Design Patterns: Elements of Reusable Object-Oriented Software by Gamma Erich, Helm Richard, Johnson Ralph, Vlissides John
- [5] Chee-Yee Chong, Srikanta P. Kumar: Sensor Networks: Evolution Opportunities and Challenges. Proceedings of the IEEE, 2003
- [6] Yildirim Kasim Sinan Kantarcı Aylin; 2014/01/01; Time Synchronization Based On Slow Flooding in Wireless Sensor Networks; IEEE Transactions on Parallel and Distributed Systems
- [7] Kasim Sinan Yildirim, Önder Gürcan. Efficient Time Synchronization in a Wireless Sensor Network by Adaptive Value Tracking. IEEE Transactions on Wireless Communications, Institute of Electrical and Electronics Engineers, 2014
- [8] Pahlavan, K., Krishnamurthy, P. Evolution and Impact of Wi-Fi Technology and Applications: A Historical Perspective. Int J Wireless Inf Networks 28, 3–19 (2021).
- [9] Oliver Gasser, Betreuerin: Corinna Schmitt Seminar Sensorknoten: Betrieb, Netze und Anwendungen SS2011 Lehrstuhl Netzarchitekturen und Netzdienste, Lehrstuhl Betriebssysteme und Systemarchitektur Fakultät für Informatik, Technische Universität München
- [10] Maharrey BK, Lim AS, Gao S. Interconnection between IP Networks and Wireless Sensor Networks. International Journal of Distributed Sensor Networks. December 2012. doi:10.1155/2012/567687
- [11] Divya Sharma, Sandeep Verma, Kanika Sharma. Network Topologies in Wireless Sensor Networks: A Review. Dept. of ECE, NITTTR, Chandigarh, UT, India

- [12] Kandris, D. Applications of Wireless Sensor Networks. Encyclopedia. Available online: <https://encyclopedia.pub/entry/17294> (accessed on 10 September 2022).
- [13] Durišić, M.P.; Tafa, Z.; Dimić, G.; Milutinović, V. A Survey of Military Applications of Wireless Sensor Networks. In Proceedings of the 2012 Mediterranean Conference on Embedded Computing (MECO), Bar, Montenegro, 19–21 June 2012.
- [14] Hii, P.; Chung, W. A Comprehensive Ubiquitous Healthcare Solution on an Android™ Mobile Device. *Sensors* 2011, 11, 6799–6815.
- [15] Nasir, A.; Soong, B.-H.; Ramachandran, S. Framework of WSN based human centric cyber physical in-pipe water monitoring system. In Proceedings of the 2010 11th International Conference on Control Automation Robotics & Vision, Singapore, 7–10 December 2010.
- [16] Unit Testing Principles, Practices, and Patterns by Vladimir Khorikov; published by Manning, Inc.; 2020
- [17] Docker. Projektowanie i wdrażanie aplikacji. Autor: Jarosław Krochmalski; wydane przez Packt Publishing; 2016
- [18] Pro Git. Written by Scott Chacon and Ben Straub, published by Apress
- [19] Git. Rozproszony system kontroli wersji. Autor: Włodzimierz Gajda; wydane przez Helion; 2013
- [20] [https://en.wikipedia.org/wiki/Queue_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Queue_(abstract_data_type))
- [21] https://en.wikipedia.org/wiki/Wireless_sensor_network
- [22] https://en.wikipedia.org/wiki/Network_Time_Protocol
- [23] https://docs.m5stack.com/en/core/m5stickc_plus
- [24] https://www.espressif.com/sites/default/files/documentation/esp32-pico-d4_datasheet_en.pdf
- [25] <https://www.freertos.org/>
- [26] https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos.html#_CPPv423xTaskCreatePinnedToCore14TaskFunction_tPCKcK8uint32_tPCv11UBaseType_tPC12TaskHandle_tK10BaseType_t
- [27] https://en.wikipedia.org/wiki/Comma-separated_values
- [28] https://en.wikipedia.org/wiki/Real-time_operating_system
- [29] <https://docs.espressif.com/projects/esp-idf/en/release-v3.3/api-reference/peripherals/can.html#:~:text=The%20ESP32's%20peripherals%20contains%20a,interpret%20such%20frames%20as%20errors.>

- [30] <https://www.python.org/>
- [31] https://en.wikipedia.org/wiki/Read%E2%80%93eval%E2%80%93print_loop
- [32] <https://www.arduino.cc/reference/en/language/functions/communication/wire/>
- [33] <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2s.html>
- [34] <https://www.arduino.cc/reference/en/libraries/m5stickcplus/>
- [35] <https://github.com/espressif/arduino-esp32/blob/master/libraries/EEPROM/src/EEPROM.h>
- [36] <https://github.com/espressif/arduino-esp32/blob/master/libraries/WiFi/src/WiFi.h>
- [37] <https://www.arduino.cc/en/software>
- [38] <https://github.com/arduino/ArduinoCore-avr/blob/master/cores/arduino/Arduino.h>
- [39] <https://code.visualstudio.com/>
- [40] <https://platformio.org/>
- [41] <https://docs.platformio.org/en/stable/what-is-platformio.html>
- [42] https://m5stack.oss-cn-shenzhen.aliyuncs.com/resource/docs/datasheet/core/MPU-6886-000193%2Bv1.1_GHIC_en.pdf
- [43] <https://www.mouser.com/datasheet/2/218/SPM1423HM4H-B-876897.pdf>
- [44] <https://en.wikipedia.org/wiki/C%2B%2B>
- [45] <https://pandas.pydata.org/>
- [46] <https://matplotlib.org/>
- [47] <https://www.wireshark.org/>
- [48] <https://support.f5.com/csp/article/K35612380>
- [49] <https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.txt>
- [50] <https://www.khanacademy.org/computing/computers-and-internet/xcae6f4a7ff015e7d:online-data-security/xcae6f4a7ff015e7d:secure-internet-protocols/a/transport-layer-security-protocol-tls>
- [51] https://en.wikipedia.org/wiki/Web_server

- [52] <https://www.ibm.com/pl-pl/cloud/learn/rest-apis>
- [53] <https://www.nginx.com/blog/lets-encrypt-tls-nginx/>
- [54] <https://getbootstrap.com/>
- [55] <https://vuejs.org/>
- [56] <https://gin-gonic.com/>
- [57] <https://pkg.go.dev/testing>
- [58] https://en.wikipedia.org/wiki/Inertial_measurement_unit
- [59] <https://docs.docker.com/compose/>
- [60] <https://git-scm.com/docs>
- [61] https://cassandra.apache.org/_/index.html
- [62] <https://www.ovhcloud.com/pl/public-cloud/what-load-balancing/>
- [63] https://dl.dell.com/topicspdf/inspiron-15-7567-laptop_setup-guide_en-us.pdf
- [64] <https://maxelektro.pl/poradniki/co-to-jest-zigbee-i-jak-dziala,461.html>
- [65] <https://home.agh.edu.pl/~km2007/misc/papers/49.pdf>
- [66] [https://pl.wikipedia.org/wiki/CSV_\(format_pliku\)](https://pl.wikipedia.org/wiki/CSV_(format_pliku))
- [67] <https://go.dev/doc/install>

Dodatek A

Kod źródłowy aplikacji węzłów

A.1 main.cpp

```
1  #include "globals.hpp"
2  #include "link.hpp"
3  #include "mpu.hpp"
4  #include "mike.hpp"
5
6  Connection conn = Connection();
7
8  SemaphoreHandle_t command_response_mutex;
9
10 static const int led = 10;
11 int led_delay = 2000;
12 static const uint8_t queue_len = 5;
13 byte streaming_device = Devices::NONE;
14
15 void Write1Byte(uint8_t Addr, uint8_t Data) {
16     Wire1.beginTransaction(0x34);
17     Wire1.write(Addr);
18     Wire1.write(Data);
19     Wire1.endTransmission();
20 }
21
22 uint8_t Read8bit(uint8_t Addr) {
23     Wire1.beginTransaction(0x34);
24     Wire1.write(Addr);
25     Wire1.endTransmission();
26     Wire1.requestFrom(0x34, 1);
27     return Wire1.read();
28 }
29
```

```
30 void power_up(){
31     Wire1.begin(21, 22);
32     Wire1.setClock(400000);
33
34     // Set ADC to All Enable
35     Write1Byte(0x82, 0xff);
36
37     // Bat charge voltage to 4.2, Current 100MA
38     Write1Byte(0x33, 0xc0);
39
40     // Enable Bat,ACIN,VBUS,APS adc
41     Write1Byte(0x82, 0xff);
42
43     // Enable Ext, LDO2, LDO3, DCDC1
44     Write1Byte(0x12, Read8bit(0x12));
45
46     // 128ms power on, 4s power off
47     Write1Byte(0x36, 0x0C);
48
49     // Set GPIO0 to LDO
50     Write1Byte(0x90, 0x02);
51
52     // Disable vbus hold limit
53     Write1Byte(0x30, 0x80);
54
55     // Set temperature protection
56     Write1Byte(0x39, 0xfc);
57
58     // Enable bat detection
59     Write1Byte(0x32, 0x46);
60 }
61
62 bool device_on(byte device, byte *response)
63 {
64     D(Serial.println("Device on")););
65     response[1] = Responses::COMMAND_OK;
66     switch(device)
67     {
68         case Devices::MPU6886:
69             D(Serial.println("MPU on")););
70             return start_MPU6886_task(&conn);
71         case Devices::MIKE:
72             D(Serial.println("Device on: Mike")););
```

```

73     return start_Mike_task(&conn);
74     // Here add other devices
75     // case Devices::Example:
76     //     D(Serial.println("Device on: Example"));
77     //     return start_Example_task(&conn);
78     default:
79         response[1] = Responses::COMMAND_UNKNOWN;
80         return false;
81     }
82 }
83
84 bool device_off(byte device, byte *response)
85 {
86     D(Serial.println("Device off"));
87     response[1] = Responses::COMMAND_OK;
88     switch(device)
89     {
90         case Devices::MPU6886:
91             D(Serial.println("MPU6886 off"));
92             return stop_MPU6886_task();
93         case Devices::MIKE:
94             D(Serial.println("MIKE off"));
95             return stop_Mike_task();
96         // Here add other devices
97         // case Devices::Example:
98         //     D(Serial.println("Example off"));
99         //     return stop_Example_task();
100        default:
101            response[1] = Responses::COMMAND_UNKNOWN;
102            return false;
103        }
104    }
105
106    bool set_device_parameters(byte device, byte *params, int *param_len)
107    {
108        D(Serial.println("Set device parameters"));
109        switch (device) {
110            case Devices::MPU6886:
111                D(Serial.println("Setting MPU parameters"));
112                return MPU::set_parameters(params, param_len);
113            case Devices::MIKE:
114                D(Serial.println("Setting MIKE parameters"));
115                return Mike::set_parameters(params, param_len);

```

```

116     // Here add other devices
117     // case Devices::Example:
118     //     D(Serial.println("Setting Example parameters"));
119     //     return Example::set_parameters(params, param_len);
120     default:
121         D(Serial.println("Command: 0x?? - (Unknown)"));
122         return false;
123     }
124 }
125
126 void send_task(void *parameter)
127 {
128     D(Serial.println("Send task"));
129     // Task responsible for sending data to the server
130     Connection *conn = (Connection *)parameter;
131     while(true){
132         conn->send_from_queue();
133     }
134 }
135
136 bool device_parameters(byte *command, int *command_len, byte
↵ *response)
137 {
138     D(Serial.println("Device parameters"));
139     *command_len -= 2;
140     if (set_device_parameters(command[1], &command[2], command_len)) {
141         response[1] = Responses::COMMAND_OK;
142         return true;
143     } else {
144         response[1] = Responses::COMMAND_UNKNOWN;
145         D(Serial.println("Wrong parameters"));
146         return false;
147     }
148 }
149
150 bool on_off(byte *command, int *command_len, byte *response)
151 {
152     D(Serial.println("on_off"));
153     // Run command
154     switch(command[2]) {
155         case Commands::OFF:
156             D(Serial.println("Turning off"));
157             if (streaming_device == Devices::NONE) {

```

```

158     response[1] = Responses::COMMAND_NO_STREAMING;
159 } else {
160     if(device_off(command[1], response)) {
161         streaming_device = Devices::NONE;
162         return true;
163     }
164 }
165 break;
166 case Commands::ON:
167     D(Serial.println("Turning on");)
168     if (streaming_device != Devices::NONE) {
169         response[1] = Responses::COMMAND_ALREADY_STREAMING;
170     } else {
171         if( device_on(command[1], response)) {
172             streaming_device = command[1];
173             return true;
174         }
175     }
176     break;
177 }
178 return false;
179 }
180
181 bool run_command(byte *command, int *command_len, byte *response)
182 {
183     D(Serial.println("run_command");)
184     // Check minimum length of command
185     if(*command_len < 3) {
186         D(Serial.println("Command too short");)
187         if ((*command_len) > 0) {
188             response[0] = command[0];
189         }
190         response[1] = Responses::COMMAND_UNKNOWN;
191         sleep(100);
192         conn.hand_shake();
193         return false;
194     }
195     response[0] = command[0];
196     // Check which command is being sent
197     switch(command[0]) {
198         case Commands::SEND_PARAMETERS:
199             return device_parameters(command, command_len, response);
200         case Commands::ON_OFF:

```

```

201     return on_off(command, command_len, response);
202 default:
203     D(Serial.println("Command: 0x?? - (Unknown)"));
204     response[1] = Responses::COMMAND_UNKNOWN;
205     return false;
206 }
207 }
208
209 void led_task(void *parameter)
210 {
211     while (true) {
212         digitalWrite(led, LOW);
213         sleep(10);
214         digitalWrite(led, HIGH);
215         sleep(led_delay);
216         if (streaming_device != Devices::NONE) {
217             digitalWrite(led, LOW);
218             sleep(10);
219             digitalWrite(led, HIGH);
220         }
221         sleep(50);
222     }
223 }
224
225 void main_task(void *parameter)
226 {
227     Connection *conn = (Connection *)parameter;
228     xTaskCreatePinnedToCore(send_task, "send_task", 4096, conn, 2, NULL,
↪ app_cpu);
229     xTaskCreatePinnedToCore(led_task, "led_task", 1024, &conn, 1, NULL,
↪ app_cpu);
230
231     Msg msg = Msg();
232     byte data[2] = {0x00, 0x00};
233     msg.data = data;
234     msg.len = 2;
235     msg.mutex = &command_response_mutex;
236     int command_max_len = 4;
237     int received_bytes = 0;
238     byte command[command_max_len];
239     bool result = false;
240     xSemaphoreGive(command_response_mutex);
241

```

```

242     while (true) {
243         conn->rec_msg(command, &received_bytes);
244         run_command(command, &received_bytes, msg.data);
245         // If you want to send a response to the command, uncomment the
↪         following lines
246         // xSemaphoreTake(command_response_mutex, portMAX_DELAY);
247         // D(Serial.println("Sending response"));
248         // conn->add_to_send_queue(&msg);
249     }
250 }
251
252 void setup()
253 {
254     D(
255         Serial.begin(115200);
256         sleep(500);
257         Serial.println("----START----");
258     );
259     EEPROM.begin(EEPROM_SIZE);
260     pinMode(led, OUTPUT);
261     command_response_mutex = xSemaphoreCreateBinary();
262     power_up();
263     conn.init();
264     xTaskCreatePinnedToCore(main_task, "main_task", 4096, &conn, 1,
↪     NULL, app_cpu);
265 }
266
267 void loop(){}

```

A.2 globals.cpp

```

1  #include "globals.hpp"
2
3
4
5  String charArrayToString(char arrChar[], int tam)
6  {
7      String s = "";
8      for (int i = 0; i < tam; i++)
9      {
10         s = s + arrChar[i];

```



```
11     }
12     return s;
13 }
14
15 void sleep(int ms)
16 {
17     vTaskDelay(ms / portTICK_PERIOD_MS);
18 }
```

A.3 globals.hpp

```
1  #ifndef GLOBALS_HPP
2  #define GLOBALS_HPP
3
4  #include <Arduino.h>
5  #include <WiFi.h>
6  #include <EEPROM.h>
7  #include <M5StickCPlus.h>
8  #include <Wire.h>
9  #include <driver/i2s.h>
10
11 #define MAX_MSG_LEN 1436
12 #define HEADER_LEN 16
13 #define EEPROM_SIZE 1
14
15 #define pro_cpu 0
16 #define app_cpu 1
17
18 #define DEBUG 0
19
20 #if DEBUG == 1
21 #define D(x) x
22 #else
23 #define D(x)
24 #endif
25 struct Devices {
26     const static byte NONE {0x00};
27     const static byte MPU6886 {0x01};
28     const static byte MIKE {0x02};
29 };
30
```

```
31 struct Msg
32 {
33     byte *data;
34     int len;
35     SemaphoreHandle_t* mutex;
36 };
37
38 struct TimeUnionStruct
39 {
40     byte sec[4];
41     byte usec[4];
42 };
43
44 union TimeUnion
45 {
46     struct TimeUnionStruct bytes;
47     struct timeval tv_now;
48 };
49
50 struct DataStruct
51 {
52     byte header[2];
53     byte time[8];
54     byte frequency[4];
55     byte count[2];
56     byte data[MAX_MSG_LEN-HEADER_LEN];
57 };
58
59 struct MsgStruct
60 {
61     byte bytes[MAX_MSG_LEN];
62 };
63
64 union DataUnion
65 {
66     struct DataStruct data;
67     struct MsgStruct msg;
68 };
69
70 String charArrayToString(char arrChar[], int tam);
71 void sleep(int ms);
72
73 #endif
```

A.4 link.cpp

```
1  #include "link.hpp"
2
3  Connection::Connection() {
4      ID = 0;
5      send_ID[0] = Commands::SEND_ID;
6      send_ID[1] = 0x00;
7      ask_ID[0] = Commands::ASK_ID;
8      ask_ID[1] = 0xFF;
9      recv_task = NULL;
10 }
11
12 void Connection::init() {
13     ID = EEPROM.read(ID_EEPROM_ADDRESS);
14     connectWiFi();
15     ntp_time();
16     wait_for_time_info();
17     block_until_client_is_connected();
18     hand_shake();
19     send_queue = xQueueCreate(5, sizeof(struct Msg));
20     recv_mutex = xSemaphoreCreateBinary();
21     xSemaphoreGive(recv_mutex);
22 }
23
24 void Connection::add_to_send_queue(Msg* data) {
25     xQueueSend(send_queue, data, portMAX_DELAY);
26 }
27
28 void Connection::send_from_queue() {
29     Msg msg = Msg();
30     // D(Serial.println(msg.len));
31     xQueueReceive(send_queue, &msg, portMAX_DELAY);
32     if (!client.connected()) {
33         block_until_client_is_connected();
34         hand_shake();
35     }
36     client.write(msg.data, msg.len);
37     xSemaphoreGive(*msg.mutex);
38 }
39
40 bool Connection::block_until_client_is_connected() {
41     if (!client.connected()) {
```

```
42     while (!connectClient()) {};  
43     return false;  
44 }  
45     return true;  
46 }  
47  
48 void Connection::rec_msg(byte* msg, int* len) {  
49     size_t bytes_to_read = 0;  
50     *len = 0;  
51     byte* data_ptr = msg;  
52  
53     while (bytes_to_read == 0) {  
54         if (!block_until_client_is_connected()) {  
55             data_ptr[0] = 0x00;  
56             return;  
57         }  
58         bytes_to_read = client.available();  
59     }  
60  
61     while (bytes_to_read != 0) {  
62         client.read((data_ptr + *len), bytes_to_read);  
63         *len += bytes_to_read;  
64         bytes_to_read = client.available();  
65     }  
66 }  
67  
68 bool Connection::connectClient() {  
69     if (!client.connect(HOST, PORT)) {  
70         D(Serial.println("Connection to host failed");)  
71         sleep(1000);  
72         return false;  
73     }  
74     D(Serial.println("Connection to host successful"););  
75     return true;  
76 }  
77  
78 void Connection::hand_shake() {  
79     bool Error = true;  
80     int len = 0;  
81  
82  
83     while (Error) {  
84         block_until_client_is_connected();
```

```
85     ID = EEPROM.read(ID_EEPROM_ADDRESS);
86     if (ID == 0xFF) {
87         D(Serial.println("Asking for ID"));
88         client.write(ask_ID, 2);
89         if (!client.connected()) continue;
90         rec_msg(msg, &len);
91
92         if (msg[0] == Responses::HANDSHAKE_OK) {
93             EEPROM.write(ID_EEPROM_ADDRESS, msg[1]);
94             EEPROM.commit();
95             D(
96                 Serial.print(msg[1], HEX);
97                 Serial.println('h');
98                 Serial.print("GOT statusOk with ID: ");
99                 Serial.println("ID stored in Flash memory");
100             );
101             Error = false;
102         } else {
103             D(
104                 Serial.print("Wrong response: ");
105                 for (int i = 0; i < len; i++) {
106                     Serial.print(msg[i], HEX);
107                 }
108                 Serial.println();
109                 Serial.println("Trying again");
110             );
111             Error = true;
112             sleep(1000);
113         }
114     } else {
115         D(
116             Serial.print("Sending ID: ");
117             Serial.print(ID, HEX);
118             Serial.println('h');
119         );
120         send_ID[1] = ID;
121         client.write(send_ID, 2);
122
123         if (!client.connected()) continue;
124         rec_msg(msg, &len);
125         switch (msg[0])
126         {
127             case Responses::HANDSHAKE_OK:
```

```
128         D(Serial.println("Received: statusOk")););
129         Error = false;
130         break;
131     case Responses::HANDSHAKE_ID_DOESNT_EXIST:
132         D(Serial.println("Received: IdDoesntExist")););
133         Error = true;
134         break;
135     case Responses::HANDSHAKE_ID_IS_ALREADY_CONNECTED:
136         D(Serial.println("Received: IdIsAlreadyConnected")););
137         Error = true;
138         break;
139     case Responses::HANDSHAKE_NO_ID_AVAILABLE:
140         D(Serial.println("Received: NoIdAvailable")););
141         Error = true;
142         break;
143     case Responses::HANDSHAKE_HANDSHAKE_NOT_VALID:
144         D(Serial.println("Received: HandshakeNotValid")););
145         Error = true;
146         break;
147     case Responses::HANDSHAKE_COMMAND_NOT_RECOGNIZED:
148         D(Serial.println("Received: CommandNotRecognized")););
149         Error = true;
150         break;
151     case Responses::HANDSHAKE_UNEXPECTED_PACKET_HEADER:
152         D(Serial.println("Received: UnexpectedPacketHeader")););
153         Error = true;
154         break;
155     case Responses::HANDSHAKE_UNEXPECTED_PACKET_BODY:
156         D(Serial.println("Received: UnexpectedPacketBody")););
157         Error = true;
158         break;
159
160     default:
161         D(
162             Serial.print("Wrong response: ");
163             for (int i = 0; i < len; i++) {
164                 Serial.print(msg[i], HEX);
165             }
166         );
167         Error = true;
168         continue;
169     }
170     if (Error) {
```

```
171         // If any error - set ID to unset and try again
172         EEPROM.write(ID_EEPROM_ADDRESS, 0xFF);
173         EEPROM.commit();
174     }
175 }
176 }
177 }
178
179 void Connection::connectWiFi() {
180     D(Serial.println("Connecting to Wi-Fi"));
181     WiFi.begin(SSID, PASSWORD);
182     while (WiFi.status() != WL_CONNECTED) {
183         sleep(500);
184         D(Serial.print("."));
185     }
186     D(
187         Serial.println("");
188         Serial.print("WiFi connected with IP: ");
189         Serial.println(WiFi.localIP());
190     );
191 }
192
193 void Connection::ntp_time() {
194     configTime(3600, 3600, ntpServer);
195     D(
196         Serial.print("Trying to get time info from: ");
197         Serial.println(ntpServer);
198         Serial.println("It could take a while.");
199     );
200 }
201
202 void Connection::wait_for_time_info() {
203     struct tm time;
204     while (!getLocalTime(&time)) {
205         D(Serial.println("Could not obtain time info"));
206     }
207     D(Serial.println("Got current time"));
208 }
```

A.5 link.hpp

```
1  #pragma once
2  #ifndef __LINK__
3  #define __LINK__
4  #include "globals.hpp"
5
6  #define ID_EEPROM_ADDRESS 0
7  #define SSID "WI-FI LOGIN"
8  #define PASSWORD "WI-FI PASSWORD"
9  #define HOST "192.168.0.254"
10 #define PORT 7122
11
12 #if DEBUG == 1
13 #define ntpServer "time.google.com"
14 #else
15 #define ntpServer HOST
16 #endif
17
18 struct Commands {
19     const static byte SEND_DATA {0x03};
20     const static byte ASK_ID {0x01};
21     const static byte SEND_ID {0x02};
22     const static byte SEND_PARAMETERS {0x04};
23     const static byte ON_OFF {0x05};
24     const static byte ON {0x01};
25     const static byte OFF {0x02};
26 };
27
28 struct Responses {
29     const static byte HANDSHAKE_OK {0x01};
30     const static byte HANDSHAKE_ID_DOESNT_EXIST {0x02};
31     const static byte HANDSHAKE_ID_IS_ALREADY_CONNECTED {0x03};
32     const static byte HANDSHAKE_NO_ID_AVAILABLE {0x04};
33     const static byte HANDSHAKE_HANDSHAKE_NOT_VALID {0x05};
34     const static byte HANDSHAKE_COMMAND_NOT_RECOGNIZED {0x06};
35     const static byte HANDSHAKE_UNEXPECTED_PACKET_HEADER {0x07};
36     const static byte HANDSHAKE_UNEXPECTED_PACKET_BODY {0x08};
37     const static byte COMMAND_UNKNOWN {0x0B};
38     const static byte COMMAND_OK {0x01};
39     const static byte COMMAND_ALREADY_STREAMING {0x09};
40     const static byte COMMAND_NO_STREAMING {0x0A};
41 };
```



```

42
43 static TaskHandle_t recv_task;
44 void receiver_task(void* queue);
45
46 class Connection {
47
48     private:
49         int ID;
50         byte send_ID[2];
51         byte ask_ID[2];
52         SemaphoreHandle_t recv_mutex;
53         QueueHandle_t send_queue;
54
55     public:
56         Connection();
57         WiFiClient client;
58         byte msg[MAX_MSG_LEN];
59         void rec_msg(byte* msg, int* len);
60         void add_to_send_queue(Msg* data);
61         void send_from_queue();
62         bool connectClient();
63         void hand_shake();
64         void connectWiFi();
65         void ntp_time();
66         void wait_for_time_info();
67         void init();
68         bool block_until_client_is_connected();
69         void start_receiver_task(QueueHandle_t* queue);
70         void stop_receiver_task();
71 };
72 #endif

```

A.6 mpu.cpp

```

1  #include "mpu.hpp"
2
3  SemaphoreHandle_t mutex6886 = xSemaphoreCreateBinary();
4
5  MPU::MPU(){}
6  byte MPU::settings[2] = {0x00, 0x00};
7  TwoWire MPU::i2c = TwoWire(0);

```

```

8
9 bool MPU::set_parameters(byte* params, int* length) {
10     if (*length != 2) {
11         return false;
12     }
13     if (params[0] >= MPUGyroRange::DPS_250 && params[0] <=
↪ MPUGyroRange::DPS_2000 && params[1] >= MPUAccelRange::G2 &&
↪ params[1] <= MPUAccelRange::G16) {
14         settings[0] = params[0];
15         settings[1] = params[1];
16         return true;
17     } else {
18         return false;
19     }
20 }
21
22 void MPU::init() {
23     init(settings[1], settings[0]);
24 }
25
26 void MPU::init(byte a_range, byte g_range) {
27     D(Serial.println("Initializing MPU"));
28     current_msg = MPU_QUEUE_LEN;
29     for (int i; i < MPU_QUEUE_LEN; i++) {
30         msg[i] = DataUnion();
31         msg[i].data.header[0] = Commands::SEND_DATA;
32         msg[i].data.header[1] = Devices::MPU6886;
33         msg_mutex[i] = xSemaphoreCreateBinary();
34         xSemaphoreGive(msg_mutex[i]);
35     }
36     D(Serial.println("Initializing MPU complete"));
37     D(Serial.println("Initializing I2C"));
38     i2c.begin(MPU_SDA_PIN, MPU_SCL_PIN);
39     D(Serial.println("Initializing I2C complete"));
40     reset();
41     D(Serial.println("Resetting MPU complete"));
42     set_accel_range(a_range);
43     set_gyro_range(g_range);
44     accel_range_f = get_accel_range();
45     gyro_range_f = get_gyro_range();
46     accel_multiplier = accel_range_f/32786.0;
47     gyro_multiplier = gyro_range_f/32786.0;
48     D(Serial.println("Initializing MPU complete"));

```

```
49     set_frequency();
50     D(Serial.println("Measuring MPU frequency complete"));
51 }
52
53 void MPU::reset() {
54     unsigned char regdata = 0x00;
55     write_NBytes(MPU_ADDRESS, MPU_PWR_MGMT_1, 1, &regdata);
56     sleep(10);
57 }
58
59 void MPU::write_NBytes(uint8_t driver_Address, uint8_t reg_Address,
60 ↪ uint8_t length, uint8_t* data) {
61     i2c.beginTransaction(driver_Address);
62     i2c.write(reg_Address);
63     i2c.write(data, length);
64     i2c.endTransmission(true);
65 }
66
67 void MPU::read_NBytes(uint8_t driver_Address, uint8_t reg_Address,
68 ↪ uint8_t length, uint8_t* data) {
69     i2c.beginTransaction(driver_Address);
70     i2c.write(reg_Address);
71     i2c.endTransmission(false);
72     i2c.requestFrom(driver_Address, length);
73     uint8_t i = 0;
74     while (i2c.available()) {
75         data[i++] = i2c.read();
76     }
77 }
78
79 void MPU::get_data(float* data_ptr) {
80     read_NBytes(MPU_ADDRESS, MPU_ACCEL_XOUT_H, 14, data);
81     for (int i = 0; i < 3; i++) {
82         calculate_from_bytes(
83             2*i,
84             &accel_multiplier,
85             &accel_range_f,
86             (data_ptr + i)
87         );
88         calculate_from_bytes(
89             2*i+8,
90             &gyro_multiplier,
91             &gyro_range_f,
```

```

90         (data_ptr + 3 + i)
91     );
92 }
93 }
94
95 DataUnion* MPU::get_msg() {
96     current_msg = (current_msg + 1) % MPU_QUEUE_LEN;
97     xSemaphoreTake(*get_current_semaphore(), portMAX_DELAY);
98     return (msg + current_msg);
99 }
100
101 SemaphoreHandle_t* MPU::get_current_semaphore() {
102     return (msg_mutex + current_msg);
103 }
104
105 void MPU::set_frequency() {
106     union {
107         float var;
108         byte bytes[4];
109     } freq_u;
110
111     freq_u.var = 582.0;
112
113     for (int i=0; i < MPU_QUEUE_LEN; i++) {
114         for (int j=0; j < 4; j++) {
115             msg[i].msg.bytes[j+10] = freq_u.bytes[j];
116             D(Serial.print(freq_u.bytes[j], HEX));
117         }
118         D(Serial.println(""));
119     }
120 }
121 }
122
123 void MPU6886_task(void* connection) {
124     Connection * conn = (Connection*)connection;
125
126     // Prepare the MPU6886
127     MPU mpu = MPU();
128     mpu.init();
129     DataUnion * msg = mpu.get_msg();
130     float *data_ptr = (float *) (msg->msg.bytes + HEADER_LEN);
131     int max_len = (MAX_MSG_LEN - HEADER_LEN)/24;
132     size_t len = 0;

```

```

133     TimeUnion timeUnion;
134     Msg mm = Msg();
135
136     // Main measurement loop
137     while (uxSemaphoreGetCount(mutex6886) > 0) {
138         // Set the time of the first measurement
139         len = 0;
140         gettimeofday(&timeUnion.tv_now, NULL);
141         timeUnion.tv_now.tv_sec += 7200;
142         memcpy(msg->data.time, timeUnion.bytes.sec, 8);
143         // Take measurements
144         while (len < max_len) {
145             mpu.get_data((data_ptr + len*6));
146             len += 1;
147         }
148         // Send data
149         mm.data = msg->msg.bytes;
150         msg->data.count[0] = (len) & 0xFF;
151         msg->data.count[1] = (len >> 8) & 0xFF;
152         mm.len = HEADER_LEN + len*24;
153         mm.mutex = mpu.get_current_semaphore();
154         conn->add_to_send_queue(&mm);
155         // Get pointer to next free message
156         msg = mpu.get_msg();
157         data_ptr = (float *) (msg->msg.bytes + HEADER_LEN);
158     }
159     D(Serial.println("Ending Wire"));
160     mpu.i2c.end();
161     D(Serial.println("Wire ended"));
162     xSemaphoreGive(mutex6886);
163     while(true){}
164 }
165
166 bool start_MPU6886_task(Connection *connection) {
167     D(Serial.println("Starting MPU6886 task"));
168     xSemaphoreGive(mutex6886);
169     xTaskCreatePinnedToCore(MPU6886_task, "mpu_task", 8192,
170     ↪ connection, configMAX_PRIORITIES -1, &task6886, pro_cpu);
171     D(Serial.println("Starting MPU6886 task complete"));
172     return true;
173 }
174
175 bool stop_MPU6886_task() {

```

```
175     D(Serial.println("Stopping MPU6886 task")););
176     D(Serial.println("Sending stop signal to MPU6886")););
177     xSemaphoreTake(mutex6886, portMAX_DELAY);
178     D(Serial.println("Sent stop signal to MPU6886")););
179     D(Serial.println("Waiting for MPU6886 task to end")););
180     xSemaphoreTake(mutex6886, portMAX_DELAY);
181     D(Serial.println("MPU6886 task ended")););
182     if (task6886 != NULL) vTaskDelete(task6886);
183     D(Serial.println("MPU6886 task stopped")););
184     return true;
185 }
186
187 void MPU::set_accel_range(byte range) {
188     unsigned char regdata = range;
189     regdata = regdata << 3;
190     write_NBytes(MPU_ADDRESS, MPU_ACCEL_CONFIG, 1, &regdata);
191     sleep(50);
192     accel_range = range;
193 }
194
195 void MPU::set_gyro_range(byte range) {
196     unsigned char regdata = range;
197     regdata = regdata << 3;
198     write_NBytes(MPU_ADDRESS, MPU_GYRO_CONFIG, 1, &regdata);
199     sleep(50);
200     gyro_range = range;
201 }
202
203 float MPU::get_accel_range() {
204     switch (accel_range) {
205         case MPUAccelRange::G2:
206             return 2.0;
207         case MPUAccelRange::G4:
208             return 4.0;
209         case MPUAccelRange::G8:
210             return 8.0;
211         case MPUAccelRange::G16:
212             return 16.0;
213         default:
214             return 2.0;
215     }
216 }
217
```

```

218 float MPU::get_gyro_range() {
219     switch (gyro_range) {
220         case MPUGyroRange::DPS_250:
221             return 250.0;
222         case MPUGyroRange::DPS_500:
223             return 500.0;
224         case MPUGyroRange::DPS_1000:
225             return 1000.0;
226         case MPUGyroRange::DPS_2000:
227             return 2000.0;
228         default:
229             return 250.0;
230     }
231 }
232
233 void MPU::calculate_from_bytes(uint8_t start_index, float* scale,
    ↪ float* range, float* destination) {
234     // Calculate the float value from two bytes obtained from the
    ↪ MPU6886
235     if ((data[start_index] & 0x80) != 0x80) {
236         *destination = (((data[start_index] & 0x7F) << 8) |
    ↪ data[start_index+1]) * *scale;
237     } else {
238         *destination = -(((data[start_index] & 0x7F) << 8) |
    ↪ data[start_index+1])) * *scale - *range;
239     }
240 }

```

A.7 mpu.hpp

```

1  #ifndef _MPU_
2  #define _MPU_
3  #include "globals.hpp"
4  #include "link.hpp"
5
6  #define MPU_ADDRESS          0x68
7  #define MPU_WHOAMI           0x75
8  #define MPU_ACCEL_INTEL_CTRL 0x69
9  #define MPU_SMPLRT_DIV       0x19
10 #define MPU_INT_PIN_CFG       0x37
11 #define MPU_INT_ENABLE         0x38

```

```
12 #define MPU_ACCEL_XOUT_H      0x3B
13 #define MPU_ACCEL_XOUT_L      0x3C
14 #define MPU_ACCEL_YOUT_H      0x3D
15 #define MPU_ACCEL_YOUT_L      0x3E
16 #define MPU_ACCEL_ZOUT_H      0x3F
17 #define MPU_ACCEL_ZOUT_L      0x40
18
19 #define MPU_TEMP_OUT_H         0x41
20 #define MPU_TEMP_OUT_L         0x42
21
22 #define MPU_GYRO_XOUT_H        0x43
23 #define MPU_GYRO_XOUT_L        0x44
24 #define MPU_GYRO_YOUT_H        0x45
25 #define MPU_GYRO_YOUT_L        0x46
26 #define MPU_GYRO_ZOUT_H        0x47
27 #define MPU_GYRO_ZOUT_L        0x48
28
29 #define MPU_USER_CTRL          0x6A
30 #define MPU_PWR_MGMT_1         0x6B
31 #define MPU_PWR_MGMT_2         0x6C
32 #define MPU_CONFIG             0x1A
33 #define MPU_GYRO_CONFIG         0x1B
34 #define MPU_ACCEL_CONFIG        0x1C
35 #define MPU_ACCEL_CONFIG2      0x1D
36 #define MPU_FIFO_EN            0x23
37
38 #define MPU_SDA_PIN             21
39 #define MPU_SCL_PIN             22
40 #define MPU_INTERRUPT_PIN       35
41
42 #define MPU_QUEUE_LEN           4
43
44 struct MPUAccelRange {
45     const static byte G2 {0x00};
46     const static byte G4 {0x01};
47     const static byte G8 {0x02};
48     const static byte G16 {0x03};
49 };
50
51 struct MPUGyroRange{
52     const static byte DPS_250 {0x00};
53     const static byte DPS_500 {0x01};
54     const static byte DPS_1000 {0x02};
```



```

55     const static byte DPS_2000 {0x03};
56 };
57
58 class MPU {
59     public:
60         MPU();
61         void init();
62         void init(byte a_range, byte g_range);
63         void get_data(float* data_ptr);
64         void set_accel_range(byte range);
65         void set_gyro_range(byte range);
66         float get_accel_range();
67         float get_gyro_range();
68         SemaphoreHandle_t* get_current_semaphore();
69         void calculate_from_bytes(uint8_t start_index, float* scale,
↪ float* range, float* destination);
70         DataUnion* get_msg();
71         uint8_t data[14];
72
73         static byte settings[2];
74         static bool set_parameters(byte* params, int* length);
75         static TwoWire i2c;
76
77     private:
78         SemaphoreHandle_t msg_mutex[MPU_QUEUE_LEN];
79         DataUnion msg[MPU_QUEUE_LEN];
80         int current_msg;
81
82         byte gyro_range;
83         byte accel_range;
84         float gyro_range_f;
85         float accel_range_f;
86         float accel_multiplier;
87         float gyro_multiplier;
88
89         void reset();
90         void set_frequency();
91         void write_NBytes(uint8_t driver_Address, uint8_t
↪ reg_Address, uint8_t length, uint8_t* data);
92         void read_NBytes(uint8_t driver_Address, uint8_t reg_Address,
↪ uint8_t length, uint8_t* data);
93 };
94

```

```
95 static TaskHandle_t task6886;  
96  
97 bool start_MPU6886_task(Connection *connection);  
98 bool stop_MPU6886_task();  
99 void MPU6886_task(void *connection);  
100  
101 #endif
```

A.8 mike.cpp

```
1  #include "mike.hpp"  
2  
3  uint32_t Mike::sample_rate = 44100;  
4  SemaphoreHandle_t mutexMike = xSemaphoreCreateBinary();  
5  
6  Mike::Mike() {  
7  }  
8  
9  bool Mike::set_parameters(byte* params, int* length) {  
10     if (*length == 1) {  
11         if (params[0] == SampleFreq::Hz_22050) {  
12             sample_rate = 22050;  
13             return true;  
14         } else if (params[0] == SampleFreq::Hz_44100) {  
15             sample_rate = 44100;  
16             return true;  
17         } else {  
18             return false;  
19         }  
20     } else {  
21         return false;  
22     }  
23 }  
24  
25 void Mike::init() {  
26     current_msg = MIKE_QUEUE_LEN;  
27  
28     union {  
29         float var;  
30         byte bytes[4];  
31     } freq_u;
```

```

32     freq_u.var = sample_rate;
33
34     for (int i; i < MIKE_QUEUE_LEN; i++) {
35         msg[i] = DataUnion();
36         msg[i].data.header[0] = Commands::SEND_DATA;
37         msg[i].data.header[1] = Devices::MIKE;
38         memcpy(msg[i].data.frequency, freq_u.bytes, 4);
39         msg_mutex[i] = xSemaphoreCreateBinary();
40         xSemaphoreGive(msg_mutex[i]);
41     }
42
43     i2s_config_t cfg_i2s = {
44         .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX |
↪ I2S_MODE_PDM),
45         .sample_rate = sample_rate,
46         .bits_per_sample =
47             I2S_BITS_PER_SAMPLE_16BIT, // is fixed at 12bit, stereo,
↪ MSB
48         .channel_format = I2S_CHANNEL_FMT_ALL_RIGHT,
49         #if ESP_IDF_VERSION > ESP_IDF_VERSION_VAL(4, 1, 0)
50         .communication_format = I2S_COMM_FORMAT_STAND_I2S,
51         #else
52         .communication_format = I2S_COMM_FORMAT_I2S,
53         #endif
54         .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
55         .dma_buf_count = 2,
56         .dma_buf_len = 128,
57     };
58
59
60
61     i2s_pin_config_t cfg_pin;
62
63     #if (ESP_IDF_VERSION > ESP_IDF_VERSION_VAL(4, 3, 0))
64         cfg_pin.mck_io_num = I2S_PIN_NO_CHANGE;
65     #endif
66
67     cfg_pin.bck_io_num = I2S_PIN_NO_CHANGE;
68     cfg_pin.ws_io_num = MIKE_PIN_CLK;
69     cfg_pin.data_out_num = I2S_PIN_NO_CHANGE;
70     cfg_pin.data_in_num = MIKE_PIN_DATA;
71
72     i2s_driver_install(I2S_NUM_0, &cfg_i2s, 0, NULL);

```

```

73     i2s_set_pin(I2S_NUM_0, &cfg_pin);
74     i2s_set_clk(I2S_NUM_0, sample_rate, I2S_BITS_PER_SAMPLE_16BIT,
↪     I2S_CHANNEL_MONO);
75 }
76
77 DataUnion* Mike::get_msg() {
78     current_msg = (current_msg + 1) % MIKE_QUEUE_LEN;
79     xSemaphoreTake(*get_current_semaphore(), portMAX_DELAY);
80     return (msg + current_msg);
81 }
82
83 SemaphoreHandle_t* Mike::get_current_semaphore() {
84     return (msg_mutex + current_msg);
85 }
86
87 void Mike::get_data(byte* data, byte* length) {
88     size_t bytes_read = 0;
89     i2s_read(I2S_NUM_0, (char *)data, MIKE_READ_LEN, &bytes_read,
↪     portMAX_DELAY);
90     // 2 bajts mean 1 sample
91     bytes_read = bytes_read / 2;
92     length[0] = bytes_read & 0xFF;
93     length[1] = (bytes_read >> 8) & 0xFF;
94 }
95
96 void mike_task(void *connection) {
97     Connection * conn = (Connection*)connection;
98     Mike mike = Mike();
99     mike.init();
100     DataUnion * msg = mike.get_msg();
101     byte *data_ptr = (msg->msg.bytes + HEADER_LEN);
102     TimeUnion timeUnion;
103     Msg mm = Msg();
104     float sample_interval = 1000000.0 / (float)mike.sample_rate;
105     float time_passed = 0;
106     gettimeofday(&timeUnion.tv_now, NULL);
107     timeUnion.tv_now.tv_sec += 7200;
108     while (uxSemaphoreGetCount(mutexMike) > 0) {
109         memcpy(msg->data.time, timeUnion.bytes.sec, 8);
110         mike.get_data(data_ptr, msg->data.count);
111         time_passed = (sample_interval * (float)(msg->data.count[0] +
↪     (msg->data.count[1] << 8)));
112         if (time_passed + timeUnion.tv_now.tv_usec > 1000000) {

```

```

113         timeUnion.tv_now.tv_sec += 1;
114         timeUnion.tv_now.tv_usec = time_passed +
↪ timeUnion.tv_now.tv_usec - 1000000;
115     } else {
116         timeUnion.tv_now.tv_usec += time_passed;
117     }
118     // Send data
119     mm.data = msg->msg.bytes;
120     mm.len = HEADER_LEN + ((msg->data.count[0] |
↪ (msg->data.count[1] << 8)) * 2);
121     mm.mutex = mike.get_current_semaphore();
122     conn->add_to_send_queue(&mm);
123     // Get pointer to next free message
124     msg = mike.get_msg();
125     data_ptr = (msg->msg.bytes + HEADER_LEN);
126 }
127 i2s_driver_uninstall(I2S_NUM_0);
128 sleep(50);
129 xSemaphoreGive(mutexMike);
130 while(true){}
131 }
132
133 bool start_Mike_task(Connection* connection) {
134     xSemaphoreGive(mutexMike);
135     xTaskCreatePinnedToCore(mike_task, "Mike", 8192, connection,
↪ configMAX_PRIORITIES - 1, &taskMike, pro_cpu);
136     return true;
137 }
138
139 bool stop_Mike_task() {
140     xSemaphoreTake(mutexMike, portMAX_DELAY);
141     xSemaphoreTake(mutexMike, portMAX_DELAY);
142     if (taskMike != NULL) vTaskDelete(taskMike);
143     return true;
144 }

```

A.9 mike.hpp

```

1 #ifndef _MIKE_
2 #define _MIKE_
3 #include "globals.hpp"

```

```
4  #include "link.hpp"
5
6  #define MIKE_QUEUE_LEN 4
7  #define MIKE_PIN_CLK 0
8  #define MIKE_PIN_DATA 34
9  #define MIKE_READ_LEN (MAX_MSG_LEN - 16)
10 #define MIKE_GAIN_FACTOR 3
11
12
13 struct SampleFreq {
14     const static byte Hz_22050 {0x00};
15     const static byte Hz_44100 {0x01};
16 };
17
18
19 class Mike {
20     public:
21         Mike();
22         void init();
23         DataUnion* get_msg();
24         SemaphoreHandle_t* get_current_semaphore();
25         void get_data(byte* data, byte* length);
26         static uint32_t sample_rate;
27         static bool set_parameters(byte* params, int* length);
28     private:
29         SemaphoreHandle_t msg_mutex[MIKE_QUEUE_LEN];
30         DataUnion msg[MIKE_QUEUE_LEN];
31         int current_msg;
32         int8_t buffer[MIKE_READ_LEN];
33         int16_t *adcBuffer;
34 };
35
36 static TaskHandle_t taskMike;
37
38 bool start_Mike_task(Connection* connection);
39 bool stop_Mike_task();
40 void Mike_task(void* connection);
41
42 #endif
```

A.10 platformio.ini

```

11 [env:m5stick-c]
12 platform = espressif32
13 board = m5stick-c
14 framework = arduino
15 monitor_speed = 115200
16 lib_deps = m5stack/M5StickCPlus@^0.0.7

```

A.11 test_tool.py

```

1  from enum import Enum
2  from functools import partial
3  from queue import Empty, Queue
4  from socket import (AF_INET, SHUT_RDWR, SO_REUSEADDR, SOCK_STREAM,
5  ↪ SOL_SOCKET,
6  ↪ socket, timeout)
7  from struct import unpack
8  from threading import Event, Thread
9  from tkinter import (BOTH, BOTTOM, LEFT, NSEW, RIGHT, TOP, Button,
10 ↪ Canvas,
11 ↪ Checkbutton, Entry, Frame, IntVar, Label,
12 ↪ Listbox, Menu,
13 ↪ OptionMenu, Radiobutton, Scale, Scrollbar,
14 ↪ Spinbox,
15 ↪ StringVar, Text, Tk, X, messagebox)
16 from typing import Dict, Tuple
17
18
19 from commands import Commands
20 from figure import Chart
21 from window import Window
22
23
24 def server(indicators: Dict[str, StringVar], send_queue: Queue,
25 ↪ receive_queue: Queue, stop_event: Event, chart_queue: Queue):
26     indicators["server_started"].set("Server Started")
27     soc = socket(AF_INET, SOCK_STREAM)
28     soc.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
29     soc.bind(("0.0.0.0", 7122))
30     soc.settimeout(1)
31     soc.listen()

```

```

26     while not stop_event.is_set():
27         try:
28             conn, addr = soc.accept()
29             print(f"Connected from {addr}")
30             client_thread = Thread(
31                 target=receiver, args=(indicators, conn, addr,
32                     ↪ send_queue, receive_queue, stop_event,
33                     ↪ chart_queue),
34                 daemon=True
35             )
36             client_thread.start()
37             client_thread.join()
38         except timeout:
39             pass
40     soc.shutdown(SHUT_RDWR)
41     soc.close()
42     indicators["server_started"].set("Server Stopped")
43
44 def sender(indicators: Dict[str, StringVar], conn: socket, send_queue:
45     ↪ Queue, stop_event: Event):
46     indicators["client_connected"].set("Client Connected")
47     while not stop_event.is_set():
48         try:
49             msg = send_queue.get(timeout=1)
50             print(f"Sending {msg}")
51             conn.send(msg)
52         except Empty:
53             pass
54     print("sender Closing socket")
55     try:
56         conn.shutdown(SHUT_RDWR)
57         conn.close()
58     except:
59         pass
60     indicators["client_connected"].set("Client Disconnected")
61
62 def receiver(indicators: Dict[str, StringVar], conn: socket, addr:
63     ↪ Tuple[str, int], send_queue: Queue, receive_queue: Queue,
64     ↪ stop_event: Event, chart_queue: Queue):
65     sender_task = Thread(
66         target=sender, args=(indicators, conn, send_queue,
67             ↪ stop_event),
68         daemon=True

```



```

63     )
64     manual_mode_task = Thread(
65         target=client_handler, args=(indicators, send_queue,
66         ↪ receive_queue, stop_event, chart_queue),
67         daemon=True
68     )
69     sender_task.start()
70     manual_mode_task.start()
71     while not stop_event.is_set():
72         try:
73             raw = conn.recv(8192)
74             if len(raw) < 10: print(f"Received {raw}")
75         except timeout:
76             continue
77         except:
78             break
79         receive_queue.put(raw)
80         if raw == b"":
81             break
82     print("receiver Closing socket")
83     try:
84         conn.shutdown(SHUT_RDWR)
85         conn.close()
86     except:
87         pass
88     manual_mode_task.join()
89     sender_task.join()
90     indicators["client_connected"].set("Client Disconnected")
91
92 def client_handler(indicators: Dict[str, StringVar], send_queue:
93 ↪ Queue, receive_queue: Queue, stop_event: Event, chart_queue:
94 ↪ Queue):
95     while not stop_event.is_set():
96         try:
97             raw = receive_queue.get(timeout=1)
98         except Empty:
99             continue
100         if raw[0] == 1:
101             print("Sending NEW_ID")
102             send_queue.put(Commands.NEW_ID.value)
103         elif raw[0] == 2:
104             print(f"Received ID: {raw[1]}")
105             send_queue.put(Commands.ID_OK.value)

```

```
103     elif raw[0] == 3 and raw[1] == 2:
104         length = unpack("<H", raw[14:16])[0]
105         print(length, raw[14:16])
106         mic = unpack('<' + 'h'*length, raw[16:length*2 + 16])
107         chart_queue.put(["mic", mic])
108     elif raw[0] == 3 and raw[1] == 1:
109         length = unpack("<H", raw[14:16])[0]
110         data = unpack('<' + 'f'*length*6, raw[16:length*24 + 16])
111         accel_x = []
112         accel_y = []
113         accel_z = []
114         gyro_x = []
115         gyro_y = []
116         gyro_z = []
117         for i, value in enumerate(data):
118             if i % 6 == 0:
119                 accel_x.append(value)
120             elif i % 6 == 1:
121                 accel_y.append(value)
122             elif i % 6 == 2:
123                 accel_z.append(value)
124             elif i % 6 == 3:
125                 gyro_x.append(value)
126             elif i % 6 == 4:
127                 gyro_y.append(value)
128             elif i % 6 == 5:
129                 gyro_z.append(value)
130         chart_queue.put([
131             ("accel_x", accel_x),
132             ("accel_y", accel_y),
133             ("accel_z", accel_z),
134             ("gyro_x", gyro_x),
135             ("gyro_y", gyro_y),
136             ("gyro_z", gyro_z)
137         ])
138
139     elif raw[0] == 4:
140         if raw[1] == 1:
141             print("Received SET OK")
142         elif raw[1] == 11:
143             print("Received SET FAIL")
144
145     elif raw[0] == 5:
```

```

146         if raw[1] == 1:
147             print("Received ON/OFF OK")
148         elif raw[1] == 9:
149             ↪ print("Received ON/OFF FAIL - MEASUREMENT ALREADY RUNNING")
150         elif raw[1] == 10:
151             ↪ print("Received ON/OFF FAIL - MEASUREMENT NOT RUNNING")
152         elif raw[1] == 11:
153             print("Received ON/OFF FAIL - WRONG COMMAND")
154
155     def command_loop(
156         indicators: Dict[str, StringVar],
157         command_queue: Queue,
158         send_queue: Queue,
159         receive_queue: Queue,
160         stop_event: Event,
161         stop_main_loop_event: Event,
162         chart_queue: Queue):
163         while not stop_main_loop_event.is_set():
164             try:
165                 command = command_queue.get(timeout=1)
166             except Empty:
167                 continue
168
169             if command == "start":
170                 if stop_event.is_set():
171                     stop_event.clear()
172                     Thread(
173                         target=server, args=(indicators, send_queue,
174                         ↪ receive_queue, stop_event, chart_queue),
175                         daemon=True
176                     ).start()
177
178             elif command == "stop":
179                 if not stop_event.is_set():
180                     stop_event.set()
181
182             elif command == "start mpu":
183                 if stop_event.is_set():
184                     print("Start server first")
185             else:

```

```

186         send_queue.put(Commands.START_STREAM_MPU.value)
187
188     elif command == "stop mpu":
189         if stop_event.is_set():
190             print("Start server first")
191         else:
192             send_queue.put(Commands.STOP_STREAM_MPU.value)
193
194     elif command == "start mike":
195         if stop_event.is_set():
196             print("Start server first")
197         else:
198             send_queue.put(Commands.START_STREAM_MIKE.value)
199
200     elif command == "stop mike":
201         if stop_event.is_set():
202             print("Start server first")
203         else:
204             send_queue.put(Commands.STOP_STREAM_MIKE.value)
205
206     elif command == "set mpu":
207         if stop_event.is_set():
208             print("Start server first")
209         else:
210             command = Commands.SET + Commands.MPU
211             if indicators['choose_dps_choice'].get() == "250 DPS":
212                 command += Commands.DPS_250
213             elif indicators['choose_dps_choice'].get() ==
214                 ↪ "500 DPS":
215                 command += Commands.DPS_500
216             elif indicators['choose_dps_choice'].get() ==
217                 ↪ "1000 DPS":
218                 command += Commands.DPS_1000
219             elif indicators['choose_dps_choice'].get() ==
220                 ↪ "2000 DPS":
221                 command += Commands.DPS_2000
222             if indicators['choose_g_choice'].get() == "2 G":
223                 command += Commands.G_2
224             elif indicators['choose_g_choice'].get() == "4 G":
225                 command += Commands.G_4
226             elif indicators['choose_g_choice'].get() == "8 G":
227                 command += Commands.G_8
228             elif indicators['choose_g_choice'].get() == "16 G":

```

```

226         command += Commands.G_16
227         send_queue.put(command)
228
229         elif command == "set mike":
230             if stop_event.is_set():
231                 print("Start server first")
232             else:
233                 command = Commands.SET + Commands.MIKE
234                 if indicators['choose_frequency_choice'].get() ==
235                     ↪ "22050 Hz":
236                     command += Commands.FREQ_22050
237                 elif indicators['choose_frequency_choice'].get() ==
238                     ↪ "44100 Hz":
239                     command += Commands.FREQ_44100
240                 send_queue.put(command)
241
242 def main():
243     root = Window()
244     root.title("Test")
245     root.geometry("800x600")
246
247     command_queue = Queue()
248     send_queue = Queue()
249     receive_queue = Queue()
250     stop_event = Event()
251     stop_main_loop_event = Event()
252
253     indicators = {
254         "server_started": StringVar(root, 'Server Stopped'),
255         "client_connected": StringVar(root, 'Client Disconnected'),
256         "choose_frequency_choice": StringVar(root, "22050 Hz"),
257         "choose_dps_choice": StringVar(root, "250 DPS"),
258         "choose_g_choice": StringVar(root, "2 G"),
259     }
260
261     control_frame = Frame(root)
262     control_frame.pack(side=TOP, fill=X)
263     control_frame2 = Frame(root)
264     control_frame2.pack(side=TOP, fill=X)
265     indicator_frame = Frame(root)
266     indicator_frame.pack(side=BOTTOM, fill=X)
267
268     start_button = Button(control_frame, text="Start Server",
269         ↪ command=partial(command_queue.put, "start"))

```

```

267 start_button.pack(side=LEFT)
268 stop_button = Button(control_frame, text="Stop Server",
    ↪ command=partial(command_queue.put, "stop"))
269 stop_button.pack(side=LEFT)
270
271 start_mpu_button = Button(control_frame, text="Start MPU",
    ↪ command=partial(command_queue.put, "start mpu"))
272 start_mpu_button.pack(side=LEFT)
273 stop_mpu_button = Button(control_frame, text="Stop MPU",
    ↪ command=partial(command_queue.put, "stop mpu"))
274 stop_mpu_button.pack(side=LEFT)
275
276 start_mike_button = Button(control_frame, text="Start Mike",
    ↪ command=partial(command_queue.put, "start mike"))
277 start_mike_button.pack(side=LEFT)
278 stop_mike_button = Button(control_frame, text="Stop Mike",
    ↪ command=partial(command_queue.put, "stop mike"))
279 stop_mike_button.pack(side=LEFT)
280
281 choose_dps_menu = OptionMenu(control_frame2,
    ↪ indicators["choose_dps_choice"], "250 DPS", "500 DPS",
    ↪ "1000 DPS", "2000 DPS")
282 choose_dps_menu.pack(side=LEFT)
283
284 choose_g_menu = OptionMenu(control_frame2,
    ↪ indicators["choose_g_choice"], "2 G", "4 G", "8 G", "16 G")
285 choose_g_menu.pack(side=LEFT)
286
287 set_mpu_button = Button(control_frame2, text="Set MPU",
    ↪ command=partial(command_queue.put, "set mpu"))
288 set_mpu_button.pack(side=LEFT)
289
290 choose_frequency_menu = OptionMenu(control_frame2,
    ↪ indicators["choose_frequency_choice"], "22050 Hz", "44100 Hz")
291 choose_frequency_menu.pack(side=RIGHT)
292
293 set_mike_button = Button(control_frame2, text="Set Mike",
    ↪ command=partial(command_queue.put, "set mike"))
294 set_mike_button.pack(side=RIGHT)
295
296 server_started_label = Label(indicator_frame, relief='solid',
    ↪ borderwidth=2, textvariable=indicators["server_started"],
    ↪ font=("Helvetica bold", 12))

```

```
297     server_started_label.pack(side=LEFT)
298     client_connected_label = Label(indicator_frame, relief='solid',
    ↪     borderwidth=2, textvariable=indicators["client_connected"],
    ↪     font=("Helvetica bold", 12))
299     client_connected_label.pack(side=LEFT)
300
301     chart_queue = Queue()
302     chart = Chart(root, chart_queue)
303     chart.pack(side=TOP, fill=X)
304     chart.start()
305     # We need to stop chart animation before root.destroy()
306     root.add_stop_function(chart.stop)
307
308     command_thread = Thread(
309         target=command_loop, args=(indicators, command_queue,
    ↪     send_queue, receive_queue, stop_event,
    ↪     stop_main_loop_event, chart_queue),
310     )
311     command_thread.start()
312     command_queue.put("stop")
313
314     root.mainloop()
315
316     if not stop_event.is_set():
317         stop_event.set()
318     stop_main_loop_event.set()
319     command_thread.join()
320
321
322 if __name__ == "__main__":
323     main()
```

Dodatek B

Wybrane fragmenty kodu źródłowego aplikacji serwera

B.1 exchange.go

```
1 package command
2
3 const defaultChannelBuffer = 10
4
5 type Exchange struct {
6     observers []chan *Command
7 }
8
9 func (e *Exchange) Dispatch(command *Command) {
10     for _, o := range e.observers {
11         o <- command
12     }
13 }
14
15 func (e *Exchange) GetSubscriberChannel() chan *Command {
16     ch := make(chan *Command, defaultChannelBuffer)
17     e.observers = append(e.observers, ch)
18     return ch
19 }
20
21 func (e *Exchange) Unsubscribe(subscriber chan *Command) {
22     for i := range e.observers {
23         if e.observers[i] == subscriber {
24             e.observers = append(e.observers[:i],
25 ↪ e.observers[i+1:]...)
26             return
27         }
28     }
29 }
```



```
26     }
27 }
28 }
29
30 func NewExchange() *Exchange {
31     return &Exchange{
32         make([]chan *Command, 0),
33     }
34 }
```

B.2 handshake.go

```
1 package handshake
2
3 import (
4     "fmt"
5     "io"
6     "server/database"
7     "server/socket/codes"
8 )
9
10 type Handler struct {
11     sensorRegister database.SensorRepository
12 }
13
14 const (
15     size = 2
16 )
17
18 func NewHandler(register database.SensorRepository) *Handler {
19     return &Handler{
20         register,
21     }
22 }
23
24 func (h *Handler) Handle(reader io.Reader) (id uint8, response []byte,
25     ↪ err error) {
26     buffer := make([]byte, size)
27     _, err = io.ReadFull(reader, buffer)
28     if err != nil {
29         return id, []byte{}, err
30     }
31 }
```

```

29     }
30     command, id := buffer[0], buffer[1]
31
32     switch command {
33     case codes.RequireAuth:
34         id, err = h.sensorRegister.ReserveNewId()
35         if err != nil {
36             return id, []byte{codes.NoIdAvailable, id},
↪ fmt.Errorf("handshake failed: %v", err)
37         }
38         return id, []byte{codes.Ok, id}, err
39     case codes.ImAuthorized:
40         if !h.sensorRegister.DoesIdExist(id) {
41             return id, []byte{codes.IdDoesntExist, id},
↪ fmt.Errorf("given id does not exist")
42         }
43         if h.sensorRegister.IsIdUsed(id) {
44             return id, []byte{codes.IdIsAlreadyConnected, id},
↪ fmt.Errorf("given id is already connected")
45         }
46         return id, []byte{codes.Ok, id}, err
47     }
48     return id, []byte{codes.HandshakeNotValid, id},
↪ fmt.Errorf("handshake is not valid")
49 }

```

B.3 interpreter.go

```

1 package interpreting
2
3 import (
4     "bufio"
5     "encoding/binary"
6     "errors"
7     "fmt"
8     "io"
9     "math"
10    "server/database"
11    "server/monitor"
12    "server/sensor"
13    "server/socket/codes"

```

```

14     "server/socket/translate"
15     "time"
16 )
17
18 type Interpreter interface {
19     Interpret() (Conclusion, error)
20     Code() byte
21 }
22
23 type interpreter struct {
24     input      *bufio.Reader
25     code       byte
26     readingRepo database.ReadingsRepository
27     id         uint8
28 }
29
30 const microsecondsInSecond = 1_000_000
31 const nanosecondsInMicrosecond = 1000
32
33 func NewInterpreter(inputStream *bufio.Reader, rr
34 ↪ database.ReadingsRepository, id uint8) Interpreter {
35     return &interpreter{
36         input:      inputStream,
37         code:        codes.Ok,
38         readingRepo: rr,
39         id:          id,
40     }
41 }
42
43 func (i *interpreter) Code() byte {
44     code := i.code
45     i.code = codes.Ok
46     return code
47 }
48
49 func (i *interpreter) Interpret() (c Conclusion, err error) {
50     _, err = i.readCommandCode()
51     if err != nil {
52         i.code = codes.CommandNotRecognized
53         return
54     }
55     p, err := i.readDataTransferHeader()
56     if err != nil {

```

```

56         i.code = codes.UnexpectedPacketHeader
57         return
58     }
59     readings, err := i.readDataTransferBody(p)
60     if err != nil {
61         i.code = codes.UnexpectedPacketBody
62     }
63     kind := monitor.IMU
64     if len(p.rt.Labels) == 1 {
65         kind = monitor.Mic
66     }
67     monitor.CreateDataStruct(len(readings), kind)
68     c = &readConclusion{
69         readings: readings,
70         repo:      i.readingRepo,
71         sensorId: i.id,
72     }
73     return
74 }
75
76 func (i *interpreter) readCommandCode() (code byte, err error) {
77     code, err = i.input.ReadByte()
78     if err != nil {
79         return
80     }
81     if code != codes.SendData {
82         err = fmt.Errorf("command code unknown: %v", code)
83     }
84     return
85 }
86
87 func (i *interpreter) readDataTransferHeader() (result *dataPackage,
88 ↪ err error) {
89     result = new(dataPackage)
90     readingType, err := i.readReadingType()
91     if err != nil {
92         return
93     }
94     seconds, err := i.readUint32()
95     if err != nil {
96         return
97     }
98     microseconds, err := i.readUint32()

```

```

98     if err != nil {
99         return
100    }
101    frequency, err := i.readFloat32()
102    if err != nil {
103        return
104    }
105    readsQuantity, err := i.readUint16()
106    if err != nil {
107        return
108    }
109    result.rt = readingType
110    result.sec = seconds
111    result.usec = microseconds
112    result.frequency = frequency
113    result.quantity = readsQuantity
114    return
115 }
116
117 func (i *interpreter) readReadingType() (rt sensor.ReadingType, err
↪ error) {
118     typeId, err := i.input.ReadByte()
119     if err != nil {
120         return
121     }
122     rt, exists := sensor.AvailableReadingTypes[typeId]
123     if !exists {
124         err = errors.New("such reading type does not exist")
125     }
126     return
127 }
128
129 func (i *interpreter) readDataTransferBody(p *dataPackage) (readings
↪ []*database.Reading, err error) {
130     translator := &translate.Translator{ReadingType: p.rt}
131     readingBuffer := make([]byte, p.rt.Bytes*p.rt.Count)
132     readings = make([]*database.Reading, 0, 256)
133     microsecondPeriod := microsecondsInSecond / p.frequency
134     for j := 0; j < int(p.quantity); j++ {
135         _, err = io.ReadFull(i.input, readingBuffer)
136         if err != nil {
137             return
138         }

```

```

139         copiedBuffer := make([]byte, len(readingBuffer))
140         copy(copiedBuffer, readingBuffer)
141
142         translated, err := translator.Translate(copiedBuffer)
143         if err != nil {
144             return nil, err
145         }
146
147         microsecondsMeasured := p.usec +
↪ uint32(microsecondPeriod*float32(j))
148         secondsMeasured := p.sec
149         for microsecondsMeasured >= microsecondsInSecond {
150             secondsMeasured += 1
151             microsecondsMeasured -= microsecondsInSecond
152         }
153
154         timestamp := time.Unix(int64(secondsMeasured),
↪ int64(microsecondsMeasured)*nanosecondsInMicrosecond).UTC()
155
156         readings = append(readings, &database.Reading{
157             Value:      translated,
158             Timestamp: timestamp,
159         })
160     }
161     return readings, nil
162 }
163
164 func (i *interpreter) readUint32() (result uint32, err error) {
165     buffer := make([]byte, 4)
166     _, err = io.ReadFull(i.input, buffer)
167     if err != nil {
168         return
169     }
170     result = binary.LittleEndian.Uint32(buffer)
171     return
172 }
173
174 func (i *interpreter) readFloat32() (result float32, err error) {
175     buffer := make([]byte, 4)
176     _, err = io.ReadFull(i.input, buffer)
177     if err != nil {
178         return
179     }

```

```

180     u32 := binary.LittleEndian.Uint32(buffer)
181     result = math.Float32frombits(u32)
182     return
183 }
184
185 func (i *interpreter) readUint16() (result uint16, err error) {
186     buffer := make([]byte, 2)
187     _, err = io.ReadFull(i.input, buffer)
188     if err != nil {
189         return
190     }
191     result = binary.LittleEndian.Uint16(buffer)
192     return
193 }

```

B.4 receiver.go

```

1  package receive
2
3  import (
4      "bufio"
5      "encoding/binary"
6      "io"
7      "server/monitor"
8      "server/sensor"
9      "time"
10 )
11
12 type ReceiverOutput struct {
13     Message []byte
14     Id      uint8
15 }
16
17 type Receiver interface {
18     Receive() error
19 }
20
21 type receiver struct {
22     conn          *bufio.Reader
23     interpreterQueue chan *ReceiverOutput
24     id            uint8

```

```

25     headerBuffer    []byte
26 }
27
28 const headerSize = 16
29 const microphoneReadSize = 2
30 const readsQuantityByte = headerSize - 2
31 const typeByte = 1
32
33 func (rec *receiver) Receive() error {
34     var err error
35     rec.headerBuffer, err = rec.conn.Peek(headerSize)
36     start := time.Now()
37     if err != nil {
38         return err
39     }
40     formatBytes :=
↪ sensor.AvailableReadingTypes[rec.headerBuffer[typeByte]].Bytes
41     formatCount :=
↪ sensor.AvailableReadingTypes[rec.headerBuffer[typeByte]].Count
42     readsQuantity :=
↪ int(binary.LittleEndian.Uint16(rec.headerBuffer[readsQuantityByte:]))
43     singleReadSize := formatBytes * formatCount
44     messageSize := singleReadSize * readsQuantity
45     packetBuffer := make([]byte, messageSize+headerSize)
46
47     _, err = io.ReadFull(rec.conn, packetBuffer)
48     if err != nil {
49         return err
50     }
51
52     monitor.ReceiveBytes(len(packetBuffer))
53     monitor.ExpectBytesWritten((8 + formatBytes + 1 + 1) * formatCount
↪ * readsQuantity)
54     kind := monitor.IMU
55     if singleReadSize == microphoneReadSize {
56         kind = monitor.Mic
57     }
58     monitor.ExpectDataStruct(readsQuantity, kind)
59     monitor.RegisterReceiveTime(time.Now().Sub(start))
60
61     rec.interpreterQueue <- &ReceiverOutput{
62         Id:      rec.id,
63         Message: packetBuffer,

```



```

64     }
65     return nil
66 }
67
68 func NewReceiver(reader io.Reader, queue chan *ReceiverOutput, id
↪ uint8) Receiver {
69     return &receiver{
70         conn:          bufio.NewReader(reader),
71         interpreterQueue: queue,
72         id:              id,
73         headerBuffer:    []byte{},
74     }
75 }

```

B.5 command.go

```

1  package serving
2
3  import (
4      "fmt"
5      "github.com/gin-gonic/gin"
6      "net/http"
7      "server/socket/codes"
8      "server/socket/command"
9  )
10
11 type Settings struct {
12     DPS      int `json:"dps"`
13     G        int `json:"g"`
14     Hz       int `json:"hz"`
15     SensorType int `json:"sensor_type"`
16 }
17
18 type CommandBody struct {
19     Id          uint8 `json:"id"`
20     Command     string `json:"command"`
21     SettingsDetails *Settings `json:"settings"`
22 }
23
24 const (
25     TurnOff      = "turn-off"

```

```
26     TurnOn      = "turn-on"
27     ImuSettings = "imu-settings"
28     MicSettings = "mic-settings"
29 )
30
31 var commandMappings = map[string]command.Command{
32     TurnOff: {
33         Command: codes.Toggle,
34         Arguments: []byte{codes.TurnOff},
35     },
36     TurnOn: {
37         Command: codes.Toggle,
38         Arguments: []byte{codes.TurnOn},
39     },
40     ImuSettings: {
41         Command: codes.Settings,
42         SensorType: codes.Imu,
43         Arguments: make([]byte, 2),
44     },
45     MicSettings: {
46         Command: codes.Settings,
47         SensorType: codes.Mic,
48         Arguments: make([]byte, 1),
49     },
50 }
51
52 var gMapping = map[int]byte{
53     2: codes.Imu2G,
54     4: codes.Imu4G,
55     8: codes.Imu8G,
56     16: codes.Imu16G,
57 }
58
59 var dpsMapping = map[int]byte{
60     250: codes.Imu250DPS,
61     500: codes.Imu500DPS,
62     1000: codes.Imu1000DPS,
63     2000: codes.Imu2000DPS,
64 }
65
66 var hzMapping = map[int]byte{
67     22050: codes.Mic22050Hz,
68     44100: codes.Mic44100Hz,
```

```

69 }
70
71 func Command(c *gin.Context) {
72     var json CommandBody
73     if err := c.ShouldBindJSON(&json); err != nil {
74         c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
75         return
76     }
77
78     err := dispatchCommand(&json)
79     if err != nil {
80         c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
81         return
82     }
83
84     c.JSON(http.StatusOK, gin.H{"status": "ok"})
85 }
86
87 func dispatchCommand(cb *CommandBody) error {
88     c, ok := commandMappings[cb.Command]
89     if !ok {
90         return fmt.Errorf("command '%v' not found", cb.Command)
91     }
92     if !sensorRegister.DoesIdExist(cb.Id) {
93         return fmt.Errorf("id %v not found", cb.Id)
94     }
95     err := fillWithSettingsDetails(&c, cb)
96     if err != nil {
97         return err
98     }
99     c.ForId = cb.Id
100    exchange.Dispatch(&c)
101    return nil
102 }
103
104 func fillWithSettingsDetails(c *command.Command, cb *CommandBody)
↪ error {
105     if c.Command == codes.Toggle {
106         if cb.SettingsDetails.SensorType == codes.Undefined ||
↪ cb.SettingsDetails.SensorType > codes.Mic {
107             return fmt.Errorf("unknown sensor type code: %v",
↪ cb.SettingsDetails.SensorType)
108         }

```

```

109     c.SensorType = byte(cb.SettingsDetails.SensorType)
110     return nil
111 }
112
113 if c.Command != codes.Settings { // every code requires arguments
114     return fmt.Errorf("unknown command: %v", c.Command)
115 }
116
117 if cb.SettingsDetails.SensorType == codes.Mic {
118     hz, ok := hzMapping[cb.SettingsDetails.Hz]
119     if !ok {
120         return fmt.Errorf("unknown Hz value: %v",
↪ cb.SettingsDetails.Hz)
121     }
122     c.Arguments[0] = hz
123     return nil
124 }
125
126 // sensor must be imu
127 dps, ok := dpsMapping[cb.SettingsDetails.DPS]
128 if !ok {
129     return fmt.Errorf("got unknown DPS value: %v",
↪ cb.SettingsDetails.DPS)
130 }
131 g, ok := gMapping[cb.SettingsDetails.G]
132 if !ok {
133     return fmt.Errorf("got unknown G value: %v",
↪ cb.SettingsDetails.G)
134 }
135 c.Arguments[0] = dps
136 c.Arguments[1] = g
137 return nil
138 }

```

B.6 readings.go

```

1 package database
2
3 import (
4     "encoding/binary"
5     "fmt"

```

```

6      "os"
7      "server/monitor"
8      "server/sensor"
9      "sync"
10     "time"
11 )
12
13 type ReadingsRepository interface {
14     SaveReadings(id uint8, readings []*Reading) error
15     Close() error
16 }
17
18 type binaryReadingsRepository struct {
19     fileNumber int
20     file       *os.File
21     saveLock   *sync.RWMutex
22 }
23
24 const gb = 1_000_000_000
25 const fileSizeLimit = 1 * gb
26 const fileSavingPath = "/data/"
27 const databaseTimeFormat = "2006-01-02 15:04:05.000000"
28 const savingThreads = 5
29 const waitForDataPeriod = time.Millisecond * 100
30
31 var processQueue = make(chan struct {
32     readings []*Reading
33     id       uint8
34 }, 1000)
35 var binaryDataQueue = make(chan []byte, 1000)
36
37 func (r *binaryReadingsRepository) SaveReadings(id uint8, readings
↵  []*Reading) (err error) {
38     processQueue <- struct {
39         readings []*Reading
40         id       uint8
41     }{readings: readings, id: id}
42
43     kind := monitor.IMU
44     _, isMic := readings[0].Value[sensor.Volume]
45     if isMic {
46         kind = monitor.Mic
47     }

```

```

48     monitor.SendDataStruct(len(readings), kind)
49
50     return nil
51 }
52
53 func (r *binaryReadingsRepository) byteifyReadings() {
54     pack := <-processQueue
55     readings := pack.readings
56     id := pack.id
57     monitor.ProcessQueueLen(len(processQueue))
58
59     kind := monitor.IMU
60     _, isMic := readings[0].Value[sensor.Volume]
61     if isMic {
62         kind = monitor.Mic
63     }
64     monitor.ReceiveDataStruct(len(readings), kind)
65
66     start := time.Now()
67     binaryData := r.obtainBinary(id, readings)
68     monitor.RegisterParseTime(time.Now().Sub(start))
69     binaryDataQueue <- binaryData
70 }
71
72 func (r *binaryReadingsRepository) saveBinary() {
73     for {
74         savingBuffer := 1000
75         binaryData := make([]byte, 0, savingBuffer)
76         for i := 0; i < savingBuffer; i++ {
77             if len(binaryDataQueue) == 0 && len(binaryData) != 0 {
78                 time.Sleep(waitForDataPeriod)
79                 if len(binaryDataQueue) == 0 {
80                     break
81                 }
82             }
83             fetchedData := <-binaryDataQueue
84             binaryData = append(binaryData, fetchedData...)
85         }
86         monitor.ByteBufferLen(len(binaryData))
87         start := time.Now()
88         r.saveLock.Lock()
89         err := r.saveToFile(binaryData)
90         if err != nil {

```

```

91         fmt.Printf("couldn't save to file: %v\n", err)
92     }
93     if r.currentFileSize() > fileSizeLimit {
94         err = r.changeFile()
95         if err != nil {
96             fmt.Printf("couldn't change file: %v\n", err)
97         }
98     }
99     r.saveLock.Unlock()
100    monitor.RegisterFileSavingTime(time.Now().Sub(start))
101 }
102 }
103
104 func (r *binaryReadingsRepository) Close() error {
105     err := r.file.Close()
106     if err != nil {
107         return err
108     }
109     return nil
110 }
111
112 func (r *binaryReadingsRepository) currentFileSize() int64 {
113     stat, _ := r.file.Stat()
114     return stat.Size()
115 }
116
117 func (r *binaryReadingsRepository) changeFile() error {
118     err := r.Close()
119     if err != nil {
120         return err
121     }
122     err = r.openNewFile()
123     if err != nil {
124         return err
125     }
126     return nil
127 }
128
129 func (r *binaryReadingsRepository) obtainBinary(id uint8, readings
↪ []*Reading) []byte {
130     result := make([]byte, 0, 84*len(readings))
131     for _, reading := range readings {
132         for readType, value := range reading.Value {

```

```

133         timeBinary := make([]byte, 8)
134         binary.LittleEndian.PutUint64(timeBinary,
↪ uint64(reading.Timestamp.UnixMicro()))
135         subresult := append([]byte{id, readType}, value...)
136         subresult = append(subresult, timeBinary...)
137         result = append(result, subresult...)
138     }
139 }
140 return result
141 }
142
143 func (r *binaryReadingsRepository) openNewFile() (err error) {
144     r.file, err = os.Create(fileSavingPath +
↪ fmt.Sprintf("%06d_values", r.fileNumber))
145     if err != nil {
146         return err
147     }
148     r.fileNumber += 1
149     return nil
150 }
151
152 func (r *binaryReadingsRepository) saveToFile(data []byte) (err error)
↪ {
153     _, err = r.file.Write(data)
154     if err != nil {
155         return fmt.Errorf("couldn't write row %v to file: %v", data,
↪ err)
156     }
157     monitor.WriteBytes(len(data))
158     return nil
159 }
160
161 type Reading struct {
162     Value      map[uint8] []byte `json:"value"`
163     Timestamp  time.Time         `json:"timestamp"`
164 }
165
166 func (r *Reading) String() string {
167     return fmt.Sprintf("%v: %v",
↪ r.Timestamp.Format(databaseTimeFormat), r.Value)
168 }
169
170 func NewReadingsRepository() ReadingsRepository {

```



```
171     result := &binaryReadingsRepository{
172         saveLock: new(sync.RWMutex),
173     }
174     err := result.openNewFile()
175     if err != nil {
176         panic(fmt.Sprintf("couldn't open file for db: %v", err))
177     }
178     for i := 0; i < savingThreads; i++ {
179         go func() {
180             for {
181                 result.byteifyReadings()
182             }
183         }()
184     }
185     go result.saveBinary()
186     return result
187 }
```

B.7 main.go

```
1 package main
2
3 import (
4     "fmt"
5     "github.com/joho/godotenv"
6     "github.com/sirupsen/logrus"
7     "os"
8     "server/database"
9     "server/monitor"
10    "server/serving"
11    "server/socket"
12    "server/socket/command"
13    "server/socket/handshake"
14 )
15
16 const (
17     ListenPortEnvKey    = "LISTEN_PORT"
18     ServePortEnvKey     = "SERVE_PORT"
19     MonitoringEnvKey    = "ENABLE_MONITORING"
20     MonitoringPathEnvKey = "MONITORING_PATH"
21 )
```

```
22
23 func main() {
24     prod := os.Getenv("DONT_LOAD_DOTENV")
25     if prod != "1" {
26         if err := godotenv.Load(".env"); err != nil {
27             panic("Could not load .env file. Does it exist?")
28         }
29     }
30     logger := logrus.New()
31     logger.SetLevel(logrus.DebugLevel)
32     logger.SetFormatter(&logrus.TextFormatter{
33         FullTimestamp: true,
34         TimestampFormat: "15:04:05.999",
35     })
36     sensorRegister := database.NewSensorRepository()
37     handshakeHandler := handshake.NewHandler(
38         sensorRegister,
39     )
40     listenPort := os.Getenv(ListenPortEnvKey)
41     servePort := os.Getenv(ServePortEnvKey)
42     readingRepository := database.NewReadingsRepository()
43     defer readingRepository.Close()
44     commandExchange := command.NewExchange()
45
46     listener := socket.NewListener(logger, sensorRegister,
↪ handshakeHandler, listenPort, readingRepository, commandExchange)
47     go listener.Listen()
48
49     if os.Getenv(MonitoringEnvKey) == "1" {
50         monitor.FilePath = os.Getenv(MonitoringPathEnvKey)
51         monitor.RunMonitoring()
52     }
53
54     router := serving.SetupRouter(logger, sensorRegister,
↪ commandExchange)
55     err := router.Run(fmt.Sprintf(":%v", servePort))
56     if err != nil {
57         logger.Fatalf("Couldn't run server: %v", err)
58     }
59 }
```

B.8 listen.go

```
1 package socket
2
3 import (
4     "bufio"
5     "bytes"
6     "fmt"
7     "github.com/sirupsen/logrus"
8     "io"
9     "net"
10    "server/database"
11    "server/monitor"
12    "server/sensor"
13    "server/socket/command"
14    "server/socket/handshake"
15    "server/socket/interpreting"
16    "server/socket/receive"
17    "time"
18 )
19
20 const interpreterThreads = 5
21
22 type listener struct {
23     logger      *logrus.Logger
24     sr          database.SensorRepository
25     hh          *handshake.Handler
26     port        string
27     rr          database.ReadingsRepository
28     interpreterQueue chan *receive.ReceiverOutput
29     ce          *command.Exchange
30 }
31
32 type connectionHandler struct {
33     conn          net.Conn
34     logger        *logrus.Entry
35     handshakeHandler *handshake.Handler
36     sensorRepository database.SensorRepository
37     readingsRepository database.ReadingsRepository
38     interpreterQueue chan *receive.ReceiverOutput
39     commandExchange *command.Exchange
40     commandRoutineKillSwitch chan bool
41     connectionAbortSignal chan bool
```

```
42 }
43
44 func (l *listener) Listen() {
45     var err error
46     defer func() {
47         if err != nil {
48             l.logger.WithFields(map[string]interface{}{
49                 "error": err,
50             }).Error("Error occurred while listening")
51         }
52     }()
53
54     l.createInterpreterThreads()
55
56     netListener, err := net.Listen("tcp", fmt.Sprintf(":%v", l.port))
57     if err != nil {
58         return
59     }
60     defer func() {
61         l.logger.Debug("Closing connection now")
62         netListener.Close()
63     }()
64     l.logger.WithFields(map[string]interface{}{
65         "port": l.port,
66     }).Debug("Created TCP server")
67
68     for {
69         conn, err := netListener.Accept()
70         if err != nil {
71             l.logger.WithFields(map[string]interface{}{
72                 "error": err,
73             }).Error("Could not read incoming connection")
74             continue
75         }
76         handler := l.connHandler(conn)
77         go handler.handleConnection()
78     }
79 }
80
81 func (l *listener) connHandler(conn net.Conn) *connectionHandler {
82     logger := l.logger.WithFields(map[string]interface{}{
83         "address": conn.RemoteAddr(),
84     })
```

```

85     return &connectionHandler{
86         conn:                conn,
87         logger:               logger,
88         handshakeHandler:     l.hh,
89         sensorRepository:     l.sr,
90         readingsRepository:    l.rr,
91         interpreterQueue:      l.interpreterQueue,
92         commandExchange:       l.ce,
93         commandRoutineKillSwitch: make(chan bool),
94     }
95 }
96
97 func (l *listener) createInterpreterThreads() {
98     for i := 0; i < interpreterThreads; i++ {
99         go l.interpret()
100     }
101 }
102
103 func (l *listener) interpret() {
104     for {
105         message := <-l.interpreterQueue
106         start := time.Now()
107         monitor.InterpreterQueueLength(len(l.interpreterQueue))
108         monitor.SendBytes(len(message.Message))
109         interpreter := interpreting.NewInterpreter(
110             bufio.NewReader(bytes.NewBuffer(message.Message)), l.rr,
111             ↪ message.Id)
112         interpreted, err := interpreter.Interpret()
113         if err != nil {
114             l.logger.Debugf("Failed to interpret message: %v\n", err)
115             continue
116         }
117         err = interpreted.Act()
118         if err != nil {
119             l.logger.Debugf("failed to act on interpreted message: %v\n",
120                 ↪ err)
121         }
122         monitor.RegisterInterpretingTime(time.Now().Sub(start))
123     }
124 }
125
126 func (ch *connectionHandler) handleConnection() {
127     var err error

```

```

126     ch.logger.Debug("Opened new connection")
127     defer func() {
128         ch.logger.Debug("Closing connection")
129         _ = ch.conn.Close()
130     }()
131
132     var id uint8
133     var handshakeResponse []byte
134
135     for {
136         id, handshakeResponse, err =
↪ ch.handshakeHandler.Handle(ch.conn)
137         if err == nil {
138             break
139         }
140         ch.logger.Errorf("error occured while handling handshake: %v\n",
↪ err)
141         if err == io.EOF {
142             return
143         }
144         _, err = ch.conn.Write(handshakeResponse)
145         if err != nil {
146             ch.logger.Errorf(
147                 "error occured while trying to write to connection: %v\n",
↪ err)
148             return
149         }
150     }
151     sensorObject := ch.useSensor(id)
152     go ch.startCommandRoutine(id)
153     defer func() {
154         ch.unuseSensor(sensorObject)
155         ch.commandRoutineKillSwitch <- true
156     }()
157     _, err = ch.conn.Write(handshakeResponse)
158     if err != nil {
159         ch.logger.Errorf("error while responding to handshake: %v\n",
↪ err)
160     }
161
162     receiver := ch.buildReceiver(id)
163     for {
164         select {

```

```

165         case <-ch.connectionAbortSignal:
166             return
167         default:
168             err = receiver.Receive()
169             if err != nil {
170                 ch.logger.Errorf("error occured while receiving data: %v\n",
↪ err)
171                 if err == io.EOF || err == io.ErrUnexpectedEOF {
172                     return
173                 }
174             }
175         }
176     }
177 }
178
179 func (ch *connectionHandler) addLoggerData(key string, value
↪ interface{}) {
180     ch.logger = ch.logger.WithFields(map[string]interface{}{
181         key: value,
182     })
183 }
184
185 func (ch *connectionHandler) useSensor(id uint8) *sensor.Sensor {
186     sensorObject, _ := ch.sensorRepository.GetById(id)
187     _ = ch.sensorRepository.MarkSensorAsBeingUsed(id)
188     ch.addLoggerData("id", id)
189     ch.logger.Debug("Established connection with sensor")
190     return sensorObject
191 }
192
193 func (ch *connectionHandler) unuseSensor(s *sensor.Sensor) {
194     _ = ch.sensorRepository.MarkSensorAsUnused(s.ID)
195     ch.logger.Debug("Connection with sensor has been broken")
196 }
197
198 func (ch *connectionHandler) buildReceiver(id uint8) receive.Receiver
↪ {
199     return receive.NewReceiver(ch.conn, ch.interpreterQueue, id)
200 }
201
202 func (ch *connectionHandler) startCommandRoutine(id uint8) {
203     sub := ch.commandExchange.GetSubscriberChannel()
204     defer ch.commandExchange.Unsubscribe(sub)

```

```
205     for {
206         select {
207             case <-ch.commandRoutineKillSwitch:
208                 return
209             case comm := <-sub:
210                 if id != comm.ForId {
211                     continue
212                 }
213                 _, err := ch.conn.Write(append([]byte{comm.Command,
↵  comm.SensorType}, comm.Arguments...))
214                 if err != nil {
215                     ch.logger.Errorf("error while writing command: %v",
↵  err)
216                     ch.connectionAbortSignal <- true
217                 }
218             }
219         }
220     }
```


Dodatek C

Skrypty użyte podczas testów systemu

C.1 splitter.py

```
1  from os import listdir, mkdir
2  from os.path import join
3  from sys import argv
4  from typing import List, Tuple
5
6  import pandas as pd
7
8  from settings import OriginalNames, get_sensor
9
10 input_folder = argv[1]
11 output_folder = argv[2]
12 ext = ".csv"
13
14
15 def get_files() -> Tuple[List[str], OriginalNames]:
16     return ([join(input_folder, file) for file in
17             ↪ listdir(input_folder) if file.endswith(ext)],
18             [file for file in listdir(input_folder) if
19             ↪ file.endswith(ext)])
20
21 def main():
22     if not output_folder in listdir(): mkdir(output_folder)
23     for file, name in zip(*get_files()):
24         file = pd.read_csv(file)
25         file["sensor"] = file["name"].apply(get_sensor)
26         for (sensor, id), file in file.groupby(["sensor",
27         ↪ "sensor_id"]):
```

```
25         file.sort_values(by="time").to_csv(join(output_folder,  
26             ↪ f"{sensor}_{id}_{name}"), index=False)  
27  
28 if __name__ == "__main__":  
29     main()
```

C.2 union.py

```
1  from sys import argv  
2  
3  import pandas as pd  
4  
5  input_file_path1 = argv[1]  
6  input_file_path2 = argv[2]  
7  output_file_path = argv[3]  
8  
9  def main():  
10     df1 = pd.read_csv(input_file_path1)  
11     df2 = pd.read_csv(input_file_path2)  
12     df = pd.concat([df1, df2])  
13     df.sort_values(by="time").to_csv(output_file_path, index=False)  
14  
15 if __name__ == "__main__":  
16     main()
```

C.3 printer.py

```
1  from datetime import datetime, timedelta  
2  from os import listdir, mkdir  
3  from os.path import join  
4  from sys import argv  
5  
6  import matplotlib.dates as mdates  
7  import pandas as pd  
8  from matplotlib import pyplot as plt  
9  
10 in_path = argv[1]  
11 out_path = argv[2]
```

```

12 dateformat = "%H:%M:%S.%f"
13
14 unit_mapping = {
15     "x-g": "x [g]",
16     "y-g": "y [g]",
17     "z-g": "z [g]",
18     "x-dps": "x [DPS]",
19     "y-dps": "y [DPS]",
20     "z-dps": "z [DPS]",
21     "volume": "amplitude",
22     "amplitude": "amplitude",
23 }
24
25 def main():
26     if out_path not in listdir(): mkdir(out_path)
27     files = listdir(in_path)
28     names = [file.split('.')[0] + '.png' for file in files]
29     names = [join(out_path, name) for name in names]
30     files = [join(in_path, file) for file in files if
31         ↪ file.endswith('.csv')]
32     for name, file in zip(names, files):
33         data = pd.read_csv(file, parse_dates=['time'])
34         data.sort_values(by='time', inplace=True)
35         fig, ax = plt.subplots()
36         labels = []
37         for (id, unit), df in data.groupby(["sensor_id", "name"]):
38             labels.append(f"{id} {unit_mapping[unit]}")
39             df.plot(ax=ax, x="time", y="value", figsize=(15, 10),
40                 ↪ fontsize=15, alpha=0.4)
41         plt.legend(labels)
42         ax.set_xticks([], minor=False)
43         ax.xaxis.set_minor_locator(mdates.AutoDateLocator())
44         ax.xaxis.set_minor_formatter(mdates.DateFormatter(dateformat))
45         plt.xticks(rotation=45, minor=True)
46         if len(labels) > 15: ax.get_legend().remove()
47         plt.tight_layout()
48         # plt.show()
49         plt.savefig(name, bbox_inches='tight')
50         plt.close()
51
52 def hist():
53     if out_path not in listdir(): mkdir(out_path)
54     files = listdir(in_path)

```

```

53     names = [file.split('.')[0] + '.png' for file in files]
54     names = [join(out_path, name) for name in names]
55     files = [join(in_path, file) for file in files if
56         ↪ file.endswith('.csv')]
57     for name, file in zip(names, files):
58         data = pd.read_csv(file, parse_dates=['time'])
59         data.sort_values(by='time', inplace=True)
60         fig, ax = plt.subplots()
61         labels = []
62         for (id, unit), df in data.groupby(["sensor_id", "name"]):
63             labels.append(f"{id} {unit_mapping[unit]}")
64             df.sort_values(by='time', inplace=True)
65             diff = (df['time'] - df['time'].shift(1))[1:]
66             print(diff)
67             (diff[diff < timedelta(milliseconds=10)]).apply(lambda x:
68                 ↪ x.total_seconds()).plot.hist(bins=12, ax=ax,
69                 ↪ alpha=0.4)
70             print((diff[diff < timedelta(milliseconds=10)]).max(),
71                 ↪ diff.max())
72         plt.legend(labels)
73         ax.set_xticks([], minor=False)
74         ax.xaxis.set_minor_locator(mdates.AutoDateLocator())
75         ax.xaxis.set_minor_formatter(mdates.DateFormatter(dateformat))
76         plt.xticks(rotation=45, minor=True)
77         if len(labels) > 15: ax.get_legend().remove()
78         plt.tight_layout()
79         plt.savefig(name, bbox_inches='tight')
80         plt.close()
81
82 if __name__ == "__main__":
83     main()

```

C.4 cut_by_time.py

```

1 from datetime import datetime
2 from os import listdir, mkdir
3 from os.path import join
4 from sys import argv
5 from typing import List, Tuple
6
7 exec(f"from {argv[1].split('.')[0]} import FILES, time_format")

```

```

8
9 import pandas as pd
10
11 def main(files: List[Tuple[str, str, str, str]]):
12     for start_time, end_time, input_file_path, output_folder,
13         ↪ output_file in files:
14         if output_folder not in listdir(): mkdir(output_folder)
15         start_time = datetime.strptime(start_time, time_format)
16         end_time = datetime.strptime(end_time, time_format)
17         data = pd.read_csv(input_file_path, parse_dates=['time'])
18         data = data[(data['time'] >= start_time) & (data['time'] <
19             ↪ end_time)]
20         data.to_csv(join(output_folder, output_file), index=False)
21
22 if __name__ == "__main__":
23     main(FILEES)

```

C.5 time_cut_receipt.py

```

1 FILES = [ # start_time, end_time, input_file_path, output_folder,
2     ↪ output_file
3     (
4         "2022-10-12 18:29:09.500000",
5         "2022-10-12 18:29:20.500000",
6         "input/mic.csv",
7         "cutted", "impulses_closeup.csv"
8     ),
9     (
10        "2022-10-12 18:29:09.500000",
11        "2022-10-12 18:29:20.500000",
12        "splitted/mergedY.csv",
13        "cutted", "edge_closeup.csv"
14    ),
15 ]
16
17 time_format = "%Y-%m-%d %H:%M:%S.%f"

```

C.6 final.py

```
1 from os import listdir, mkdir
2 from os.path import join
3 from sys import argv
4 from typing import List, Tuple
5
6 import pandas as pd
7
8
9 def time_stability():
10     pd.set_option("display.precision", 8)
11     input_folder = "splitted"
12     output_folder = "time_stability"
13     if output_folder not in listdir(): mkdir(output_folder)
14     imu_files = {
15         "title": "IMU",
16         1:[
17             "ImuDPSY_0_kula.csv",
18         ],
19         2:[
20             "ImuDPSY_0_kula.csv",
21             "ImuDPSY_1_kula.csv",
22         ],
23         3:[
24             "ImuDPSY_0_kula.csv",
25             "ImuDPSY_1_kula.csv",
26             "ImuDPSY_2_kula.csv",
27         ],
28         4:[
29             "ImuDPSY_0_kula.csv",
30             "ImuDPSY_1_kula.csv",
31             "ImuDPSY_2_kula.csv",
32             "ImuDPSY_3_kula.csv",
33         ]
34     }
35     mic_files = {
36         "title": "MIC",
37         1:[
38             "Mic_0_mic.csv",
39         ],
40         2:[
41             "Mic_0_mic.csv",
```

```

42         "Mic_1_mic.csv",
43     ],
44     3:[
45         "Mic_0_mic.csv",
46         "Mic_1_mic.csv",
47         "Mic_2_mic.csv",
48     ],
49     4:[
50         "Mic_0_mic.csv",
51         "Mic_1_mic.csv",
52         "Mic_2_mic.csv",
53         "Mic_3_mic.csv",
54     ]
55 }
56 results = []
57 for i in range(1,5):
58     results.append( f"\nIMU {i} nodes statistics:\n" +
59         claculate_time_stability(imu_files[i], input_folder)
60     )
61     results.append( f"\nMicrophone {i} nodes statistics:\n" +
62         claculate_time_stability(mic_files[i], input_folder)
63     )
64     with open(join(output_folder, "time_stability.txt"), "w") as f:
65         f.write("\n".join(results))
66
67 def claculate_time_stability(files: List[str], input_folder: str):
68     files = [join(input_folder, f) for f in files]
69     files = [pd.read_csv(f, parse_dates=["time"]) for f in files]
70     delta_files = []
71     for file in files:
72         file.sort_values(by="time", inplace=True)
73         file["delta_time"] = file["time"].diff()
74         file.drop(0, inplace=True)
75         delta_files.append(file)
76
77     joined = pd.concat(delta_files)
78     return str(joined["delta_time"].describe())
79
80 def time_sync():
81     files = [
82         "ImuDPSZ_0_kula.csv",
83         "ImuDPSZ_1_kula.csv",
84         "ImuDPSZ_2_kula.csv",

```

```

85         "ImuDPSZ_3_kula.csv",
86     ]
87     files = [pd.read_csv(join('splitted', f), parse_dates=["time"])
88         ↪ for f in files]
89     maxes = [file["value"].max() for file in files]
90     mins = [file["value"].min() for file in files]
91     files = [file[file!=maxes[0]] for file in files]
92     files = [file[file!=mins[0]] for file in files]
93     files = [file[file.value.notnull()] for file in files]
94     points = [file.iloc[[0, -1]] for file in files]
95     first_point = pd.concat([point[:1] for point in points])
96     second_point = pd.concat([point[-1:] for point in points])
97     first_diff = []
98     second_diff = []
99     for i, time1 in enumerate(first_point["time"]):
100         first_diff.extend([
101             (i, j, abs(time2 - time1).total_seconds()*1_000)
102             ↪ for j, time2 in enumerate(first_point["time"])
103         ])
104     for i, time1 in enumerate(second_point["time"]):
105         second_diff.extend([
106             (i, j, abs(time2 - time1).total_seconds()*1_000)
107             ↪ for j, time2 in enumerate(second_point["time"])
108         ])
109     fdf = pd.DataFrame(first_diff, columns=["sensor1", "sensor2",
110         ↪ "diff"])
111     sdf = pd.DataFrame(second_diff, columns=["sensor1", "sensor2",
112         ↪ "diff"])
113     fdf = fdf[df["sensor1"] != df["sensor2"]]
114     sdf = sdf[sdf["sensor1"] != sdf["sensor2"]]
115     print("Points:")
116     print('\n'.join([
117         ↪ f'sensor: {i}, first: {first_point["time"].iloc[i]}, us: {second_point
118         ↪ for i in range(4)
119     ]))
120     print("First point:")
121     print('\n'.join([
122         ↪ f'{sensor1} {sensor2} {diff:0.3f} ms'
123         ↪ for sensor1, sensor2, diff in first_diff
124     ]))
125     print("Second point:")
126     print('\n'.join([

```



```

124         f'{sensor1} {sensor2} {diff:0.3f} ms'
125         for sensor1, sensor2, diff in second_diff
126     ]))
127     print("First point statistics:")
128     print(fdf.describe())
129     print("Second point statistics:")
130     print(sdf.describe())
131     print("Full statistics:")
132     print(pd.concat([fdf, sdf]).describe())
133
134
135 def main():
136     time_stability()
137     # time_sync()
138
139 if __name__ == "__main__":
140     main()

```

C.7 wav_creator.py

```

1 import pandas as pd
2 import wavio
3
4 if __name__ == '__main__':
5     data = pd.read_csv("input/first_test.csv", parse_dates=["time"])
6     data.sort_values(["time"], inplace=True)
7     volumes = data.loc[data['sensor_id'] == 0, ['value']].to_numpy()
8     samplerate = 44100
9     wavio.write("sound.wav", volumes, samplerate, sampwidth=2)

```

C.8 mic_synchro.py

```

1 from datetime import datetime
2
3 import pandas as pd
4
5 if __name__ == "__main__":
6     time_format = "%Y-%m-%d %H:%M:%S.%f"
7     impulseTimes = [

```

```

8         ("2022-10-12 18:29:10.000000", "2022-10-12 18:29:10.200000"),
9         ("2022-10-12 18:29:10.800000", "2022-10-12 18:29:11.000000"),
10        ("2022-10-12 18:29:11.800000", "2022-10-12 18:29:12.000000"),
11        ("2022-10-12 18:29:12.800000", "2022-10-12 18:29:13.000000"),
12        ("2022-10-12 18:29:13.800000", "2022-10-12 18:29:14.000000"),
13        ("2022-10-12 18:29:14.800000", "2022-10-12 18:29:15.000000"),
14        ("2022-10-12 18:29:15.800000", "2022-10-12 18:29:16.000000"),
15        ("2022-10-12 18:29:16.800000", "2022-10-12 18:29:17.000000"),
16        ("2022-10-12 18:29:17.800000", "2022-10-12 18:29:18.000000"),
17        ("2022-10-12 18:29:18.800000", "2022-10-12 18:29:19.000000"),
18        ("2022-10-12 18:29:19.800000", "2022-10-12 18:29:20.000000"),
19    ]
20
21    inputFilePath = "input/mic.csv"
22    data = pd.read_csv(inputFilePath, parse_dates=["time"])
23    data.sort_values(["time"], inplace=True)
24
25    impulsesData = []
26    for impulseTime in impulseTimes:
27        impulseData = data[(data['time'] >= impulseTime[0]) &
28                           → (data['time'] <= impulseTime[1])]
29        impulsesData.append(impulseData)
30
31    sensorIds = range(0, 4, 1)
32    for impulseIndex, impulseData in enumerate(impulsesData):
33        times = []
34        for sensorId in sensorIds:
35            time = impulseData[(impulseData['sensor_id'] == sensorId)
36                               → & (impulseData['value'].abs() >= 10000)].iloc[0].time
37            times.append(time)
38            print(f'First read over 10000 volume '
39                  f'for sensor {sensorId} in '
40                  f'impulse nr {impulseIndex}: {time}')
41        timesPd = pd.DataFrame(times).sort_values(0)
42        greatestDifference = timesPd.iloc[3] - timesPd.iloc[0]
43        smallestDifference = timesPd.iloc[2] - timesPd.iloc[1]
44        print(f'Greatest difference '
45              f'for this impulse '
46              f'is {greatestDifference.dt.microseconds}')
47        print(f'Smallest difference '
48              f'for this impulse '
49              f'is {smallestDifference.dt.microseconds}')

```

C.9 roll_synchro.py

```

1  from datetime import datetime
2
3  import pandas as pd
4
5  if __name__ == "__main__":
6      time_format = "%Y-%m-%d %H:%M:%S.%f"
7      edgeTimes = [
8          ("2022-10-04 20:37:36.400000", "2022-10-04 20:37:36.500000"),
9          ("2022-10-04 20:37:38.400000", "2022-10-04 20:37:38.500000"),
10         ("2022-10-04 20:37:40.820000", "2022-10-04 20:37:40.900000"),
11         ("2022-10-04 20:37:42.000000", "2022-10-04 20:37:42.200000"),
12         ("2022-10-04 20:37:43.300000", "2022-10-04 20:37:43.500000"),
13         ("2022-10-04 20:37:44.200000", "2022-10-04 20:37:44.400000"),
14         ("2022-10-04 20:37:45.600000", "2022-10-04 20:37:45.800000"),
15         ("2022-10-04 20:37:46.700000", "2022-10-04 20:37:46.900000"),
16         ("2022-10-04 20:37:48.000000", "2022-10-04 20:37:48.100000"),
17         ("2022-10-04 20:37:49.100000", "2022-10-04 20:37:49.300000"),
18         ("2022-10-04 20:37:50.200000", "2022-10-04 20:37:50.400000"),
19     ]
20     inputFilePath = "splitted/mergedY.csv"
21     data = pd.read_csv(inputFilePath, parse_dates=["time"])
22     data.sort_values(["time"], inplace=True)
23
24     edgesData = []
25     for edgeTime in edgeTimes:
26         edgeData = data[(data['time'] >= edgeTime[0]) & (data['time']
27             ↪ <= edgeTime[1])]
28         edgesData.append(edgeData)
29
30     sensorIds = range(0, 4, 1)
31     for edgeIndex, edgeData in enumerate(edgesData):
32         times = []
33         for sensorId in sensorIds:
34             time = edgeData[(edgeData['sensor_id'] == sensorId) &
35                 (
36                     (edgeData['value'] == 247.90308) |
37                     (edgeData['value'] == -249.99237)
38                 )].iloc[0].time
39             times.append(time)
40         print(f'First stauration '
41             f'for sensor {sensorId} in ')

```

```
41         f'edge nr {edgeIndex}: {time}')
```

```
42     timesPd = pd.DataFrame(times).sort_values(0)
```

```
43     greatestDifference = timesPd.iloc[3] - timesPd.iloc[0]
```

```
44     smallestDifference = timesPd.iloc[2] - timesPd.iloc[1]
```

```
45     print(f'Greatest difference '
```

```
46           f'for this edge '
```

```
47           f'is {greatestDifference.dt.microseconds}')
```

```
48     print(f'Smallest difference '
```

```
49           f'for this edge '
```

```
50           f'is {smallestDifference.dt.microseconds}')
```